

Python によるデータ解析と高速化

Ichiro KOMAE

2026 年 4 月 12 日

目次

はじめに	11
第 1 章 このテキストの使い方	13
1.1 誰のためのテキストか	13
1.2 何を順に学ぶのか	13
1.3 本書で大切にしたい姿勢	14
1.4 章の読み進め方	14
1.5 併用するとよい公開資料	14
1.6 この本の後半で扱う実践的な内容	15
第 2 章 シェルとファイル操作	17
2.1 シェルの基本と環境	17
2.1.1 Windows では WSL を使う	17
2.2 コマンドの基本と使い方	17
2.2.1 コマンド、オプション、引数	17
2.2.2 困ったときは説明を読む	18
2.3 ファイルとディレクトリの操作	18
2.3.1 今どこにいるかを確認する	18
2.3.2 移動する	19
2.3.3 パスを読む	19
2.3.4 ディレクトリを作る	20
2.4 コピー、移動、削除	20
2.4.1 コピーする: cp	20
2.4.2 移動する、名前を変える: mv	21
2.4.3 削除する: rm	21
2.4.4 ワイルドカード	22
2.5 ファイルの内容確認とデータ抽出	22
2.5.1 cat	22
2.5.2 less	23
2.5.3 head	23
2.5.4 tail	24
2.5.5 wc	24
2.5.6 grep	24
2.5.7 awk	25

2.5.8	パイプ	25
2.6	環境変数と PATH	26
2.6.1	環境変数を使う	26
2.6.2	展開と引用	27
2.6.3	PATH と実行ファイル	28
2.7	リダイレクトとログ保存	28
2.7.1	echo、標準出力、標準エラー	28
2.7.2	> と >>	29
2.7.3	1> と 2>	29
2.7.4	tee で見ながら保存する	29
2.8	権限とスクリプトの実行	30
2.8.1	権限を読む	30
2.8.2	スクリプトを直接実行する	31
第 3 章	Python 環境を整える	33
3.1	Python の確認と管理方針	33
3.1.1	何を分けて考えるべきか	33
3.1.2	今の Python を確認する	33
3.2	pyenv による複数バージョンの管理	33
3.2.1	macOS での準備	33
3.2.2	WSL / Ubuntu での準備	34
3.2.3	pyenv を入れる	34
3.2.4	pyenv をシェルへ組み込む	34
3.2.5	バージョンを入れて切り替える	34
3.3	venv でプロジェクトごとの仮想環境を作る	35
3.3.1	pip と python -m pip	36
3.4	解析用ライブラリと JupyterLab	36
3.4.1	JupyterLab を併用する	36
3.5	プロジェクト管理ツール uv の利用	37
3.5.1	プロジェクトの初期化	37
3.5.2	依存関係の追加と環境の同期	38
3.5.3	コマンドの実行とバージョン固定	38
3.5.4	標準ライブラリと外部ライブラリ	38
3.5.5	解析用のライブラリを入れる	39
3.6	引数・設定ファイルとコードの分離	39
3.6.1	コマンドライン引数と設定ファイルを使い分ける	39
3.6.2	設定ファイルとマジックナンバーの排除	40
第 4 章	Python の基本	41

4.1	データ型と基本操作	41
4.1.1	変数と基本型	41
4.1.2	数値と演算子	41
4.1.3	文字列	42
4.2	データ構造	42
4.2.1	リスト、タプル、辞書	42
4.2.2	リストのメソッド	42
4.2.3	インデックスとスライス	43
4.3	制御構文	43
4.3.1	条件分岐	43
4.3.2	反復処理	44
4.4	関数とモジュール	45
4.4.1	関数を分ける理由	45
4.5	保守と実践的な書き方	47
4.5.1	エラーは失敗ではなく情報である	47
第5章	入出力とファイル	51
5.1	テキストファイルの入出力と探索	51
5.1.1	ファイルを開き、読み書きの基本を押さえる	51
5.2	文字列データの解析と整形	52
5.2.1	文字列を値へ変換し、結果を書き出す	52
5.3	オブジェクトと構造化データの保存	54
5.3.1	JSON と Pickle / PKL	54
5.4	発展的なトピック	55
5.4.1	日付と時刻を数値として扱う	55
5.4.2	保存形式について: ベストプラクティスへのステップ	55
第6章	NumPy による数値計算	57
6.1	NumPy が扱う考え方	57
6.1.1	配列計算の基本概念	57
6.1.2	ndarray と属性	57
6.1.3	NumPy が速くなりやすい理由	58
6.2	最小限のコード例	58
6.2.1	配列を作る	58
6.2.2	よく使う操作	59
6.2.3	要素ごとの演算とユニバーサル関数	60
6.2.4	集計と axis	60
6.2.5	添字、スライス、条件抽出	61
6.2.6	形を変える	62

6.2.7	配列をつなぐ、分ける	62
6.2.8	形とブロードキャスト	63
6.2.9	統計量とヒストグラム	64
6.3	よくある落とし穴	64
6.3.1	演算子の意味を取り違える	64
6.3.2	axis と条件式	65
6.3.3	dtype と np.empty	65
6.3.4	コピーとビュー	66
6.3.5	浮動小数点数の比較	66
6.4	解析へのつながり	67
6.4.1	解析で NumPy を使うときの視点	67
6.4.2	差分、ソート、ヒストグラム	67
6.4.3	線形代数の入口	67
6.4.4	公式資料の使い方	69
第 7 章	pandas による大規模データ解析	71
7.1	DataFrame の考え方	71
7.1.1	列選択、並べ替え、集約	71
7.1.2	pandas が速くなりやすい理由	72
7.2	読み込みから書き出しまでの基本操作	72
7.2.1	CSV を毎回読み直さず Parquet へ変換する	73
7.2.2	欠損値と型の扱い	73
7.2.3	結合と時系列処理	74
7.2.4	DataFrame を TeX で出力する	74
7.3	行処理と並列化の判断	75
7.3.1	改善ステップ	75
7.4	さらに大規模なデータへの対応	77
第 8 章	Matplotlib による可視化	79
8.1	基本的な作図と可視化	79
8.1.1	最小限のヒストグラム	79
8.1.2	軸ラベル、凡例、目盛り	79
8.1.3	plot 関数へ渡すデータ	80
8.2	実践的な作図（データに応じた図の選択）	81
8.2.1	相関の可視化：論理的改善ステップ	81
8.2.2	複数の図を並べる (subplots)	83
8.2.3	凡例とレイアウト	84
8.2.4	エラーバーと対数軸 (log-log プロット)	85
8.2.5	結果例の読み方	86

8.2.6	画像フォーマットの選択：ラスタとベクタ	86
8.3	オブジェクト指向 API による複雑な図の管理	87
8.3.1	Figure, Axes, Axis, Artist の役割	87
8.3.2	オブジェクト指向 API で描く	91
8.3.3	補助関数で作図をまとめる	91
8.4	見た目とスタイルの管理	92
8.4.1	rcParams で見た目をそろえる	92
8.4.2	設定ファイル (.mplstyle) で見た目を共通化する	93
8.4.3	共通スタイルの一部だけを上書きする	94
8.5	比較しやすい図を作る	94
8.5.1	Matplotlib の数式表記を使う	94
8.6	よくある落とし穴	95
8.6.1	plt と ax を混ぜて迷子になる	95
8.6.2	文字列がカテゴリ軸になる	95
8.6.3	savefig と余白調整の順序	95
8.6.4	公式資料から似た図を探す	96
第 9 章	SciPy とフィット	97
9.1	フィットの考え方	97
9.1.1	この本でよく使う分布	97
9.1.2	SciPy の使い方	97
9.2	Astropy で補う時刻と単位	98
9.2.1	Astropy で単位を扱う	99
9.3	フィットを進める手順	99
9.3.1	初期値と制約	100
9.3.2	残差とフィット範囲	100
9.4	重み付きフィット	101
9.4.1	誤差をどう重みに反映するか	101
第 10 章	Python 解析の高速化	103
10.1	まず測る	103
10.1.1	Jupyter では %timeit も使える	103
10.2	計算量の見積もり	103
10.2.1	オーダーとは何か	104
10.2.2	オーダーが改善する例: 重複判定	105
10.2.3	オーダーは同じで定数倍だけ改善する例	106
10.3	実装を軽くする	107
10.3.1	NumPy と pandas による高速化	107
10.3.2	メモリとコピーの管理	107

10.3.3	分割読み込み、並列化、ストリーミング	107
10.4	プロファイリングと低レベル実装	108
10.4.1	プロファイラを使う	108
10.4.2	C や C++ へ書き換えるという選択肢	108
10.4.3	Python から C/C++ を呼び出す	109
10.4.4	外部実装を参考にする時の見方	109
10.5	結果例と報告の仕方	110
第 11 章	課題: 同時観測イベントの探索	113
11.1	課題の準備	113
11.1.1	まず配布コードを動かす	113
11.1.2	gzip ファイルの中身を見る	113
11.1.3	小さいデータで確認する	114
11.2	課題で求めること	114
11.2.1	出力としてほしいもの	115
11.2.2	正しさ確認の目安	115
11.3	提出物と発展課題	116
11.3.1	発展課題	116
付録 A	SSH, 鍵認証, ファイル転送	117
A.1	SSH の基本	117
A.1.1	サーバーへ接続する	117
A.1.2	SSH 鍵を作る	117
A.1.3	公開鍵をサーバーへ登録する	118
A.1.4	ssh-agent を使う	118
A.2	接続設定と GitHub 連携	118
A.2.1	~/.ssh/config を書く	118
A.2.2	GitHub に SSH 鍵を登録する	119
A.3	ファイル転送	119
A.3.1	scp の最小例	119
A.3.2	rsync を推奨する理由	119
付録 B	Git と GitHub の基本	121
B.1	Git の基本概念	121
B.1.1	リポジトリと公開範囲	121
B.1.2	Git で何をしているのか	121
B.2	ローカルで履歴を作る	122
B.2.1	最初の設定	122
B.2.2	新しいリポジトリを作る	122
B.2.3	編集、確認、記録の流れ	122

B.2.4	.gitignore を使って除外する	123
B.3	GitHub と接続する	124
B.3.1	既存のリポジトリを持ってくる	124
B.3.2	手元のリポジトリを GitHub へ初めて載せる	124
B.3.3	GitHub と同期する	126
B.3.4	GitHub で使う URL には HTTPS と SSH がある	126
B.4	認証と最小ワークフロー	126
B.4.1	GitHub の認証: PAT と SSH 鍵	126
B.4.2	初心者が押さえるべき最小ワークフロー	127
付録 C	正規表現の基本	129
C.1	正規表現とは何か	129
C.1.1	文字どおりの一致とメタ文字	129
C.1.2	ワイルドカードとの違い	129
C.2	よく使う部品	130
C.2.1	アンカー (行頭と行末)	130
C.2.2	文字クラス [...] と代表的な省略記法	130
C.2.3	量指定子 *, +, ?	131
C.3	物理・データ解析での具体例	131
C.3.1	ログから必要な行だけを探す	131
C.3.2	検出器 ID やセンサー名を選別する	131
C.3.3	Python で抽出と変換を同時に行う	132
C.4	Python の re と grep/awk の使い分け	132
C.4.1	Python では raw string を使う	132
C.4.2	search, match, fullmatch を使い分ける	132
C.4.3	コマンドラインで十分な場面と、Python に移すべき場面	133

はじめに

このテキストは、Python を使ってデータを読み、整形し、図を描き、分布を調べるための入門書である。単に文法を覚えるのではなく、結果の正しさを自分で確かめ、処理時間を測り、改善点を説明できるようになることを目標にする。

本書の構成は Python 公式チュートリアルの流れを意識している。前半では、変数、リスト、関数、ファイル読み書きといった基礎を確認する。中盤では NumPy、pandas、Matplotlib、SciPy、Astropy を用いた実践的な解析へ進む。後半では、大規模データの高速化や低レベル言語との連携に触れ、最後に総合演習へ進む。

特に、Python 公式チュートリアルの「An Informal Introduction to Python」「More Control Flow Tools」「Data Structures」「Modules」「Input and Output」「Errors and Exceptions」「Brief Tour of the Standard Library」で扱われている基礎事項を、データ解析の文脈でつなぎ直すことを意識している。公式チュートリアルを置き換えるというより、その内容を土台にして、解析実装で本当に困る点まで掘り下げることを目指す。

コマンド例は、macOS 上のターミナルで作業する状況を自然な基準として書いている。Terminal.app でも iTerm2 でも構わない。Windows ユーザーは PowerShell を用いてもよいが、Windows Subsystem for Linux (WSL) の使用を強く推奨する。

第 1 章 このテキストの使い方

1.1 誰のためのテキストか

このテキストは、次のような読者を想定している。

- Python をこれから学びたい人
- 多少は書けるが、解析コードをどう整理すればよいか分からない人
- 小さなデータでは動くが、大きなデータになると急に遅くなる理由を知りたい人
- 結果の図をそれらしく描くことはできても、正しさの確認やフィットの説明に自信がない人

逆に、すでに Python で大規模解析を日常的に行っており、NumPy や pandas の内部挙動にも慣れている読者にとっては、前半の内容は基礎的に見えるだろう。その場合は、後半の高速化や総合演習から読んでも構わない。

本書は、単に「使い方を覚える」ためだけの資料ではない。研究や課題で自力の解析を組み立てるとき、何を順に確かめればよいか、どの段階で道具を変えるべきかまで含めて学ぶことを目指している。したがって、短いサンプルコードであっても、なぜその書き方になっているのかを説明する。

1.2 何を順に学ぶのか

本書では、大きく分けて三段階で学ぶ。

第一段階では、Python の基本的な書き方と、ファイルを読むための最低限の技術を扱う。ここでは、短いスクリプトを自分で動かし、何が入力で何が出力なのかをはっきり意識する。

第二段階では、NumPy、pandas、Matplotlib、SciPy を使い、実際の解析に必要な表現力を身につける。配列、表形式データ、ヒストグラム、フィットといった道具が、この段階の中心になる。

第三段階では、高速化と設計を学ぶ。遅い実装と速い実装を比べ、どこで時間を使っているのかを測り、アルゴリズムの違いが速度にどう響くのかを考える。この延長として、Python だけでなく C や C++ へ処理を移す発想も扱う。

この順序には理由がある。NumPy や pandas を早く使いたくなるのは自然だが、入力と出力、型、反復、条件分岐が曖昧なままでは、便利なライブラリを使っても何が起きているか分からなくなる。基礎を先に固めるのは遠回りではなく、後半を理解する最短経路である。

1.3 本書で大切にしたい姿勢

本書で繰り返し強調するのは、次の三点である。

- 結果をそのまま信じず、自分で確かめること
- 遅いと感じたら、どこが遅いかを測ること
- コードが動く理由と、速くなる理由を言葉で説明すること

特にデータ解析では、コードが実行できたことと、結果が正しいことは同じではない。さらに、正しいコードであっても、データ量が増えた途端に実用にならなくなることがある。本書では、その両方を区別して考える。

もう一つ大切なのは、結果を言葉に直すことである。たとえば「NumPy を使ったら速くなった」と書くだけでは不十分で、どの操作を Python ループから配列演算へ移したのか、どの程度速くなったのか、メモリ使用量はどう変わったのか、といった説明が必要になる。レポートや口頭説明では、この差がそのまま理解の差として現れる。

1.4 章の読み進め方

各章には、できるだけ次の順序で内容を置く。

1. その章で扱う考え方の意味
2. 最小限のコード例
3. よくある落とし穴
4. 解析へのつながり

最初は、コードを丸ごと理解しようとしなくてよい。まずは「何を入力し、何を出しているか」を追う。その後で「なぜこの書き方を選んだか」を読むと、理解しやすい。

特に初学者は、1 行ずつ読んで意味を取ろうとして全体像を見失いがちである。まずは関数名、引数、返り値、ファイル名、出力先だけを追い、流れをつかんでから細部へ戻る方が理解しやすい。本書のコード例も、その読み方を意識して配置している。

1.5 併用するとよい公開資料

本書は独立して読めるように書いているが、コマンドの細かいオプションやライブラリの仕様は版によって変わることがある。そのため、必要に応じて公式の公開資料も併用した方がよい。本書の記述も、以下のような公開資料の内容を踏まえて整理している。

- Python 全般: [Python 公式チュートリアル](#) と [Python 標準ライブラリ](#)
- JupyterLab: [JupyterLab Documentation](#)

- uv: [uv Documentation](#)
- pyenv: [pyenv README / Documentation](#)
- NumPy: [NumPy Documentation](#)
- pandas: [pandas Documentation](#)
- Matplotlib: [Matplotlib Documentation](#)
- SciPy: [SciPy Documentation](#)
- Astropy: [Astropy Documentation](#)
- Git: [Git Documentation](#)
- GitHub: [GitHub Docs](#)

特にシェルの `ssh`、`rsync`、`ls`、`chmod` などは、環境ごとに細かな違いがある。細部を確認したいときは `man ssh` や `man rsync` のようにマニュアルを開く習慣も有効である。

1.6 この本の後半で扱う実践的な内容

本書の後半では、単なる文法紹介ではなく、実際のデータを扱うときに何が問題になるかを扱う。そこでは、正しさの確認、バグの切り分け、図の作成、フィット、速度改善といった要素が一つの流れとして現れる。

最後の章には具体的な総合演習を置くが、それは本書で扱った一般的な考え方をまとめて使うための実践例という位置付けである。したがって、前半の章では特定の演習に閉じず、より広いデータ解析へそのまま応用できる書き方と考え方を重視する。

言い換えると、後半へ行くほど「何を書くか」よりも「どう考えるか」が主題になる。ライブラリの関数名は将来変わることがあるが、入力を確認する、分布を見る、速度を測る、遅い理由を切り分ける、といった考え方は長く残る。

第 2 章 シェルとファイル操作

2.1 シェルの基本と環境

本書では、ファイル操作やスクリプト実行をターミナルから行う。まず言葉を分けておく。

- ターミナル: 文字で命令を入力するためのアプリケーション
- シェル: ターミナルの中で動き、入力されたコマンドを解釈して実行するプログラム

macOS では `zsh`、Linux や WSL では `bash` を使うことが多い。`zsh` と `bash` はどちらもシェルであり、本書の基本コマンドはほぼ同じように使える。自分がどのシェルを使っているかを調べるには、次のように実行する。

```
$ echo $SHELL
```

本書では、シェルで入力するコマンド例の先頭に `$` を付ける。`$` 自体は「ここから先が入力するコマンドである」という印であり、読者が実際に打ち込むのはその後ろの部分である。これに対して、出力結果やファイル内容の例には `$` を付けない。

2.1.1 Windows では WSL を使う

本書のコマンド例は macOS や Linux のシェルを前提にしている。そのため、Windows で読み進める場合は、コマンドプロンプトや PowerShell をそのまま使うより、WSL (Windows Subsystem for Linux) を入れて Ubuntu などの Linux 環境を使う方が混乱しにくい。導入の際は次のように実行する。

```
$ wsl --install
```

WSL を使う利点は、単にコマンド名がそろうことだけではない。後半で扱う並列処理や周辺ツールの中には、Linux 系環境での利用を前提としているものがある。OS ごとの差でつまづくより、最初から UNIX 系の作法へ寄せた方が、解析そのものに集中しやすい。

2.2 コマンドの基本と使い方

2.2.1 コマンド、オプション、引数

シェルで打つ 1 行は、多くの場合「コマンド本体」「オプション」「引数」から成る。たとえば次のような形である。

```
$ ls -l results
$ cp data/raw.txt backup/raw.txt
```

1 行目では、`ls` がコマンド本体、`-l` がオプション、`results` が引数である。2 行目では、`cp` がコマンド本体で、`data/raw.txt` と `backup/raw.txt` が引数である。一般に、オプションは動作を変え、引数は対象のファイル名やディレクトリ名を与える。

オプションには短い書き方（`-` の後に 1 文字）と、長い書き方（`--` の後に英単語）が用意されていることが多い。たとえば、使い方を表示するオプションとしては `-h` と `--help` の両方がサポートされていることが多い。

2.2.2 困ったときは説明を読む

コマンドを丸暗記する必要はない。忘れたときは、その場で説明を読む方が正確である。短い使い方を知りたいときは `-h` や `--help`、より詳しい説明を読みたいときは `man` を使う。実際の調べ方は次のようになる。

```
$ python3 -h
$ python3 --help
$ man rsync
```

`--help` や `-h` は、そのコマンドが受け付けるオプションや引数の書式を短く表示することが多い。ただし、すべてのコマンドが両方を実装しているわけではない。macOS の `ls` のように `--help` を受け付けないコマンドもあるので、その場合は `man ls`、`man ssh`、`man rsync` のように `man` を引く方が確実である。

`man` を開いたあとは、上下キーや `Space` で移動し、`/pattern` で検索できる。`q` を押せば終了する。最初のうちは「使い方を忘れたら検索エンジン」ではなく、「まず `--help` か `man`」という順序にした方が、手元の環境に合った説明へすぐたどり着ける。

2.3 ファイルとディレクトリの操作

2.3.1 今どこにいるかを確認する

シェルでは、いま作業しているディレクトリが常に基準になる。これをカレントディレクトリという。現在の位置（パス）やそこにあるファイルの一覧を確認するには、次のように打つ。

```
$ pwd
$ ls
$ ls -l
$ ls -a
```

`pwd` は現在位置を表示する。`ls` は一覧を表示する。`ls -l` は詳細表示、`ls -a` は隠しファイルも含めて表示する。ファイル名やディレクトリ名が `.`（ドット）で始まるものは隠し

ファイルとして扱われ、通常の `ls` では表示されない。`.gitignore` や `.bashrc` などがこれに該当する。ここで `pwd` の結果として、次のように表示されたでしょう。

```
/Users/ikomae/analysis
```

もし `pwd` が上のように表示されていれば、今いる場所は `/Users/ikomae/analysis` である。その状態で `ls` を実行すると、そのディレクトリの中身が表示される。

2.3.2 移動する

`cd` はディレクトリを移動するためのコマンドである。次のようなディレクトリ構成を例に考える。

```
analysis/  
|-- data/  
|  |-- events.txt  
|-- scripts/  
     |-- plot.py
```

いま `/Users/ikomae/analysis` にいるとする。このとき、次の 4 つの `cd` の意味はそれぞれ違う。

```
$ cd data  
$ cd ..  
$ cd ~/analysis/scripts  
$ cd -
```

`cd data` は `analysis/data` へ入る。`cd ..` は 1 つ上のディレクトリへ戻る。`cd ~/analysis/scripts` はホームディレクトリを基準に `scripts` へ移動する。`cd -` は 1 つ前にいた場所へ戻る。

2.3.3 パスを読む

ファイルやディレクトリの場所は、パスで表す。パスには絶対パスと相対パスがある。

表 2.1 よく出てくるパス表記

表記	意味
<code>/abs/data.txt</code>	絶対パス。どこにいても同じ場所を指す
<code>~/analysis</code>	絶対パス。~ はホームディレクトリ
<code>data.txt</code>	相対パス。今いるディレクトリを基準に解釈される
<code>./x.txt</code>	相対パス。・ は現在のディレクトリ
<code>../x.txt</code>	相対パス。.. は 1 つ上のディレクトリ

たとえば、現在位置が `/abs/analysis` なら、次の 2 つのコマンドは同じファイルを指している。

```
$ cat data/events.txt
$ cat /abs/analysis/data/events.txt
```

一方で、現在位置が `/abs` なら、`cat data/events.txt` は別の意味になる。したがって、コマンドを読むときは「今どこにいるか」と「そのパスが絶対か相対か」を必ずセットで考える必要がある。

2.3.4 ディレクトリを作る

`mkdir` はディレクトリを作る。`-p` を付けると、親ディレクトリがなくてもまとめて作れる。

```
$ mkdir output
$ mkdir -p output/2026/04
```

1 行目は `output` だけを作る。2 行目は `output` や `output/2026` がまだ無くても、必要な親ディレクトリごと作る。解析では日付ごとの出力先を作ることが多いので、`mkdir -p` はよく使う。

2.4 コピー、移動、削除

2.4.1 コピーする: `cp`

`cp` は元のファイルやディレクトリを残したまま複製を作る。たとえば次のようなディレクトリ構造があるとする。

```
work/
|-- backup/
`-- raw/
    |-- events.txt
```

ここで、`raw` ディレクトリ内のファイルを `backup` へコピーするには次のように実行する。

```
$ cp raw/events.txt backup/events.txt
```

実行すると、ディレクトリ構造は次のようになる。

```
work/
|-- backup/
|   |-- events.txt
|-- raw/
    |-- events.txt
```

コピー後も `raw/events.txt` は残る。`backup/events.txt` が新しく増えるだけである。これが `cp` の基本である。

ディレクトリごと複製したい場合は `-r` を付ける。

```
$ cp -r raw raw_backup
```

2.4.2 移動する、名前を変える: mv

mv は「移動」と「改名」の両方に使う。どちらになるかは、最後の引数が何を指しているかで決まる。

```
work/  
|-- result.txt
```

この状態から、ファイル名を変更するには次のように実行する。

```
$ mv result.txt result_old.txt
```

実行後、ディレクトリ内は次のようになる。

```
work/  
|-- result_old.txt
```

この場合は、同じディレクトリの中で名前だけが変わる。

次に、ファイルを別のディレクトリへ移動することを考えよう。work ディレクトリの下に output ディレクトリと result.txt というファイルがあるとする。

```
work/  
|-- output/  
|-- result.txt
```

ここで、result.txt を output ディレクトリへ移動するには次のようにすればよい。

```
$ mv result.txt output/
```

実行後、ディレクトリ構成は次のようになる。

```
work/  
|-- output/  
    |-- result.txt
```

この場合は、result.txt が output/ の中へ移る。元の場所からは消える。cp とは異なり、元ファイルは残らない。

2.4.3 削除する: rm

rm はファイルを削除する。重要なのは、通常の rm はゴミ箱へ移すのではなく、その場で完全に削除するという点である。いくつか例を挙げる。

```
$ rm result_old.txt  
$ rm -i result_old.txt  
$ rm -r old_output
```

rm result_old.txt はファイルを削除する。rm -i result_old.txt は削除前に確認を出す。rm -r old_output はディレクトリを中身ごと再帰的に削除する。

特に `rm -r` は強力である。対象のパスを誤ると大きな被害になるので、削除前に `pwd` と `ls` で場所を確かめる習慣を持った方がよい。

2.4.4 ワイルドカード

複数のファイルを一度に指定したいときは、ワイルドカードが役に立つ。よく使うのは `*` であり、これは「0 文字以上の任意の文字列」を表す。重要なのは、`cat` や `rm` が `*` を理解しているのではなく、シェルが先に展開してから各コマンドへ渡しているという点である。

```
$ cp data/*.csv backup/  
$ rm test_*.csv
```

`cp data/*.csv backup/` とすると複数の CSV をまとめてコピーできる。`rm test_*.csv` なら条件に合うファイルを一括削除できる。

したがって、ワイルドカードを使うときは、実際に何が展開されるかを意識した方がよい。削除系のコマンドでは、先に `ls test_*.csv` のように確認してから実行する方が安全である。

2.5 ファイルの内容確認とデータ抽出

解析では、データを読む前にまず中身を確認する。そのための基本コマンドと、最低限覚えておきたいオプションを最初に表へまとめる。

表 2.2 中身確認と抽出でよく使うコマンドと主要オプション

コマンド	主なオプションや書式	主な用途
<code>cat</code>	<code>-n</code>	短いファイルをそのまま読む、複数ファイルを連結する
<code>less</code>	<code>-N</code> , <code>/pattern</code> , <code>q</code>	長いファイルをスクロールしながら読む
<code>head</code>	<code>-n N</code>	先頭の数行だけを見る
<code>tail</code>	<code>-n N</code> , <code>-f</code>	末尾の数行を見る、ログの追記を監視する
<code>wc</code>	<code>-l</code> , <code>-w</code> , <code>-c</code>	行数、単語数、バイト数を数える
<code>grep</code>	<code>-i</code> , <code>-v</code> , <code>-n</code> , <code>-A N</code> , <code>-B N</code>	条件に合う行だけを抽出する
<code>awk</code>	<code>-F</code> , <code>if (...)</code> , <code>sum += ...</code>	列単位で取り出す、簡単な整形や集計を行う

ここからは、各コマンドの役割と主要なオプションを順に説明する。

2.5.1 `cat`

`cat` はファイルの中身をそのまま標準出力へ流す。短い設定ファイルや、数行だけの確認用テキストを一気に見たいときに向く。複数のファイルを並べて渡すと、内容を順に連結して表示できる。

```
$ cat data/events.txt
$ cat header.txt body.txt
$ cat log/*.log
$ cat -n config.txt
```

3 行目のように、前節のワイルドカードを使えば複数のログファイルをまとめて連結して表示できる。ただし、巨大なログや観測データへそのまま使うと画面が一気に流れて読めない。長いファイルは、次に述べる `less` や `head`、`tail` を使う方が自然である。

-n 行番号を付けて表示する。設定ファイルやログの何行目に何があるかを確認したいときに便利である。

2.5.2 less

`less` は長いファイルをスクロールしながら読むためのコマンドである。大きなデータを「まず中身だけ見たい」とときには、最初に `less` を使うことが多い。

```
$ less data/events.txt
$ less -N data/events.txt
```

`less` は単なる表示コマンドではなく、表示中に検索や移動ができる。ログや観測データの形式を確認するときは、次の操作を覚えておくとよい。

-N 行番号を表示する。後で「どの行に何があったか」を参照しやすくなる。

Space 1 画面ぶん先へ進む。

b 1 画面ぶん戻る。

/pattern 指定した文字列を前方検索する。たとえば `/ERROR` と打てば、`ERROR` を含む次の行へ飛べる。

n 直前の検索条件の次の一致へ進む。

q 表示を終了してターミナルへ戻る。

2.5.3 head

`head` はファイルの先頭だけを見るためのコマンドである。どのような形式のデータが入っているか、ヘッダやコメント行はどのようなになっているかを確認したいときに向く。

```
$ head data/events.txt
$ head -n 20 data/events.txt
```

-n N 先頭から `N` 行だけ表示する。既定では 10 行なので、少し長めに見たいときに使う。

2.5.4 tail

tail はファイルの末尾だけを見るためのコマンドである。ログファイルの最新部分を確認したいときや、計算結果が末尾へ追記される形式のファイルを見るときに使う。

```
$ tail log.txt
$ tail -n 30 log.txt
$ tail -f log.txt
```

-n N 末尾から N 行だけ表示する。直近 20 行や 30 行だけ見たいときに使う。

-f ファイルの末尾を監視し、追記された内容を続けて表示する。実行中のジョブのログを見るときに有用である。

2.5.5 wc

wc はファイルの行数、単語数、バイト数などを数えるコマンドである。特に -l は「何行あるか」を知るためによく使う。データ件数やログの大きさをざっと見積もるときの第一歩になる。

```
$ wc data/events.txt
$ wc -l data/events.txt
$ wc -w memo.txt
$ wc -c output.bin
```

-l 行数を数える。最もよく使う。

-w 単語数を数える。自然言語テキストの確認で使うことが多い。

-c バイト数を数える。ファイルサイズを概算したいときに役立つ。

2.5.6 grep

grep は、条件に合う行だけを抽出するためのコマンドである。ログからエラー行を探す、コメント行を除く、特定の測定条件を含む行だけ取り出す、といった用途で頻繁に使う。

```
$ grep "ERROR" log.txt
$ grep -n "ERROR" log.txt
$ grep -A 2 -B 1 "ERROR" log.txt
$ grep -i "warning" log.txt
$ grep -v "^#" data.txt
$ grep -rn "TODO" src/
```

-i 大文字と小文字を区別せずに検索する。error と ERROR を同じものとして扱いたいときに使う。

-v 一致した行ではなく、一致しなかった行を出す。コメント行やノイズ行を除外したいときに便利である。

- n 一致した行の行番号も表示する。後で元ファイルを見直すときに役立つ。
- A N 一致した行の後ろを N 行ぶん追加で表示する。エラー行の直後にある補足メッセージも一緒に見たいときに便利である。
- B N 一致した行の前を N 行ぶん追加で表示する。エラーの直前に何が起きていたかを見たいときに使う。
- r ディレクトリの中を再帰的に検索する。複数ファイルにまたがって同じ文字列を探したいときに使う。たとえば `grep -rn "TODO" src/` と書けば、`src/` 以下のすべてのファイルをたどり、TODO を含む行を行番号付きで表示できる。日常的なログ確認では -A や -B の方を先に使うことが多いが、複数ファイル検索ではこちらも重要である。

grep では正規表現も使える。たとえば `^` は行頭、`$` は行末を表すので、`grep "^#" data.txt` と書けば「# で始まる行」だけを拾える。また、複数のログファイルをまとめて調べたいなら、`grep -r "ERROR" log/` のようにディレクトリごと渡せばよい。詳細は付録 C を参照されたい。

2.5.7 awk

awk は、テキストを列データとして扱って抽出や整形を行うコマンドである。空白区切りのログや表形式テキストを扱うときには非常に強力である。

```
$ awk '{print $1}' data.txt
$ awk '{print $1, $3}' data.txt
$ awk -F, '{print $2}' data.csv
$ awk '{ if ($3 > 10) print $1, $3 }' data.txt
$ awk '{ sum += $3 } END { print sum }' data.txt
```

既定では空白やタブが区切り文字として使われる。そのため、空白区切りの観測データなら `$1` が 1 列目、`$2` が 2 列目という意味になる。条件分岐は `if` を明示的に書くこともできるので、「3 列目が 10 より大きい行だけ表示する」といった処理の流れが読みやすい。また、各行で値を足し合わせて最後に合計や平均を出す、といった簡単な計算もそのまま書ける。

-F 区切り文字を指定する。CSV のようにカンマ区切りなら -F, のように書く。

`{print $1}` 1 列目を表示する最も基本的な形である。`{print $1, $3}` のようにすれば複数列を並べて出せる。

`{ if ($3 > 10) print $1, $3 }` 条件分岐を明示的に書く形である。最初のうちは、このように `if` を書いた方が処理の意味を追いやすい。

`{ sum += $3 } END { print sum }` 各行で 3 列目を足し上げ、最後に合計を表示する。簡単な集計や平均計算の出発点として便利である。

2.5.8 パイプ |

さらに、強力な手段としてパイプ (`|`) がある。パイプは、あるコマンドの標準出力を次のコマンドの入力へそのまま流し込む機能である。単機能のコマンドを組み合わせたい結

果を得るのが UNIX 環境の流儀である。

```
$ cat log.txt | grep "ERROR"
$ grep "ERROR" log.txt | wc -l
$ cat events.txt | grep -v "noise" | awk '{print $2}'
```

1 行目は log.txt の内容を読み出し、その中から ERROR を含む行だけを表示する。2 行目では、抽出結果をさらに wc -l へ渡し、エラー件数を数えている。3 行目では、ノイズ行を除外したうえで 2 列目だけを取り出している。

このように、まず全体を見る、次に条件で絞る、最後に列を抜く、という手順を 1 行の中でつなげられる。後の章で tee やリダイレクトを学ぶと、ここへログ保存や標準エラー処理も組み合わせられるようになる。

2.6 環境変数と PATH

2.6.1 環境変数を使う

シェルやプログラムには、「名前」と「値」の組として渡される設定がある。これが環境変数である。環境変数は、シェルの中だけで完結するものではなく、そこから起動した Python や各種コマンドにも引き継がれる。現在設定されているよく使う環境変数の値を見るには、次のように打つ。

```
$ echo $HOME
$ echo $SHELL
$ echo $PATH
```

HOME はホームディレクトリ、SHELL は使っているシェル、PATH は実行ファイルを探すディレクトリの並びを表す。後で出てくる PYENV_ROOT や DATA_ROOT も同じ仲間であり、「プログラムへ外から値を渡す仕組み」と考えると分かりやすい。新たに環境変数を設定するには次のようにする。

```
$ export DATA_ROOT="$HOME/data/events"
$ echo $DATA_ROOT
```

export すると、そのシェルから起動する後続のコマンドにも値が渡る。たとえば Python スクリプト側で DATA_ROOT を読めば、データ置き場をコードへ埋め込まずに済む。

ただし、この設定は通常そのターミナルを閉じると消える。毎回使う値ならシェルの設定ファイルに export ... を書いておく。どのファイルを編集するかは今使っているシェルで決まる。2.1 節で述べたように、echo \$SHELL を実行することで使用しているシェルの種類を確認することができる。/bin/zsh と出た場合は ~/.zshrc、/bin/bash と出た場合は ~/.bashrc に export ... を追記すればよい。

2.6.2 展開と引用

シェルでは、コマンドを実行する前に `$HOME` のような変数展開や、`$(pwd)` のようなコマンド置換が行われる。ここで重要なのは、引用符の種類によって展開の有無が変わることである。

```
$ echo $HOME
$ echo "$HOME"
$ echo '$HOME'
$ name="analysis result"
$ echo $name
$ echo "$name"
$ echo "$(pwd)"
$ echo "`pwd`"
```

1 行目と 2 行目は、どちらも `$HOME` を展開する。多くの場合、`HOME` には空白が入らないので、見た目の結果は同じになる。しかし、クォートなしの `$HOME` は展開後に空白で分割されたり、ワイルドカードとして再解釈されたりする可能性がある。これに対して `"$HOME"` は 1 つの引数として扱われるので、安全である。したがって、「変数は基本的にダブルクォートで囲む」と覚えた方がよい。

3 行目のシングルクォート `'...'` は、中の文字列をほぼそのまま扱う。そのため `$HOME` は展開されず、文字どおり `$HOME` と表示される。4 行目と 5 行目の違いは、空白を含む値で見ると分かりやすい。 `name="analysis result"` のあとで `echo $name` と書くと、シェルは `analysis` と `result` を別々の単語として扱う。これに対して `echo "$name"` なら、空白を含んだまま 1 つの値として渡る。6 行目の `$(pwd)` は `pwd` の出力で置き換える書き方であり、7 行目のバッククォート記法も同じ目的で使われる。ただし、新しく書くなら `$(...)` の方が読みやすく、入れ子にもできるので自然である。

この違いは、設定ファイルへ 1 行追記するときに特に重要である。たとえば `echo 'export PATH="$HOME/bin:$PATH"' >> ~/.zshrc` のようにシングルクォートで囲むと、`$HOME` や `$PATH` を展開せず、その文字列をそのまま `~/.zshrc` へ追記できる。もしダブルクォートで囲むと、その場で現在の値へ展開されて書き込まれるので、意図とずれることがある。

また、空白と改行の扱いにも注意が必要である。クォートの外では、空白やタブや改行が単語の区切りになる。そのため、空白を含むファイル名はクォートしないと壊れる。

```
$ mkdir "analysis result"
$ cp "analysis result/data.txt" backup/
```

1 つの長いコマンドを複数行へ分けたいときは、行末に `\` を置いて改行を継続する。

```
$ python analyze.py \
  --input data/events.txt \
  --output result.txt
```

改行は通常その場でコマンドの終わりになるので、`\` を付けずに改行すると別のコマンド

として解釈される。コマンド例を写すときは、空白、クォート、改行のどれが意味を持っているかを意識した方が安全である。

2.6.3 PATH と実行ファイル

シェルが `python3` や `ls` を実行できるのは、それらが実行ファイルとして保存されており、しかもシェルがその場所を知っているからである。その探索に使う環境変数が `$PATH` である。実際に実行ファイルがどこから使われているかを探すには、次のコマンドを用いる。

```
$ which python3
$ which ls
$ echo $PATH
```

`which python3` を実行すると、たとえば Homebrew の Python なら `/opt/homebrew/bin` の下、仮想環境なら `.venv/bin` の下の実行ファイルが表示される。シェルは `$PATH` に並んだディレクトリを左から順に調べ、最初に見つかった実行ファイルを使う。

`bin` という名前は、実行ファイルを置くディレクトリによく使われる。後で出てくる `.venv/bin` の `bin` も、この慣習に従っている。

自分が作ったシェルスクリプトや実行ファイルを別のディレクトリ（例えば `~/my_scripts`）に置き、どこからでもコマンド名だけで呼び出したい場合は、次のように現在の `$PATH` の先頭に自分のディレクトリを追加して上書きすればよい。これもシェル起動時に毎回行う必要があるなら、`~/.zshrc` などに記載しておく。

```
$ export PATH="$HOME/my_scripts:$PATH"
```

2.7 リダイレクトとログ保存

2.7.1 echo、標準出力、標準エラー

シェルのコマンドは、単に画面へ文字を出すだけではなく、出力先を切り替えられる。ここで区別したいのは、通常の結果を流す標準出力（standard output, `stdout`）と、エラーメッセージを流す標準エラー（standard error, `stderr`）である。

`echo` は標準出力へ文字列を出す最も基本的なコマンドであり、動作確認や設定ファイルへの追記でよく使う。

```
$ echo "hello"
$ echo "$HOME"
```

1 行目は単に `hello` を出力する。2 行目は前節で述べた変数展開により、`$HOME` の値を出力する。ここで重要なのは、`echo` 自体は画面へ出しているだけであり、その出力先は次の節のリダイレクトで変えられるという点である。

2.7.2 > と >>

> は標準出力をファイルへ書き出す。既存のファイルがあれば中身を上書きする。

```
$ echo "alpha" > notes.txt
$ cat notes.txt
alpha
```

この例では、echo の結果が画面ではなく notes.txt に保存される。すでに notes.txt が存在していた場合も、その内容は新しい内容で置き換わる。したがって、> は便利だが、うっかり使うと既存ファイルを消してしまう。

>> は追記であり、既存の内容を残したまま末尾に足す。

```
$ echo "alpha" > notes.txt
$ echo "beta" >> notes.txt
$ cat notes.txt
alpha
beta
```

設定ファイルの末尾へ 1 行追加したいときは、この >> を使う。たとえば ~/.zshrc に環境変数の設定を足すときに自然である。逆に、> を使うと既存の設定を消してしまうので注意が必要である。

2.7.3 1> と 2>

> は実際には 1> と同じ意味であり、標準出力だけをファイルへ送る。標準エラーを別ファイルへ分けたいときは 2> を使う。

```
$ python analyze.py > output.log
$ python analyze.py 1> output.log 2> error.log
```

1 行目では通常の出力だけが output.log に入り、エラーメッセージは画面に残る。2 行目では、通常の出力を output.log に、エラーメッセージを error.log に分けて保存している。長い処理を回すときは、「結果は保存したいが、失敗したときの原因も別に残したい」ということが多いので、この書き方は有用である。

標準出力と標準エラーを同じファイルへまとめたいときは、2>&1 を使う。

```
$ python analyze.py > run.log 2>&1
```

これは、標準エラーの行き先を標準出力と同じ先へ合わせる、という意味である。ログを 1 本にまとめたいときに使う。

2.7.4 tee で見ながら保存する

> や >> を使うと出力はファイルへ送られ、画面には出なくなる。実行中の様子も見たいが、同時に保存もしたいときは tee が便利である。tee は標準入力を受け取り、その内容を画面へ出しつつファイルへも書く。

```
$ python analyze.py | tee run.log
$ python analyze.py 2>&1 | tee run.log
```

1 行目は標準出力だけを画面と `run.log` の両方へ流す。2 行目は標準エラーもまとめてから `tee` へ渡しているので、実質的に「全部を見ながら全部を保存する」形になる。処理が数分から数十分かかる場合、進捗を画面で見つつ、あとで見返すためのログも残せるので実用的である。

`tee` に `-a` を付けると追記になる。

```
$ python analyze.py 2>&1 | tee -a run.log
```

この場合は、既存の `run.log` を消さずに末尾へ足していく。毎日の実行結果を 1 つのログへ蓄積したい場合に向く。

2.8 権限とスクリプトの実行

2.8.1 権限を読む

ファイルには権限がある。読み取れるか、書き込めるか、実行できるかを `rwX` で表す。`ls -l` を使うと確認しやすい。

```
-rwxr-xr-x 1 user staff 1234 Apr 8 10:00 run_analysis.sh
-rw-r--r-- 1 user staff 411 Apr 8 10:05 notes.txt
-rw----- 1 user staff 411 Apr 8 10:05 id_ed25519
```

最初の 10 文字のうち、先頭 1 文字はファイル種別であり、残り 9 文字が権限である。3 文字ずつに区切って、所有者、グループ、その他の利用者に対する権限を表す。文字と権限の対応は表 2.3 のとおりである。

権限を変更するには `chmod` コマンドを用いる。所有者 (`user`) に実行権限 (`x`) を付与したければ `u+x`、その他の利用者 (`other`) の読み込み権限 (`r`) と書き込み権限 (`w`) を剥奪したければ `o-rw`、グループ (`group`) とその他 (`other`) の権限を読み取り (`r`) のみにしたければ `go=r` のように指定すればよい。`a+x` のようにして、すべてのユーザー (`all`) の権限を変更することもできる。

```
$ chmod u+x my_script.sh
$ chmod o-rw notes.txt
$ chmod go=r read_only.txt
$ chmod a+x run_analysis.sh
```

表 2.3 に示すように、権限は数字で指定することもできる。数字指定では権限を足し算で表す。たとえば 7 は `rwX`、6 は `rw-`、5 は `r-x` である。したがって 755 は所有者が `rwX`、それ以外が `r-x` を意味し、644 は所有者が `rw-`、それ以外が `r--` を意味する。秘密鍵のように本人だけが読めればよいものには 600 がよく使われる。設定を変更する例を下に示す。

表 2.3 権限の記号と数字

記号	数字	意味
r	4	読み取り
w	2	書き込み
x	1	実行

```
$ chmod 755 run_analysis.sh
$ chmod 644 notes.txt
$ chmod 600 ~/.ssh/id_ed25519
```

2.8.2 スクリプトを直接実行する

Python スクリプトは `python3 hello.py` の形で実行するなら、実行権限がなくてもよい。これに対して `./hello.py` のように直接実行する場合には、どのプログラムで解釈するかをファイル先頭に書いておく必要がある。その先頭行の `#!` で始まる記法を、シバン (shebang) という。

```
1 #!/usr/bin/env python3
2
3 print("hello")
```

このように記述した上で、ファイルそのものに対して直接実行できるようにするため、実行権限を付与する。所有者に対して権限を付与する場合は、付与の対象 (u) を省略することができる。

```
$ chmod +x hello.py
$ ./hello.py
```

`#!/usr/bin/env python3` は、「python3 を見つけてこのファイルを実行せよ」という意味である。いきなり言葉だけで覚える必要はないが、「直接実行にはシバンと実行権限の両方が必要だ」という構造だけは理解しておいた方がよい。

第 3 章 Python 環境を整える

3.1 Python の確認と管理方針

3.1.1 何を分けて考えるべきか

Python を使い始めるときは、次の 3 つを分けて考える必要がある。

1. Python 本体を入れること
2. 複数バージョンの Python を切り替えること
3. プロジェクトごとの仮想環境を作ること

まず Python 本体が必要であり、そのうえで Python 3.11 と 3.12 を切り替えたいならバージョン管理ツールが必要になる。さらに、同じ Python 3.12 でもプロジェクト A と B で依存ライブラリが違えば、仮想環境で分ける必要がある。この 3 段階を分けて理解した方が、あとで混乱しにくい。

3.1.2 今の Python を確認する

macOS でも WSL でも、最初にいま見えている Python を調べておく方がよい。現在利用可能な Python を確認するには、次のように実行する。

```
$ python3 --version
$ which python3
$ python3 -c "import sys; print(sys.executable)"
```

`python3 --version` でバージョンを確認し、`which python3` と `sys.executable` で実体の場所を確認する。複数バージョンを切り替えるようになると、この確認は非常に重要になる。

3.2 pyenv による複数バージョンの管理

Python 本体の導入と切り替えには `pyenv` が分かりやすい。`pyenv` は、シェルから見える `python` をディレクトリ単位で切り替えるための道具であり、複数バージョンの Python を使い分けるときの基準として扱いやすい。

3.2.1 macOS での準備

`pyenv` は Python 本体をビルドして入れるので、macOS では Xcode Command Line Tools が必要になる。すでにインストールされているか、パスを確認するには次のように実行する。

```
$ xcode-select -p
```

すでに Homebrew が正常に動いているなら、Command Line Tools は入っていることが多い。上のコマンドでパスが表示されれば、そのまま進んでよい。表示されない場合は、次のコマンドを実行してインストールする。

```
$ xcode-select --install
```

3.2.2 WSL / Ubuntu での準備

WSL の Ubuntu では、ビルドに必要なライブラリを先に入れておく。一括で導入するには次のコマンドを実行する。

```
$ sudo apt update
$ sudo apt install -y build-essential curl git libssl-dev zlib1g-dev \
  libbz2-dev libreadline-dev libsqlite3-dev libffi-dev liblzma-dev tk-dev
```

3.2.3 pyenv を入れる

```
$ curl -fsSL https://pyenv.run | bash
```

3.2.4 pyenv をシェルへ組み込む

前章で確認したように、pyenv を使うには自身のシェルの種類に合わせた設定ファイル（/bin/zsh なら ~/.zshrc、/bin/bash なら ~/.bashrc）の末尾へ初期化コードを書く必要がある。以下では >> を使って設定ファイルの末尾へ 1 行ずつ追記している。> ではなく >> を使うのは、既存の設定を消さないためである。

```
$ echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
$ echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
$ echo 'eval "$(pyenv init - zsh)"' >> ~/.zshrc
$ exec "$SHELL"
```

使っているシェルが bash の場合は、次のように ~/.bashrc に対して設定を追記する。

```
$ echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
$ echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
$ echo 'eval "$(pyenv init - bash)"' >> ~/.bashrc
$ exec "$SHELL"
```

3.2.5 バージョンを入れて切り替える

pyenv を使う場合は、pyenv install でバージョンを追加し、pyenv global、pyenv local、pyenv shell で切り替える。たとえば新しいバージョンを追加して確認するには次のように行う。

```
$ pyenv install 3.11.11
$ pyenv install 3.12.10
$ pyenv versions
```

その後、システム全体で普段使う標準のバージョンを決めるなら、次のように `pyenv global` を実行する。

```
$ pyenv global 3.12.10
$ python --version
$ which python
```

`pyenv global` は普段使うバージョンを決める。これに対して `pyenv local` は、そのディレクトリだけ別のバージョンを使いたいときに使う。特定のプロジェクト用ディレクトリ内でバージョンを固定するには、次のような手順を踏む。

```
$ mkdir analysis311
$ cd analysis311
$ pyenv local 3.11.11
$ python --version
$ cat .python-version
```

`pyenv local` を実行すると `.python-version` が作られ、そのディレクトリでは指定したバージョンが優先される。これによって、同じマシンの中でプロジェクト A は 3.11、プロジェクト B は 3.12 という使い分けができる。

`pyenv shell 3.11.11` は、そのターミナルを閉じるまでの一時的な切り替えである。検証用に 1 回だけ古いバージョンで動かしたいときに便利である。

3.3 venv でプロジェクトごとの仮想環境を作る

Python 本体のバージョンを決めたら、そのバージョンの上にプロジェクト専用の仮想環境を作る。依存ライブラリの衝突を避けるためである。仮想環境を作り、そこに入るための基本コマンドは次の通りである。

```
$ python -m venv .venv
$ source .venv/bin/activate
$ which python
$ which pip
$ python --version
$ pip --version
```

仮想環境へ入ると、プロンプトの先頭に `(.venv)` が付くことが多い。ターミナルの表示は次のように変わる。

```
(.venv) ikomae@macbook analysis $
```

`which python` と `which pip` を見ると、`.venv/bin/python` と `.venv/bin/pip` が使われて

いることが分かる。これは、`source .venv/bin/activate` が `$PATH` の先頭に `.venv/bin` を追加しているからである。

3.3.1 pip と python -m pip

仮想環境へ入っているなら、通常は `pip` と `python -m pip` は同じ環境の `pip` を指す。したがって、作業中は `pip install numpy` と書いてよい。

一方で、いまどの Python が有効か不安なときや、スクリプトや文書の中で「この Python に対応する pip を使う」と明示したいときは、`python -m pip` の方が誤解が少ない。実務では両方見るが、意味の違いを分かって使うことが重要である。実際にライブラリを入れる例を挙げる。

```
$ pip install --upgrade pip
$ pip install numpy pandas matplotlib scipy astropy tqdm
```

作業を終えたら `deactivate` で仮想環境から抜けられる。

3.4 解析用ライブラリと JupyterLab

3.4.1 JupyterLab を併用する

対話的に試しながら解析したいときは、JupyterLab も便利である。小さなコード片をその場で動かし、数値や図をすぐ確認できるため、探索的な作業では特に役立つ。これを導入・起動するには次のように実行する。

```
$ pip install jupyterlab
$ jupyter lab
```

JupyterLab の利点は、コードを実行できるだけでなく、その前後に説明文、数式、図の読み方を書き添えられる点にある。解析の途中経過を他人へ見せるときも、自分で後から読み返すときも、コードと説明が同じノートブックに並んでいる方が理解しやすい。たとえば次のような書式で Markdown セルを書くことができる。

```
# 解析メモ

- 入力ファイル: `events_2024_01_01.txt.gz`
- 閾値: `signal > 5.0`

 $\Delta t$  の分布を確認する
```

ただし、JupyterLab は最終成果物の置き場所ではなく、試行錯誤の場として使う方が整理しやすい。解析手順を再現したい段階では、ノートブック中で固めた処理を Python スクリプトへ戻し、入力や出力を明示した方が後で困りにくい。

3.5 プロジェクト管理ツール uv の利用

uv は Python 本体の導入もできるが、本書ではそれよりも「プロジェクト単位の管理」に使う道具として位置付ける方が自然である。特に、依存関係、ロックファイル、`.python-version`、Git 管理、パッケージ化、ワークスペース管理といった、プロジェクト全体の整理で力を発揮する。

3.5.1 プロジェクトの初期化

多くの人は、まずシンプルに `uv init` を使えばよい。オプションは後から必要になったときに足せばよい。新しいプロジェクトを始めるには、次のようにディレクトリを作ってから初期化する。

```
$ mkdir analysis_uv
$ cd analysis_uv
$ uv init
```

既定の `uv init` では、少なくとも次のようなファイルやディレクトリが作られる。(tree もしくは `ls -a` コマンドで確認できる。)

```
.git/
.gitignore
.python-version
README.md
main.py
pyproject.toml
```

つまり、`uv init` は単に Python ファイルを 1 つ置くだけではなく、Git 管理の土台までまとめて用意する。この点が `venv` 単体との大きな違いである。

既定動作を変えたいときだけオプションを使えばよい。

- `uv init --bare`: `pyproject.toml` だけを作る
- `uv init --vcs none`: Git 初期化をしない
- `uv init --no-pin-python`: `.python-version` を作らない
- `uv init --package` や `uv init --lib`: スクリプトではなくパッケージやライブラリとして始める

特に、コードを再利用可能なモジュールやライブラリとして育てたいなら、`--package` や `--lib` を意識した方がよい。単発スクリプトで終わるのか、将来 `import` される部品になるのかで、適切な初期構成は変わる。

3.5.2 依存関係の追加と環境の同期

`uv add` は依存ライブラリを追加し、`pyproject.toml` とロック情報を更新する。複数同時に指定する場合は次のようにする。

```
$ uv add numpy pandas matplotlib scipy astropy tqdm
```

`uv sync` は、`pyproject.toml` とロックファイルに基づいて環境をそろえる。ここで初めて `.venv` が作られることもある。実行コマンドは次の通りである。

```
$ uv sync
```

過去の自分の解析や、他人から受け取ったプロジェクトを再現するときは、この `uv sync` が重要になる。リポジトリを取得して `uv sync` すれば、必要な依存関係をまとめてそろえやすい。ここで Git の基本が必要になるので、付録 B もあわせて読むとよい。

3.5.3 コマンドの実行とバージョン固定

`uv run` は、そのプロジェクトの環境でコマンドを実行する。スクリプトを直接走らせるときと、Python 本体を起動したいときとで書き分ける。いくつか例を挙げる。

```
$ uv run main.py
$ uv run python --version
$ uv run python -m pytest
```

実行対象が Python スクリプトそのものであるなら、`uv run main.py` と書けばよい。わざわざ `python` を挟む必要はない。一方で、`-m` を使いたいときや、Python 本体へオプションを渡したいときは `uv run python ...` と書く。

`uv` もプロジェクト単位で Python バージョンを固定できる。`pyenv` がシェル全体の切り替えを担当し、`uv` がプロジェクト側の固定を担当する、と分けて考えると整理しやすい。固定や一発実行のコマンド例は次の通りである。

```
$ uv python pin 3.12
$ uv run python --version
$ uv run --python 3.11 python --version
```

3.5.4 標準ライブラリと外部ライブラリ

Python には、最初から使える標準ライブラリと、自分で追加する外部ライブラリがある。

- 標準ライブラリの例: `math`, `pathlib`, `glob`, `argparse`
- 外部ライブラリの例: `numpy`, `pandas`, `matplotlib`, `scipy`, `astropy`

標準ライブラリは Python 本体に含まれている。外部ライブラリは、目的に応じて自分で追加する。解析では外部ライブラリが非常に重要だが、まずは「何が標準で、何が追加なのか」を意識しておく整理しやすい。

3.5.5 解析用のライブラリを入れる

本書の後半で使う主なライブラリを表 3.1 に示す。

表 3.1 本書で使う主なライブラリと役割

ライブラリ	主な役割
NumPy	数値配列の高速処理
pandas	表形式データの読み込みと加工
Matplotlib	図の作成
SciPy	フィットと数値計算
Astropy	時刻・単位・座標の扱い
tqdm	進捗表示

venv を使うなら `pip install`、uv を使うなら `uv add` と書けばよい。それぞれの書き方を以下に示す。

```
$ pip install numpy pandas matplotlib scipy astropy tqdm
```

```
$ uv add numpy pandas matplotlib scipy astropy tqdm
```

3.6 引数・設定ファイルとコードの分離

3.6.1 コマンドライン引数と設定ファイルを使い分ける

同じ解析スクリプトを何度も使うなら、入力ディレクトリや出力先をコードに埋め込むより、コマンドライン引数で渡す方がよい。Python では `argparse` が標準で使える。もっともシンプルな構成例を下に示す。

```
1 import argparse
2
3 parser = argparse.ArgumentParser()
4 parser.add_argument("input_dir")
5 parser.add_argument("--outdir", default="output")
6 args = parser.parse_args()
7
8 print(args.input_dir, args.outdir)
```

ただし、何でも引数にすればよいわけではない。毎回ほぼ同じ値まで毎回書かせると、使いにくいスクリプトになる。

値の置き場所は、表で機械的に覚えるより、用途ごとに言葉で整理した方が分かりやすい。

コマンドライン引数 実行ごとに変わる値を置く。入力ディレクトリ、出力先、日付範囲、並列数などが典型である。

設定ファイル プロジェクト単位では固定だが、将来変更し得る値を置く。較正ファイルのパス、検出器 ID、閾値、図の保存形式などが当てはまる。

環境変数 マシン依存の値や秘密情報を置く。DATA_ROOT、API トークン、認証情報などが典型である。

コード内の定数 定義として固定したい値や単位変換を書く。NS_PER_SECOND や物理定数のように、意味が仕様として決まっているものを置く。

入力ファイルのパス、出力先、しきい値、作業ディレクトリのように「実行条件」であるものは、原則としてハードコーディングしない方がよい。コードを書き換えずに切り替えられるようにしておく、と、再利用と再現が楽になる。

3.6.2 設定ファイルとマジックナンバーの排除

めったに変わらない値は、設定ファイルへまとめる方が使いやすい。形式は JSON でも TOML でもよい。ここでは標準ライブラリだけで扱える JSON の例 (config.json) を示す。

```
{
  "calibration_path": "calib/detector_a.csv",
  "coincidence_window_ns": 250,
  "figure_format": "pdf"
}
```

この JSON ファイルを Python のスクリプトから読み込む場合のコード例は次のようになる。

```
\begin{lstlisting}[language=Python]
import json
from pathlib import Path

config = json.loads(Path("config.json").read_text())
window_ns = config["coincidence_window_ns"]
\end{lstlisting}
```

このようにしておけば、同じコードを保ったまま設定だけを差し替えられる。Python 3.11 以降なら tomllib を使って TOML を読むこともできる。どの形式を選ぶにしても、「たまたま変わるがコード本体ではない値」を分離することが大切である。

250 や 3600 のような数値が説明なしにコードへ裸で出てくると、後から読んだ人は意味を判断しにくい。このような数値はマジックナンバーと呼ばれる。詳しくは [4.5.1.3 節](#) で述べるが、意味のある数値には名前を付け、実行条件なら引数や設定ファイルへ出す、という整理を早い段階で身に付けた方がよい。

第 4 章 Python の基本

4.1 データ型と基本操作

4.1.1 変数と基本型

Python では、値に名前を付けて使う。たとえばデータを表すいくつかの変数を定義すると次のようになる。

```
1 n_rows = 12
2 threshold = 3600.0
3 label = "A"
4 use_log = True
```

`n_rows` は整数、`threshold` は浮動小数点数、`label` は文字列、`use_log` は真偽値である。解析では、整数と浮動小数点数を区別して扱う感覚が重要になる。たとえば、件数は整数だが、フィットで得られるパラメータは浮動小数点数であることが多い。

Python は変数宣言を先に書かなくてよいが、その代わり自分で型を意識して読む必要がある。特に 1 と 1.0 の違い、文字列としての '12' と整数としての 12 の違いは、ファイル読み込みや数値計算で頻繁に問題になる。

4.1.2 数値と演算子

Python は、対話的な計算機として使い始めるのが理解しやすい。四則演算、べき乗、剰余、整数除算など、基本的な計算の書き方は次の通りである。

```
1 2 + 3
2 7 / 3
3 7 // 3
4 7 % 3
5 2 ** 10
```

`/` は浮動小数点数として割り算を行い、`//` は整数部分を返す。観測時間を秒へ直す、ビン番号を求める、インデックスを分ける、といった処理では、この違いが重要になる。特に `//` と `%` は、周期的な区切りやグループ分けでよく現れる。

浮動小数点数の計算では、表示された値が見た目通りに内部で表現されているとは限らない。これは Python 特有の問題ではなく、一般的な計算機の表現上の性質である。後半でフィットや統計量を扱うときにも、この前提を意識しておきたい。

4.1.3 文字列

ファイル名、日付時刻、ヘッダ、ラベルなど、解析コードでは文字列を扱う場面が多い。文字列は単なる表示用のデータではなく、分割、検索、整形の対象でもある。基本的な操作例を挙げる。

```
1 name = "detector_A"
2 date = "2026-04-04"
3
4 print(name.upper())
5 print(date.replace("-", "/"))
6 print(name.split("_"))
```

split、replace、strip のような操作は、ファイル読み込みの前処理で頻繁に使う。特に split は、空白区切りやカンマ区切りの 1 行を各列へ分ける最初の道具である。NumPy や pandas を使う前に、この感覚を持っておくとデータ形式の理解が速い。

4.2 データ構造

4.2.1 リスト、タプル、辞書

複数の値をまとめたいときには、いくつかの基本的な入れ物がある。よく使う 3 種類の定義例を下に示す。

```
1 labels = ["A", "B", "C", "D"]
2 point = (12, 18)
3 row = {"id": 42, "value": 3.5, "flag": True}
```

リストは順序付きで変更可能、タプルは順序付きで変更不可、辞書は名前付きの値を持つ。最初のうちは辞書が読みやすいが、大きなデータを大量に処理するときはオーバーヘッドが大きくなることもある。この違いは後の高速化章で再び出てくる。

どの入れ物を選ぶかは、見た目の好みではなく、何をしたいかで決まる。位置でアクセスするならリストやタプル、名前でアクセスするなら辞書が自然である。小さなコードでは差が見えにくいだが、処理量が増えると選択の重みが増してくる。

4.2.2 リストのメソッド

Python 公式チュートリアルでも詳しく扱われているように、リストにはいくつかの基本メソッドがある。これらを知っていると、簡単な前処理や集約を自然に書ける。主な操作は次のようになる。

```
1 values = [3, 1, 4]
2 values.append(1)
3 values.sort()
4 last = values.pop()
```

`append` は末尾へ追加し、`sort` はその場で並べ替え、`pop` は末尾要素を取り出して削除する。リストメソッドの多くは「その場で変更する」点に注意が必要である。新しいリストを返すのではなく、元のリスト自体を書き換える操作が多いので、どこで状態が変わるかを意識して読む必要がある。

解析コードでは、ファイル一覧の作成、簡単なバッファ、途中結果の蓄積などにリストを使うことが多い。ただし、大量の数値そのものを主役として扱う段階では、後で NumPy 配列へ移った方が自然になることが多い。

4.2.3 インデックスとスライス

解析では、配列や文字列の一部だけを取り出したい場面が非常に多い。Python では、`[]` の中に位置を書くことで要素を取り出し、`:` を使うことで範囲を切り出せる。次のように記述する。

```
1 values = [10, 20, 30, 40, 50]
2
3 first = values[0]
4 last = values[-1]
5 middle = values[1:4]
```

最初の要素が 0 番目であること、終点の添字は含まれないこと、負の添字で末尾から数えられることは、最初にしっかり覚えておきたい。時刻列や列名の一部を取り出すとき、この基本がそのまま使われる。

文字列に対しても同じ考え方が使える。例えば `hhmmss[0:2]` で時、`hhmmss[2:4]` で分を取り出せる。この一貫性が Python の扱いやすさの一つである。

4.3 制御構文

4.3.1 条件分岐

条件によって処理を変えるには `if` を使う。

```
1 if value <= threshold:
2     print("small")
3 else:
4     print("large")
```

解析コードでは、閾値判定、ファイル名の選別、欠損値の除外など、条件分岐が頻繁に現れる。重要なのは、「なぜその条件で分けているのか」を自分で説明できることだ。

条件分岐が増えて読みにくくなったときは、条件式そのものを変数へ切り出すと分かりやすくなる。たとえば `is_valid_row = value <= threshold` のように一度名前を付けるだけでも、後で読むときの負担がかなり減る。

条件は左から右へ読んで意味が分かる形に書く方がよい。否定が重なる条件や、複数の

and と or が混ざる条件は、括弧や一時変数で整理した方が安全である。解析コードでは、条件の誤読がそのままバグになる。

4.3.2 反復処理

複数の値を順に調べるには for を使い、次のように取り出す。

```
1 for label in labels:
2     print(label)
```

要素の番号（添字）も必要なら enumerate を組み合わせると便利である。

```
1 for i, value in enumerate(labels):
2     print(i, value)
```

一方で、データ点が非常に多いとき、Python レベルの for ループが速度の瓶頸になることがある。本書では後半で、その回避方法を詳しく扱う。

初学者の段階では、まず正しく読める for 文を書くことが先である。速く書くことを意識するのは、その後でよい。読みやすい基本形を知っているからこそ、後で NumPy や pandas へ置き換えたときに、何を省略しているかが分かる。

4.3.2.1 range と while

for 文と並んで、反復処理でよく使うのが range と while である。range は連続した整数を順に生成し、while は条件が成り立つ間くり返す。次のように記述する。

```
1 for i in range(5):
2     print(i)
3
4 count = 0
5 while count < 3:
6     print(count)
7     count += 1
```

range(5) は 0 から 4 までを返す。ファイル番号やビン番号のような添字処理では便利である。一方、while は終了条件を自分で管理する必要があるので、無限ループにしないよう注意が必要である。時刻順に読み進めて、条件を満たす間だけ比較するような処理では while が自然な場合もある。

4.3.2.2 break, continue, else

反復処理では、途中で打ち切ったり、ある条件だけ飛ばしたりしたくなる。Python では break と continue がそのために使える。

```
1 for value in values:
2     if value < 0:
3         continue
4     if value > threshold:
5         break
```

6 `print(value)`

`continue` はその回だけを飛ばし、`break` はループ全体を終了する。探索範囲を打ち切る処理では特に重要である。さらに Python には `for ... else` という構文もあり、`break` せずに最後まで回り切ったときだけ `else` 節が実行される。頻出ではないが、見かけても驚かないようにしておくといよい。

4.3.2.3 内包表記

短い変換や簡単なフィルタであれば、内包表記を使うとすっきりと構築できる。

```
1 values = [1, 2, 3, 4, 5]
2 squares = [x * x for x in values]
3 large_values = [x for x in values if x >= 3]
```

内包表記は便利だが、何段階もの条件や複雑な変換を 1 行に押し込むと読みにくくなる。初学者のうちは、読みやすさを失わない範囲で使う方がよい。長くなりそうなら、普通の `for` 文や補助関数へ戻した方が親切である。

4.4 関数とモジュール

4.4.1 関数を分ける理由

同じ処理を何度も書かないためだけでなく、処理の意味を分けるためにも関数は重要である。一例として、時刻を表す文字列から秒数を計算する関数を考える。

```
1 def hhmmss_to_seconds(hhmmss):
2     hour = int(hhmmss[0:2])
3     minute = int(hhmmss[2:4])
4     second = int(hhmmss[4:6])
5     return hour * 3600 + minute * 60 + second
```

この関数は短い、「文字列で与えられた時刻を秒へ変換する」という役割が明確である。解析コードでは、このような小さな関数が積み重なって全体の読みやすさを作る。

関数を切り出す利点は再利用だけではない。入力が何で、出力が何かをはっきりさせることで、テストしやすくなる。短い補助関数ほど後回しにされがちだが、解析の正しさはこうした小さな部品の積み重ねで決まることが多い。

4.4.1.1 関数の引数

関数を書くときは、引数の意味をはっきりさせることが重要である。Python では位置引数だけでなく、デフォルト値付き引数やキーワード引数も使える。

```
1 def histogram_path(name, suffix=".pdf"):
2     return f"{name}{suffix}"
3
```

```

4 print(histogram_path("value_hist"))
5 print(histogram_path("value_hist", suffix=".png"))

```

デフォルト引数を使うと、よく使う設定を省略できる一方で、必要なら上書きできる。コマンドライン引数と同じで、解析コードの柔軟性は「固定値をどこに置くか」で決まることが多い。関数でも同じ発想が使える。

ただし、可変オブジェクトをデフォルト引数に置くと予想外の共有が起きる。この罠は Python 公式チュートリアルでも強調されている典型例である。初学者の段階では、リストや辞書をデフォルトにしたくなったら一度立ち止まった方がよい。以下の書き方は避けるべきである。

```

1 def bad_append(value, buffer=[]):
2     buffer.append(value)
3     return buffer

```

この例では、`buffer` が呼び出しのたびに新しく作られるわけではない。そのため、前回の状態が次回にも残る。安全に書くなら `None` を使って関数内部で初期化する方がよい。初学者がつまづきやすいので、早めに知っておく価値が高い。

4.4.1.2 モジュールと import

別ファイルに書かれた関数を使うには次のように `import` する。

```

1 import math
2 import glob
3 import argparse

```

Python の解析では、標準ライブラリと外部ライブラリを自然に組み合わせる。どの名前がどこから来たものかを意識して読む習慣を付けておくと、大きなコードも追いやすくなる。

特に、同じ名前に見えても標準ライブラリと外部ライブラリで役割が大きく異なることがある。インポート文は単なるおまじないではなく、そのファイルが何に依存しているかを示す地図でもある。

4.4.1.3 自作モジュールへ分ける

解析が少し大きくなると、1 つのファイルへ全部を書くのはすぐに苦しくなる。よく使う関数や定数は別ファイルへ分け、モジュールとして読み込む方が見通しが良い。たとえば次のようなファイル構造で分けるとする。

```

analysis_project/
|-- constants.py
|-- physics.py
`-- main.py

```

このように分けたとき、`physics.py` 内の関数を `main.py` から呼び出す例を次に示す。

```

1 # physics.py

```

```

2 import numpy as np
3
4 def deg_to_rad(degrees):
5     return degrees * np.pi / 180.0
6
7
8 # main.py
9 import physics
10
11 theta = physics.deg_to_rad(45.0)

```

この形にしておくと、座標変換、時刻変換、定数定義のような再利用しやすい処理を 1 か所へまとめられる。後半で大きめの課題に進むと、コードを速くすることと同じくらい、どこに何を書くかを整理することが重要になる。

4.4.1.4 スクリプトとして実行する

Python ファイルは、他のファイルから読み込まれることもあれば、直接実行されることもある。その違いを区別する基本形が次である。

```

1 def main():
2     print("run")
3
4 if __name__ == "__main__":
5     main()

```

この形にしておくと、モジュールとして読み込んだときには関数だけを使え、直接実行したときだけ本体が動く。後半の解析スクリプトでも、この形を基本として採用すると整理しやすい。

4.5 保守と実践的な書き方

4.5.1 エラーは失敗ではなく情報である

初心者のうちは、エラーメッセージが出るとそこで止まってしまいがちである。しかし、トレースバックは「どこで何が起きたか」を教えてくれる重要な情報である。

たとえば `FileNotFoundError` ならファイルパスが違うかもしれないし、`ValueError` なら想定した形式でデータを読めていないかもしれない。大切なのは、英語の文章全体を恐れるのではなく、例外名と最後の数行を見ることである。

解析では、最初から一度もエラーを出さずに進む方が珍しい。問題はエラーが出ることでなく、エラーを無視したり、意味を確かめずに場当たりの直したりすることである。例外名、発生行、直前に読んでいたデータの形を確認する習慣を付けておくと、修正の質が安定する。

4.5.1.1 docstring とヘルプ

関数やモジュールの役割を後から思い出せるようにするには、短い説明を書いておくとうい。Python では次のように、関数定義の直後に文字列を書くと docstring（ドキュメント文字列）になる。

```
1 def read_table(path):
2     """空白区切りの表を読み、各行を辞書として返す。"""
3     ...
```

`help(read_table)` を実行すると、この説明を後から確認できる。コメントと同じように見えるが、実行時にも参照できる点異なる。解析コードが増えてくると、どの関数が何を返すのかを書いておくことの価値が大きくなる。

短い関数なら 1 行の docstring で十分だが、少し長い処理では複数行にして、少なくとも「何をするか」「主要な引数」「返り値」「起こり得る例外」を書いておく読みやすい。たとえば次のような形である。

```
1 def read_table(path):
2     """空白区切りの表を読み込む。
3
4     Args:
5         path: 入力ファイルへのパス。
6
7     Returns:
8         各行を辞書として格納したリスト。
9
10    Raises:
11        FileNotFoundError: ファイルが存在しない場合。
12    """
13     ...
```

流儀はいくつかあるが、最初の 1 行を「要約」、その後ろを詳細と考えると整理しやすい。特に、他人が `help()` やエディタ上の補完で関数を読むときは、最初の 1 行がそのまま関数の説明になることが多い。

4.5.1.2 名前付けと PEP 8

Python には PEP 8 という代表的なスタイルガイドがある。すべてを最初から覚える必要はないが、少なくとも変数名、関数名、空白の入れ方を次のようにそろえるだけでコードはかなり読みやすくなる。

```
1 MAX_EVENTS = 1000
2
3 def read_event_table(path):
4     total_count = 0
5     ...
```

一般には、変数名や関数名は `snake_case`、定数は大文字の `UPPER_CASE` にすることが多

い。さらに、演算子の前後に空白を入れる、1 行を詰め込みすぎない、といった基本だけでも効果は大きい。スタイルの統一は飾りではなく、バグを見つけやすくするための道具だと考える方が実践的である。

PEP 8 を最初から全部暗記する必要はないが、少なくとも次の規則は守る価値が高い。

- インデントはタブではなく 4 スペースにする
- 関数名や変数名は `snake_case`、クラス名は `CamelCase`、定数は `UPPER_CASE` にする
- 演算子 (`=`, `+`, `-` など) やカンマの後ろには空白を入れる
- トップレベルの関数定義やクラス定義の間には空行を入れて、かたまりを分ける
- 1 行を長くしすぎない。PEP 8 では 79 文字が目安だが、少なくとも横に流れすぎて読みにくい行は分割する
- `import` は標準ライブラリ、外部ライブラリ、自作モジュールの順にまとめる

この程度でも、見た目のばらつきはかなり減る。重要なのは「絶対に厳密であること」より、「同じプロジェクトの中で揃っていること」である。

4.5.1.3 マジックナンバーを避ける

コード中に 250 や 3600 のような数値が突然現れると、その値が何を表すのかを読む側が推測しなければならない。このような、説明なしに裸で現れる数値をマジックナンバーと呼ぶ。これを避けるには、次のように意味のある名前を持たせる。

```
1 NS_PER_SECOND = 1_000_000_000
2 COINCIDENCE_WINDOW_NS = 250
3
4 dt_ns = second_diff * NS_PER_SECOND + ns_diff
5 is_coincident = abs(dt_ns) <= COINCIDENCE_WINDOW_NS
```

250 をそのまま書くより、`COINCIDENCE_WINDOW_NS` と書いた方が意味が明確である。3600 なら `SECONDS_PER_HOUR`、86400 なら `SECONDS_PER_DAY` といった具合に、意味を名前へ出した方が読みやすい。

ただし、どんな値でもコード内定数にすればよいわけではない。物理定数や単位換算のように定義として固定したい値はコード内の定数に向いている。一方で、入力パス、出力先、閾値、利用する検出器番号のように実行条件として変わり得る値は、コマンドライン引数や設定ファイルへ出した方がよい。値の置き場所そのものが設計である、という意識を持つことが重要である。

4.5.1.4 例外処理

エラーは読むべき情報だが、予想される失敗を自分で受け止めたい場面もある。例えば、1 行だけ形式が壊れているデータを見つけて記録し、解析全体は続けたいことがある。そのときに使うのが次のような `try` と `except` の組み合わせである。

```
1 try:
```

```
2     value = float(text)
3 except ValueError:
4     print("数値へ変換できませんでした")
```

ただし、何でも `except:` で受けてしまうと、本来止まるべきバグまで隠してしまう。どの例外を受けるのかを明示し、何を記録して、何を諦めて、何を継続するのかを自分で決めることが重要である。

第 5 章 入出力とファイル

この章では、Python でファイルを開き、複数のファイルを集め、1 行ずつ読みながら値へ変換する基本を扱う。ここで重要なのは、特定のデータ形式に慣れることではなく、入力を見て必要な情報を取り出す流れを理解することである。

5.1 テキストファイルの入出力と探索

5.1.1 ファイルを開き、読み書きの基本を押さえる

Python でテキストファイルを読むときの基本は `open` である。さらに `with` を組み合わせると、読み終えた後に自動でファイルが閉じられる。テキストファイルを 1 行ずつ読む基本形は次のようになる。

```
1 with open(path, "r", encoding="utf-8") as f:
2     for line in f:
3         print(line.rstrip())
```

`read` でまとめて読む方法もあるが、大きなファイルでは 1 行ずつ読む方が安全である。まずはこの形に慣れておくとよい。

このとき、`line` には改行文字が含まれている点にも注意したい。表示だけなら `rstrip` で末尾を落とせば十分だが、区切り文字を扱うときには改行の有無が結果に影響することがある。テキスト処理では、見えていない文字を意識することが重要である。

5.1.1.1 pathlib でパスを扱う

ファイルパスを単なる文字列として扱うこともできるが、Python では次のように `pathlib.Path` を使うと見通しが良くなる。

```
1 from pathlib import Path
2
3 root = Path("data")
4 path = root / "2026" / "04" / "sample.txt"
5
6 print(path.exists())
7 print(path.name)
```

`/` 演算子でパスをつなげられるため、文字列連結より読みやすい。ファイル名だけ取りたい、親ディレクトリへ戻りたい、拡張子を変えたい、といった操作も自然に書ける。解析コードでは、パス操作を文字列で済ませるより安全ことが多い。`pathlib` を用いて新しく出力先を作る処理は次のようになる。

```

1 outdir = Path("output")
2 outdir.mkdir(parents=True, exist_ok=True)
3 outfile = outdir / "summary.txt"

```

ディレクトリがなければ作る、すでにあってもエラーにしない、という処理は解析コードで非常によく使う。こうした定型操作を安全に書ける点でも `pathlib` は便利である。

5.1.1.2 ファイルオブジェクトのメソッド

ファイルを開いた後には、`read`、`readline`、`write` などのメソッドが使える。Python 公式チュートリアルでも強調されているように、何をどの粒度で読むかでコードの性格が変わる。たとえば次のように記述する。

```

1 with open(path, "r", encoding="utf-8") as f:
2     first_line = f.readline()
3
4 with open("out.txt", "w", encoding="utf-8") as f:
5     f.write("hello\n")

```

`readline` は 1 行だけ確認したいとき、`write` は整形済み文字列をそのまま出したいときに便利である。大きな入力全体を一気に読む前に、最初の数行だけ確認するという使い方もよく行う。

5.1.1.3 複数のファイルを集める

ログや測定結果が複数ファイルに分かれている場合は、まず対象ファイルの一覧を作る必要がある。Python では `glob.glob` がよく使われる。基本的な探索方法は次の通りである。

```

1 import glob
2
3 paths = sorted(glob.glob("data/**/*.txt", recursive=True))

```

`glob.glob` はシェルのワイルドカードに近い感覚で使える。複数ファイルを扱う処理では、最初に見つけた件数や先頭数件のパスを表示して確認する習慣を付けておくとよい。

また、順序が重要な処理では `sorted` を付ける意味も大きい。ファイル探索の結果が環境依存の順序で返ると、処理結果やログの見え方が毎回変わり、デバッグしにくくなる。時系列や連番を扱う場面では、最初に順序を固定する方が安全である。

5.2 文字列データの解析と整形

5.2.1 文字列を値へ変換し、結果を書き出す

多くの表形式データは、1 行の中に複数の列が並ぶ形で保存されている。テキストとして読んだだけでは、まだ単なる文字列である。そこから列に分け、整数や浮動小数点数へ変換して初めて解析に使える。たとえば空白区切りの 1 行を分解する例は次のようになる。

```

1 line = "17_2026-04-04_12:34:56_15.2_OK"
2 columns = line.split()
3
4 record_id = int(columns[0])
5 date_text = columns[1]
6 time_text = columns[2]
7 value = float(columns[3])
8 status = columns[4]

```

ここで重要なのは、どの列をどの型で持つかを自分で決めることである。後で pandas を使う場合でも、この感覚を持っていると列の意味を見失いにくい。

文字列をどの段階で数値へ変換するかも設計の一部である。すぐ数値化してよい列もあれば、元の文字列表現を残しておいた方が確認しやすい列もある。読みやすい解析コードでは、どこでどの変換を行ったかが追えるようになっている。

5.2.1.1 ファイルへ書き出す

解析では、読むだけでなく結果を書き出す場面も多い。最も単純には、テキストとして 1 行ずつ書いていけばよい。

```

1 lines = ["run_id_count_mean", "1_120_3.42", "2_118_3.37"]
2
3 with open("summary.txt", "w", encoding="utf-8") as f:
4     for line in lines:
5         f.write(line + "\n")

```

"w" は新しく書くモード、"a" は追記モードである。既存ファイルを上書きしてよいのか、別名で保存すべきかは、実験ノートやレポート再現性にも関わる。出力ファイル名の設計は、解析の一部として考えるべきである。

5.2.1.2 文字列整形

結果を画面やファイルへ出すときには、数値を読みやすく整えることも大切である。Python では、以下のような f-string（フォーマット済み文字列リテラル）が最もよく使われる。

```

1 mean = 3.14159
2 sigma = 0.27182
3
4 text = f"mean={mean:.3f}, sigma={sigma:.3f}"
5 print(text)

```

`:.3f` は小数点以下 3 桁で表示するという意味である。解析結果をレポートへ貼る前に、このような整形で見やすくしておくと、図や本文との対応が取りやすい。

Python 公式チュートリアルでは、f-string のほかに `str.format` や手動整形も紹介されている。現在の実務では f-string が最も使いやすいが、古いコードを読むと別の書き方が出て

くることがある。複数の流儀があることは知っておきたい。

5.3 オブジェクトと構造化データの保存

5.3.1 JSON と Pickle / PKL

結果や設定を構造化して保存したいときは、JSON が便利である。テキストとして読めて、Python では標準ライブラリ `json` ですぐ扱える。基本的な保存と読み込みの記述は次のように行う。

```
1 import json
2
3 config = {"threshold": 3.5, "bins": 40}
4
5 with open("config.json", "w", encoding="utf-8") as f:
6     json.dump(config, f, indent=2)
7
8 with open("config.json", "r", encoding="utf-8") as f:
9     loaded_config = json.load(f)
```

大きな表形式データの本体を JSON で持つのは効率が悪いことも多いが、設定や要約結果を保存するには便利である。人間が読める形式と、Python が簡単に読める形式を両立しやすい。

5.3.1.1 Pickle / PKL

Python オブジェクトをそのまま保存したいときは、`pickle` もよく使われる。拡張子は慣習的に `.pkl` を使うことが多い。辞書、リスト、NumPy 配列、学習済みモデル、中間集計結果などを素早く退避するには便利である。処理の記述例を下に示す。

```
1 import pickle
2
3 result = {"mean": 3.14, "sigma": 0.27, "values": [1.0, 2.0, 3.0]}
4
5 with open("result.pkl", "wb") as f:
6     pickle.dump(result, f)
7
8 with open("result.pkl", "rb") as f:
9     loaded_result = pickle.load(f)
```

"wb" と "rb" の `b` はバイナリモードを意味する。JSON と違って人間には読めないが、Python では構造を保ったまま簡単に保存できる。ただし、`pickle` は Python 専用であり、他言語との受け渡しには向かない。また、信頼できない `.pkl` ファイルを `load` してはならない。共有用の正式な保存形式というより、自分の解析途中結果を手早く再利用するためのローカル形式と考える方が安全である。

ここでいう「危険」とは、単に壊れたファイルを読んでエラーになるという意味ではない。`pickle.load()` は、ファイルの内容に従って Python オブジェクトを復元する過程で任意のコード実行につながる可能性がある。したがって、メール添付や外部配布物として受け取った不明な `.pkl` を気軽に開いてはならない。自分で保存したもの、あるいは十分に信頼できる相手が生成したものだけを読む、という運用が必要である。

5.4 発展的なトピック

5.4.1 日付と時刻を数値として扱う

日付や時刻を含むデータでは、文字列のまま扱うより、比較や計算がしやすい形へ変換した方がよい。標準ライブラリの `datetime` は、そのための基本的な道具となる。実際のコードは次のようになる。

```
1 from datetime import datetime
2
3 text = "2026-04-04_12:34:56.123456"
4 dt = datetime.strptime(text, "%Y-%m-%d_%H:%M:%S.%f")
5 unix_time = dt.timestamp()
```

秒単位の差を見たいのか、より細かい時間差を見たいのかによって、どの単位で値を持つかは変わる。ただし、どの表現を選んでも、一貫して使うことが大切である。

`datetime` は便利だが、何でも `datetime` オブジェクトのまま持てばよいわけではない。差分計算を大量に繰り返すなら、途中で秒やナノ秒の数値へ変換した方が扱いやすいことも多い。人間に読みやすい表現と計算しやすい表現を分けて考えることが重要である。

さらに、タイムゾーンが関わるデータでは、見かけの時刻だけを比較すると誤解が生じる。研究用のログや観測データでは UTC を使うことが多いが、どの基準時刻を採用しているかを最初に確認する習慣を付けたい。

5.4.2 保存形式について: ベストプラクティスへのステップ

データをファイルに保存・読み出しする際、多くの初学者は何も考えずに `.csv` (CSV 形式) を使い続ける。しかし、データが数百万行に達すると、処理時間の大半が「ファイルの準備」に消えるようになる。

5.4.2.1 Step 1: テキスト形式 (CSV, JSON) の限界

テキスト形式は文字コード (例: UTF-8) を使って人間がそのまま読める形で保存する。CSV/TSV は非常に汎用性が高く、C++ や R、Excel など、どの言語でも読めるのが強みである。

問題点: コンピュータはディスク上の文字列 (例: 「1.2345」という 6 文字の羅列) を、計算に使える実数値にいちいち変換 (パース) しなければならない。データ量が数 GB に及ぶ

と、この「文字列から実数への変換処理」だけで何十分も待たされることになる。

5.4.2.2 Step 2: バイナリ形式への移行と、どれを選ぶかの判断基準

メモリ上の数値を人間には読めない生のバイト列としてそのままファイルに記録する「バイナリ形式」を使えば、読み込み時間は一瞬（ディスクの転送速度の限界）になる。

しかし、バイナリ形式にはいくつかデファクトスタンダードがあり、目的に合わせて選ばなければならない。

- **NPY / NPZ (NumPy 標準)**: NumPy だけで完結する小さな解析なら最も手軽。ただし、Python・NumPy 以外（C++や R など）からは読みづらく使い捨てになりがちである。
- **Pickle / PKL**: Python オブジェクトをそのまま保存できるため、中間結果や学習済みモデルの一時保存に便利である。ただし、Python 以外ではほぼ使えず、信頼できないファイルを読み込んでではない。共同研究の交換形式というより、ローカルな作業用キャッシュとして使うべき形式である。
- **HDF5**: 大規模な数値データをディレクトリの階層構造のように管理する形式。C/C++ (Geant4 など)、Fortran、R 等さまざまな言語の公式ライブラリが存在する。他のグループや別言語のプログラムにデータを渡す場合は、HDF5 を選ぶのが最も安全で汎用性が高い。
- **Parquet**: 列指向のバイナリフォーマット。後述する Pandas や Polars などの DataFrame をそのまま保存でき、特定の列だけを高速に読み込むことが得意。Hadoop や Spark などのビッグデータエコシステムでも標準的である。分析チーム内での巨大データのやり取りにはこれが現在の最強の選択肢である。

解析の初期段階ではデータの中身を目視確認するために CSV を使うことが多いが、大量のイベントデータを何回も読み直す段階になったら、**必ず最初に Parquet や HDF5 への変換スクリプトを書いてバイナリ化することが全体を高速化する第一歩である**。`.pkl` はその補助として、途中結果や作業用キャッシュを手元で素早く保存する用途に向いている。

Parquet は「保存形式」の名前であり、pyarrow はその形式を pandas から読み書きする代表的な実装の一つである。したがって、「Parquet を使う」と「pyarrow を入れる」は同じ話ではない。多くの環境では `read_parquet()` や `to_parquet()` の背後で pyarrow が使われるため、これが入っていないと Parquet を扱えないことがある。

第 6 章 NumPy による数値計算

NumPy は、Python で数値配列を扱うときの基本となるライブラリである。本章では、まず配列計算の考え方と ndarray の基本を確認し、その後で配列生成、演算、形の変更、集計、ブロードキャストを順に扱う。後半では、初学者がつまづきやすい点を整理し、最後に解析でどのように使うかを具体例で見る。

6.1 NumPy が扱う考え方

6.1.1 配列計算の基本概念

Python のリストは柔軟だが、数値配列として大量の値を処理するには向いていないことがある。NumPy は、同じ型の数値を多次元配列 ndarray として持ち、配列全体へまとめて演算をかけるためのライブラリである。最も基本的な NumPy 配列の定義と差分計算の例を示す。

```
1 import numpy as np
2
3 times = np.array([10, 15, 23, 41], dtype=np.int64)
4 deltas = times[1:] - times[:-1]
```

このような差分計算は、Python の for ループでも書けるが、NumPy では配列全体に対してまとめて書ける。後の図 10.1 に示すように、この差は速度にも大きく効く。

ここで重要なのは、NumPy が単に「速いリスト」ではないという点である。NumPy 配列は、同じ型の数値をまとめて持ち、一つの式で大量の要素を処理するための道具である。したがって、設計の発想も「1 要素ずつ何をするか」から「配列全体にどのような変換をかけるか」へ変わる。

6.1.2 ndarray と属性

NumPy の中心は ndarray である。配列を作ったら、まず shape、ndim、dtype を確認する癖を付けると混乱しにくい。

```
1 import numpy as np
2
3 a = np.array([[1.0, 2.0, 3.0],
4               [4.0, 5.0, 6.0]])
5
6 print(a.shape) # (2, 3)
7 print(a.ndim) # 2
```

```
8 print(a.dtype) # float64
9 print(a.size) # 6
10 print(a.itemsize) # 8
```

shape は各軸の長さ、ndim は次元数、dtype は要素型、size は総要素数、itemsize は 1 要素あたりのバイト数である。NumPy の初学者は「配列の中身」ばかり見がちだが、実際には shape と dtype を見れば、多くの不具合の原因を早い段階で絞り込める。

```
(2, 3)
2
float64
6
8
```

出力を見れば、この配列が「2 行 3 列の float64 配列」であることがすぐ分かる。NumPy のトラブルの多くは、この 5 行のどれかを見れば原因に近づける。

6.1.3 NumPy が速くなりやすい理由

NumPy が速くなりやすい理由は、単に「ライブラリだから」ではない。主な理由は三つある。

1. 数値を連続したメモリにまとめて持てること
2. Python レベルのループを減らせること
3. 内部で低レベルに最適化された処理を使えること

特に重要なのは、Python で 1 要素ずつ処理する回数が減ることである。大きなデータでは、この差がそのまま処理時間の差になる。

もう一つの利点は、式がそのまま数学的な記述に近づくことである。たとえば差分、閾値判定、正規化、平均、分散といった操作は、配列演算で書いた方が意図が明確になることが多い。読みやすさと速度が同時に改善する場面は少なくない。

6.2 最小限のコード例

6.2.1 配列を作る

NumPy では、最初に「どの形の配列を、どの型で作るのか」を決める。np.array 以外にも、一定値で埋める、等間隔の値を作る、といった入口が多い。

```
1 a = np.array([1, 2, 3], dtype=np.float64)
2 zeros = np.zeros((2, 3))
3 ones = np.ones((2, 3))
4 tmp = np.empty((2, 3))
5 grid = np.arange(0, 10, 2)
6 x = np.linspace(0.0, 1.0, 5)
```

```

7
8 print(a)
9 print(zeros.shape)
10 print(grid)
11 print(x)

[1. 2. 3.]
(2, 3)
[0 2 4 6 8]
[0. 0.25 0.5 0.75 1. ]

```

`np.array` は既存の Python リストなどから配列を作る。`np.zeros` と `np.ones` は初期値が明確なので安全である。`np.arange` は整数刻みの列、`np.linspace` は区間を等分した実数列を作る。解析では、ビン中心、時間軸、周波数軸を作る場面で `linspace` をよく使う。

`zeros.shape` が `(2, 3)` なので、`zeros` と `ones` は 2 行 3 列の配列である。`grid` は 0 から 8 まで 2 刻み、`x` は 0 から 1 を 5 点で等分した座標列である。`np.empty` はメモリだけ確保して値を初期化しないため高速だが、初期値は未定である。後で必ず全要素を上書きする場合にだけ使う方が安全である。

6.2.2 よく使う操作

解析でよく使う NumPy の操作として、条件に合う要素だけを取り出すフィルタ、順序をそろえるソート、隣り合う要素の差を取る差分、分布を数えるヒストグラムがある。これらは別々の操作に見えるが、解析ではまとめて使うことが多い。

```

1 times = np.array([41, 10, 23, 15, 42])
2
3 selected = times[times > 20]
4 sorted_times = np.sort(times)
5 deltas = np.diff(sorted_times)
6 hist, edges = np.histogram(times, bins=[0, 20, 40, 60])
7
8 print(selected)
9 print(sorted_times)
10 print(deltas)
11 print(hist)
12 print(edges)

[41 23 42]
[10 15 23 41 42]
[ 5  8 18 1]
[2 1 2]
[ 0 20 40 60]

```

この例で行っていることは次のとおりである。

- フィルタ: `times > 20` は `[True, False, True, False, True]` に相当する条件配列を作

り、その位置にある要素だけを `selected` として取り出す。

- ソート: `np.sort(times)` は値を昇順に並べる。観測時刻やエネルギーのように順序が意味を持つ量では、後続の処理の前に並べ替えることが多い。
- 差分: `np.diff(sorted_times)` は隣り合う値の差を返す。返る配列の長さは元の配列より 1 つ短い。イベント時刻が昇順に並んでいれば、これは到着間隔として読める。
- ヒストグラム: `np.histogram(times, bins=[0, 20, 40, 60])` は、区間ごとの個数を `hist`、区間境界を `edges` として返す。この例では `[0, 20)` に 2 個、`[20, 40)` に 1 個、`[40, 60]` に 2 個入っている。

ブール配列によるフィルタは NumPy の基本である。条件式の結果をそのまま添字にできるので、`(times > 20) & (times < 40)` のような複数条件も簡潔に書ける。一方で、差分は順序に依存するので、時刻列のようなデータでは `np.diff(times)` をいきなり使わず、まずソートが必要かどうかを確認した方が安全である。

6.2.3 要素ごとの演算とユニバーサル関数

NumPy の演算子や多くの関数は、原則として要素ごとに働く。NumPy の公式ドキュメントでは、`exp`、`sqrt`、`sin` のような要素ごと関数をユニバーサル関数 (ufunc) と呼んでいる。

```
1 a = np.array([1.0, 2.0, 3.0])
2 b = np.array([10.0, 20.0, 30.0])
3
4 print(a + b)
5 print(a * 2)
6 print(a ** 2)
7 print(np.exp(a))
8 print(np.sqrt(a))
```

```
[11. 22. 33.]
[2. 4. 6.]
[1. 4. 9.]
[ 2.71828183  7.3890561 20.08553692]
[1. 1.41421356 1.73205081]
```

配列どうしの四則演算は、形がそろっていれば要素ごとに実行される。`np.exp(a)` や `np.sqrt(a)` も同様である。関数が 1 要素に対して何をするかを知っていれば、その関数が配列全体へどう作用するかも追やすい。

6.2.4 集計と axis

NumPy の集計関数は、配列全体に対しても、特定の軸ごとにも使える。`axis` の意味は初学者がつかずきやすいので、コードで確認するのがよい。

```
1 matrix = np.array([[1.0, 2.0, 3.0],
2                     [4.0, 5.0, 6.0]])
3
```

```

4 total = matrix.sum()
5 col_sum = matrix.sum(axis=0)
6 row_sum = matrix.sum(axis=1)
7 col_max = matrix.max(axis=0)
8
9 print(total)
10 print(col_sum)
11 print(row_sum)
12 print(col_max)

```

```

21.0
[5. 7. 9.]
[ 6. 15.]
[4. 5. 6.]

```

total は全要素の和である。axis=0 は「0 軸をつぶす」、すなわち各列ごとの集計になり、axis=1 は各行ごとの集計になる。言葉だけで覚えるより、「どの軸が消えて、何が残るか」を shape で確認する方が理解しやすい。

6.2.5 添字、スライス、条件抽出

NumPy の添字は Python リストと似ているが、多次元配列とブール配列が自然に使える点が重要である。

```

1 times = np.array([10, 15, 23, 41, 42])
2 print(times[0])
3 print(times[-1])
4 print(times[1:4])
5 print(times[times > 20])
6
7 grid = np.arange(12).reshape(3, 4)
8 print(grid[1, 2])
9 print(grid[:, 1])
10 print(grid[1, :])

```

```

10
42
[15 23 41]
[23 41 42]
6
[1 5 9]
[4 5 6 7]

```

times[1:4] は 1 番目から 3 番目までを取り出す。grid[:, 1] は全行の 2 列目、grid[1, :] は 2 行目全体である。多次元配列に対して for row in grid: と書くと、最初の軸に沿って 1 行ずつ取り出される。反復自体はできるが、NumPy を使うなら、まず配列演算で書けないかを先に考える方が自然である。

6.2.6 形を変える

NumPy を使い始めたら、配列の `shape` を意識することが重要である。1 次元配列なのか、行列なのか、列ベクトルとして扱いたいのかで、同じ演算でも結果が変わる。

```
1 a = np.arange(12)
2 matrix = a.reshape(3, 4)
3 flat = matrix.ravel()
4 transposed = matrix.T
5 column = np.array([1.0, 2.0, 3.0])[:, None]
6
7 print(matrix.shape)
8 print(matrix)
9 print(flat)
10 print(transposed)
11 print(column.shape)
12 print(column)
```

```
(3, 4)
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
(3, 1)
[[1.]
 [2.]
 [3.]]
```

`reshape` は形を変える。`ravel` は可能ならビューとして 1 次元へ潰す。`T` は転置である。`[:, None]` は新しい軸を 1 本追加し、1 次元配列を列ベクトルのように扱う書き方である。後で述べるブロードキャストでは、この 1 本の軸追加が非常に重要になる。

6.2.7 配列をつなぐ、分ける

複数の配列をつないだり、1 つの配列を分割したりする操作もよく使う。特に、前処理済みデータを縦に積む、特徴量を列方向に並べる、といった場面で必要になる。

```
1 x = np.array([1, 2, 3])
2 y = np.array([4, 5, 6])
3
4 joined = np.concatenate([x, y])
5 stacked = np.stack([x, y], axis=0)
6
7 matrix = np.arange(12).reshape(3, 4)
8 left, right = np.split(matrix, 2, axis=1)
```

```

9
10 print(joined)
11 print(stacked.shape)
12 print(stacked)
13 print(left)
14 print(right)

```

```

[1 2 3 4 5 6]
(2, 3)
[[1 2 3]
 [4 5 6]]
[[0 1]
 [4 5]
 [8 9]]
[[ 2 3]
 [ 6 7]
 [10 11]]

```

`concatenate` は既存の軸に沿ってつなぐ。`stack` は新しい軸を追加して積む。`split` は指定した軸に沿って分割する。見た目が似ていても結果の `shape` は異なるので、実行後に必ず `shape` を見る方がよい。

6.2.8 形とブロードキャスト

NumPy では、配列の形状をそろえなくても、自動拡張の規則に従って演算できる場合がある。この規則をブロードキャストという。

```

1 x = np.array([1.0, 2.0, 3.0])
2 y = np.array([10.0, 20.0, 30.0])
3
4 z = x + y
5 matrix = x[:, None] + y[None, :]

```

後半の `matrix` では、1 次元配列を軸付きで広げることで、3 行 3 列の和の表を作っている。こうした自動拡張の規則をブロードキャストという。最初は少し難しいが、二つの配列の全組合せを調べるような場面で非常に強力である。上記の操作によるブロードキャストの結果は次のようになる。

```

z =
[11. 22. 33.]

matrix =
[[11. 21. 31.]
 [12. 22. 32.]
 [13. 23. 33.]]

```

この結果から、`z` は要素ごとの和、`matrix` は全組合せの表であることが分かる。便利ではあるが、配列サイズが大きいと一気にメモリを消費するので、実データでは常に形と大きさ

を見積もる必要がある。

6.2.9 統計量とヒストグラム

NumPy には、平均、分散、最小値、最大値、ヒストグラムなど、解析の基本となる集計関数がそろっている。代表的な統計量とヒストグラムを計算する例を下に示す。

```
1 values = np.array([1.0, 2.0, 2.0, 3.0, 4.0])
2 mean = values.mean()
3 std = values.std(ddof=0)
4 hist, edges = np.histogram(values, bins=[0.0, 2.0, 4.0, 6.0])
5
6 print(f"mean={mean:.2f}")
7 print(f"std={std:.2f}")
8 print(hist)
9 print(edges)
```

`np.histogram` は図を描かずに分布だけを数えたいときに便利である。ビン境界 `edges` も同時に返るため、後で自分で曲線を重ねたり、複数データを比較したりしやすい。可視化と数え上げを分けて考えると、処理の見通しが良くなる。実際にこれらの統計量を出力した結果は次のようになる。

```
mean = 2.40
std = 1.02
[1 3 1]
[0. 2. 4. 6.]
```

この例では、`[0, 2)` に 1 個、`[2, 4)` に 3 個、`[4, 6]` に 1 個入っている。平均や標準偏差だけでは分布の形は分からず、ヒストグラムだけでは条件間比較がしにくい。数値要約と分布表示を組み合わせる読むことが、データ解析では基本になる。

6.3 よくある落とし穴

6.3.1 演算子の意味を取り違える

`*` は要素ごとの積であり、行列積ではない。行列積が欲しいときは `@` か `np.dot` を使う。

```
1 A = np.array([[1.0, 2.0],
2               [3.0, 4.0]])
3 B = np.array([[5.0, 6.0],
4               [7.0, 8.0]])
5
6 print(A * B)
7 print(A @ B)
```

この違いを理解せずに書くと、コードは動いているのに意味だけが間違う、という厄介な状態になる。特に線形代数や座標変換では注意が必要である。

6.3.2 axis と条件式

axis の意味は、初学者が最初に混乱しやすい箇所である。axis=0 は列ごとの集計、axis=1 は行ごとの集計になる。また、配列どうしの条件結合では and や or ではなく、& や | を使う。

```
1 matrix = np.array([[1, 2, 3],
2                     [4, 5, 6]])
3 print(matrix.sum(axis=0))
4 print(matrix.sum(axis=1))
5
6 times = np.array([10, 15, 23, 41, 42])
7 mask = (times > 10) & (times < 40)
8 print(mask)
9 print(times[mask])
```

```
[5 7 9]
[ 6 15]
[False True True False False]
[15 23]
```

times > 10 & times < 40 のように括弧を省略すると、優先順位の違いで意図しない式になる。NumPy の条件式では「各比較を括弧で囲む」を基本にした方が安全である。

6.3.3 dtype と np.empty

NumPy では、混在した型を入れると配列全体の dtype が自動的に決まる。この挙動を意識していないと、数値計算のつもりが文字列配列になっていた、ということが起こる。

```
1 a = np.array([1, 2.5])
2 b = np.array([1, "2"])
3 tmp = np.empty(5)
4
5 print(a.dtype)
6 print(b.dtype.kind)
7 print(np.issubdtype(a.dtype, np.number))
8 print(np.issubdtype(b.dtype, np.number))
9 print(tmp.shape)
```

```
float64
U
True
False
(5,)
```

a は浮動小数点へそろえられる。これに対して b は dtype.kind == "U"、つまり Unicode 文字列の配列になっており、もはや数値計算用ではない。tmp は 0 で初期化されないで、

値を見る前に必ず全要素を書き込む必要がある。配列生成直後に dtype と初期値の有無を確認することが重要である。

6.3.4 コピーとビュー

NumPy は便利だが、配列をむやみにコピーすると逆に重くなる。一方で、意図せずビューを通じて元配列を書き換えてしまうこともある。この違いを理解せずに値を書き換えると、意図しない副作用や余分なメモリ使用を招く。

```
1 a = np.arange(6)
2
3 view = a[1:4]
4 view[:] = -1
5
6 copy = a[[0, 2, 4]]
7 copy[:] = 99
8
9 safe = a.copy()
```

この例では、スライスで作った view は元の a とデータを共有するので、view[:] = -1 とすると a も変わる。これに対して、整数配列やブール配列による取り出しは新しい配列を返すので、copy[:] = 99 としても元の a は変わらない。reshape や ravel も可能ならビューを返すため、「独立した配列が欲しい」ときは copy() を明示した方が安全である。

```
a after view = [ 0 -1 -1 -1 4 5]
copy = [99 99 99]
safe = [ 0 -1 -1 -1 4 5]
```

この出力では、view を書き換えた時点で元配列 a が変化しているのに対し、copy は独立していることが見て取れる。

6.3.5 浮動小数点数の比較

浮動小数点数の丸め誤差にも注意が必要である。理論上は等しいはずの値でも、計算手順によって最後の数桁がずれることがある。そのため、a == b のような直接比較より、許容誤差を設けた比較を使う方が安全なことが多い。

```
1 a = 0.1 + 0.2
2 print(a == 0.3)
3 print(np.isclose(a, 0.3))
```

```
False
True
```

6.4 解析へのつながり

6.4.1 解析で NumPy を使うときの視点

大規模データでは、型を適切に選ばないとメモリを無駄に使う。たとえば、識別子や時刻差なら `int64` が自然だが、カテゴリ番号が小さい範囲なら `int16` や `int32` でも十分なことがある。大規模データでは、この積み重ねが効いてくる。

また、平均や分散だけでなく、差分、閾値判定、並べ替え、ヒストグラムまで NumPy の中で一貫して処理できると、コードの往復が減り、解析の流れを追いやすくなる。

6.4.2 差分、ソート、ヒストグラム

解析では、まず時刻順に並べ、次に差分を取り、その後で条件を掛けて分布を見る、という流れがよく現れる。NumPy ではこの流れをほぼそのまま式として書ける。

```
1 order = np.argsort(times)
2 sorted_times = times[order]
3 deltas = np.diff(sorted_times)
4 mask = (deltas >= 0) & (deltas <= 250)
5 hist, edges = np.histogram(deltas[mask], bins=50)
6 peak_bin = hist.argmax()
```

`np.argsort` は並べ替え順の添字を返す。`np.diff` は隣接差分である。`hist.argmax()` は最も高いビンの位置を返す。イベント時刻、スペクトル、波形特徴量のいずれでも、この種の処理は頻出である。

```
sorted_times = [10 15 23 41 42]
deltas = [ 5 8 18 1]
peak_bin = 0
```

6.4.3 線形代数の入口

NumPy は単なる配列操作だけでなく、線形代数の入口としても重要である。小規模な連立方程式や座標変換なら、まず NumPy の `linalg` を使うだけで十分ことが多い。

```
1 A = np.array([[1.0, 1.0],
2               [1.0, -1.0]])
3 b = np.array([3.0, 1.0])
4 x = np.linalg.solve(A, b)
```

校正係数の決定、座標変換、最小二乗法の前処理などでは、こうした線形代数の操作が自然に現れる。大規模で高度な最適化は後の SciPy 章で扱うが、入口は NumPy で十分に理解できる。

6.4.3.1 Python リストを用いた数値計算の限界

Python の標準リストは非常に柔軟であり、どんな型のものでも放り込める。しかしいざ大量の数値を計算しようとする、この「なんでも入る」という性質が最大のネックとなる。避けるべきリストを用いた数値計算の例を示す。

```
1 # 100万個のリストを作る
2 py_list = list(range(1000000))
3
4 # すべての要素を2倍にする。forループと内包表記が必要
5 doubled_list = [x * 2 for x in py_list]
```

一見普通に動くが、Python のリストは「あちこちに散らばっている多様なサイズの箱へのポインタ（地図）」の集まりである。CPU は一つの値を計算するたびに「この値は何型か」「どこに置かれているか」を確認しなければならず、絶望的に遅い。

6.4.3.2 NumPy のベクタライズ化

これに対し NumPy デイメンショナル配列 (ndarray) は、同じ型の数値 (int64 や float64 など) をメモリ上に「すき間なく一列に並べた団地」として構成される。型確認が不要で、C 言語レベルの一括演算回路が直結する。NumPy 配列へ変換してから一括計算するコードは次の通りである。

```
1 import numpy as np
2
3 # 1. リストから NumPy アレイへの変換（メモリ上にきれいに並べ直す）
4 np_array = np.array(py_list)
5
6 # 2. アレイはベクトル演算が可能（for文を書かなくても全要素を一気に計算できる）
7 doubled_array = np_array * 2
8
9 # 3. アレイからリストへ戻すことも可能だが、重いので通常は最後だけにする
10 back_to_list = doubled_array.tolist()
```

6.4.3.3 Pandas DataFrame との変換

後述する Pandas DataFrame は、この NumPy 配列を「列」として束ねた拡張版である。分析の途中で NumPy 独自の計算関数を使いたい場合、DataFrame を NumPy に戻すことができる。Pandas の DataFrame・Series と NumPy アレイを相互変換するには、主に次のように書く。

```
1 import pandas as pd
2
3 # NumPy アレイから pandas の DataFrame（二次元）に変換
4 df = pd.DataFrame(np_array, columns=["Value"])
5
6 # Pandas のデータから NumPy アレイに戻す
7 # \code{.to_numpy()} は背後で持っているC言語配列を直接取り出すため一瞬で終わる
```

```
8 values_array = df["Value"].to_numpy()
```

6.4.4 公式資料の使い方

NumPy の基礎を復習するときは、[NumPy quickstart](#) の順序が参考になる。特に、配列生成、添字とスライス、`shape` の変更、ブロードキャスト、コピーとビューは何度も見返す価値がある。本章では解析に必要な文脈を足して説明しているので、まず本章で流れをつかみ、細かな API 名を確認したいときに `quickstart` を引く、という使い方が自然である。

第7章 pandas による大規模データ解析

7.1 DataFrame の考え方

pandas の中心にある DataFrame は、列名付きの表である。観測ログや測定結果のように「行が記録、列が属性」という形のデータには特に向いている。

```
1 import pandas as pd
2
3 df = pd.read_csv(path, sep=r"\s+", comment="#", header=None)
```

DataFrame の利点は、列ごとに意味を持たせながら、まとめて処理を書けることにある。

DataFrame を使うと、「何列目か」を覚える必要が減り、列名を通して処理の意味が見えやすくなる。特に、ファイル読み込み、列変換、フィルタ、集計、書き出しまでを一つの流れて扱える点は大きい。解析の途中で確認用の表を少し表示したいときにも便利である。

DataFrame の基本操作は、Python 公式チュートリアル「データ構造」章に相当する考え方を、表形式へ拡張したものだと考えると理解しやすい。リストや辞書を知っていると、列名によるアクセスや行集合に対するフィルタの意味が見えやすい。

```
1 print(df.head())
2 print(df.dtypes)
```

大きなデータを読むときほど、最初に head() と dtypes を確認する価値がある。列名は正しいか、型は意図通りか、欠損はないかを早い段階で見ただけで、多くのバグを防げる。

dtypes に現れる int64 は整数、float64 は浮動小数点数、object は文字列や混在型として扱われていることが多い。さらに、時刻へ変換した列は datetime64[ns] のような表示になる。ここで重要なのは、「数値に見えるから数値型だろう」と決めつけず、実際の型を確認することである。特に欠損値が混ざると整数列が浮動小数点数へ変わったり、pandas の nullable 型である Int64 を使った方が自然になったりする。

7.1.1 列選択、並べ替え、集約

pandas を使い始めたら、まず列選択と並べ替えを自然に書けるようになりたい。

```
1 selected = df[["date", "value"]]
2 sorted_df = df.sort_values("value")
3 top10 = sorted_df.tail(10)
```

ここで重要なのは、行ごとの処理より先に、表全体の形を整えることである。必要列だけを取り出し、順序をそろえ、不要な行を除く。この前処理がきれいだと、その後の集計も読

みやすくなる。

```
1 summary = df.groupby("category")["value"].mean()
```

groupby は、カテゴリごとに平均や件数を出したいときの基本である。研究データでは、検出器ごと、条件ごと、日ごとのように、同じ操作を複数群へ適用したい場面が多い。手書きのループよりも、列の意味を保ったまま集約できる点が pandas の強みである。

```
category
A 3.41
B 3.08
C 2.95
Name: value, dtype: float64
```

この結果例では、カテゴリごとの平均値をすぐ比較できる。必要に応じて件数や標準偏差も一緒に出せば、単に平均が違うのか、ばらつきやサンプル数も違うのかを確認できる。

7.1.2 pandas が速くなりやすい理由

pandas が速くなりやすいのは、列単位でデータを処理し、内部でベクトル化された操作を使えるからである。Python の辞書リストを 1 行ずつ回して書くより、列としてまとめて扱う方が速く、コードも短くなりやすい。

ただし、すべての場面で pandas が最適とは限らない。行ごとに複雑な条件分岐を大量に行う場合や、データ構造自体を細かく制御したい場合には、NumPy や自作構造の方が向くこともある。

したがって、「とりあえず pandas を使う」よりも、「列として扱える問題かどうか」を考える方が重要である。表として整理できるなら pandas は強力だが、近傍探索や状態遷移のように行どうしの関係が複雑になると、別の設計の方が自然なこともある。

7.2 読み込みから書き出しまでの基本操作

read_csv を使うときは、必要な列だけ読む、型を指定する、コメント行を飛ばす、といった工夫が重要である。

```
1 df = pd.read_csv(
2     path,
3     compression="gzip",
4     sep=r"\s+",
5     comment="#",
6     header=None,
7     usecols=[0, 1, 2, 3],
8     names=["id", "date", "time", "value"],
9 )
```

このように列を絞るだけでも、読み込み時間とメモリ消費はかなり変わる。

さらに、dtype を指定しておくで、読み込み後の型変換を減らせる。列数や行数が大きい

データでは、どの列が本当に必要かを考えるだけで、後続の処理全体が軽くなる。読み込みは解析の最初の一步だが、同時に最適化の出発点でもある。

たとえば `dtype={"id": "int64", "value": "float64"}` のように書けば、列ごとの型を明示できる。型を固定しておくのと、読み込むたびに推定結果が揺れるのを防げるだけでなく、メモリ消費の見積もりもしやすくなる。

7.2.1 CSV を毎回読み直さず Parquet へ変換する

前章で述べたように、巨大なテキストファイルを何度も `read_csv` するのは無駄が大きい。列構造が固まったら、一度だけ CSV から DataFrame を作り、その後は Parquet として保存して再利用する方が実務的である。

```
1 raw_df = pd.read_csv(  
2     "events.csv.gz",  
3     sep=r"\s+",  
4     comment="#",  
5     header=None,  
6     names=["id", "date", "time", "value"],  
7 )  
8 raw_df.to_parquet("events.parquet")  
9  
10 df = pd.read_parquet("events.parquet", columns=["id", "time", "value"])
```

`read_parquet` なら、文字列のパースを毎回やり直さずに済み、必要列だけを読むこともできる。日々の解析では、この「最初に1回だけテキストを整理して、以後は Parquet を読む」という流れを作るだけで、待ち時間が大きく減る。なお、Parquet の入出力には `pyarrow` などのバックエンドが必要なことが多い。

ここで区別したいのは、Parquet がファイル形式の名前であり、`pyarrow` はその読み書きを実装するライブラリだという点である。したがって、`to_parquet()` や `read_parquet()` が失敗したときは、「Parquet が壊れている」のではなく、「必要な実装がまだ入っていない」ことがある。

```
1 df["ratio"] = df["value"] / df["value"].mean()  
2 selected = df[df["ratio"] > 1.5]
```

このように、列同士の演算をそのまま式として書ける点が `pandas` の強みである。行ごとの `for` ループを書く前に、列単位で表現できないかを考える習慣を持つとよい。

7.2.2 欠損値と型の扱い

現実のデータでは、すべての行がきれいに埋まっているとは限らない。空欄や壊れた値が混ざると、`pandas` では `NaN` として表現されることが多い。

```
1 missing_mask = df["value"].isna()  
2 clean_df = df.dropna(subset=["value"])
```

欠損値をどう扱うかは、単なる実装問題ではなく解析方針そのものである。除外してよいのか、別扱いすべきか、補完してよいのかを、物理的な意味と合わせて考える必要がある。型の自動推定に任せきりにせず、重要な列の型は自分で確認したい。

NaN は「数値ではない」という意味だが、pandas では実質的に「欠損」を表す印としてよく使われる。注意すべきなのは、NaN == NaN が False になることである。したがって、欠損判定には == ではなく isna() や notna() を使う。また、整数列へ NaN が入ると浮動小数点数へ変わることがあるので、列型が意図せず変わっていないかも確認する必要がある。

7.2.3 結合と時系列処理

複数の表をまとめたり、時刻順に並べ替えたりする処理も pandas の重要な役割である。

```
1 combined = pd.concat([df1, df2], ignore_index=True)
2 combined = combined.sort_values("time")
```

複数ファイルを月単位や日単位で読んで、最後にまとめるという流れは現実の解析でよく現れる。どの段階で結合するか、結合後にどの列で並べ替えるかを明確にしておくと、ファイルまたぎの不具合を減らしやすい。

```
1 combined["time"] = pd.to_datetime(combined["date"] + "_" + combined["time"])
2 combined = combined.sort_values("time")
```

文字列のままでは、見た目には時刻に見えても、差分計算や順序比較には向かないことがある。早い段階で型を変えておくと、その後の処理が自然になる。

7.2.4 DataFrame を TeX で出力する

解析結果として集約した表などを LaTeX のレポートに直接貼り付けたい場合は、手作業で打ち直すのではなく to_latex() を用いると便利である。

```
1 # pandas のバージョンにより引数の推奨が変わる場合があるが、一般的な利用法
2 tex_table = summary.reset_index().to_latex(
3     index=False,
4     caption="カテゴリごとの平均値",
5     label="tab:category_mean",
6     float_format="%.3f", # 小数点以下3桁に整える
7     escape=False,
8 )
9
10 with open("tables/category_mean.tex", "w", encoding="utf-8") as f:
11     f.write(tex_table)
```

ここで index=False は左端の連番インデックスを出力しないための指定であり、reset_index() は groupby 結果の添字を普通の列へ戻すための操作である。さらに escape=False は、セル中にすでに TeX 記法が入っている場合にそのまま通したいときだけ使う。通常の文字列だけなら既定値のままエスケープさせた方が安全である。

これをテキストファイルへ保存し、本ドキュメント側でそのファイルを \input して読み

込めば、解析スクリプトからレポートまでのパイプラインが完成する。表を手で打ち直さないことは、再現性の面でも大きい。

7.3 行処理と並列化の判断

Pandas の DataFrame は数百万行のデータを保持できるが、各行に対して複雑な文字列処理や条件判定を加えようとした時、コードの書き方一つで実行時間が数百分から数秒へと劇的に変わる。

7.3.1 改善ステップ

7.3.1.1 Step 1: 絶対に避けるべき for ループ (iterrows の限界)

Python 初学者が最も陥りやすい罠が、「DataFrame の行を 1 つずつ取り出して処理する」という直感的な書き方である。

```
1 def heavy_computation(val):
2     return val ** 2
3
4 # 絶望的に遅い：1行ずつPython空間でオブジェクトが作られ、評価される
5 results = []
6 for index, row in df.iterrows():
7     results.append(heavy_computation(row["value"]))
8 df["value_sq"] = results
```

このコードは 10 万行を超えたあたりから全く終わらなくなる。Pandas がデータフレームとしてデータを一括管理している利点を全て打ち消し、Python の重い関数呼び出しを数百万回繰り返しているからである。さらに iterrows() は各行を毎回 Series として作り直すため、列ごとの型情報が崩れやすく、オブジェクト生成コストも大きい。どうしても行ループが必要なら itertuples() の方が軽いことが多いが、基本方針は「まず行ループ自体を避ける」である。

7.3.1.2 Step 2: Pandas ネイティブな apply() への移行

行ごとに処理を適用したい場合、Pandas が用意している apply() メソッドを使うと、少なくともコードを pandas の文脈に保ったまま書ける。axis=1 は「列方向に見て、各行へ関数を適用する」という意味であり、axis=0 なら各列に適用する。ここを取り違えると、意図しない単位で計算してしまう。

```
1 # for 文を書かずに、関数そのものを列に適用（マッピング）する
2 df["value_sq"] = df.apply(lambda row: heavy_computation(row["value"]), axis=1)
```

ただし、axis=1 を付けた apply() も各行に対して Python 関数を呼ぶため、列演算ほど速いわけではない。つまり、apply() は「for 文を隠した便利関数」であって、万能の高速化ではない。上の例のように単に二乗したいだけなら、実際には次のように列演算で書く方が自

然で速い。

```
df["value_sq"] = df["value"] ** 2
```

`apply()` は「列演算だけでは書きにくい、まだ pandas の文脈の中で処理したい」場合の中間手段として理解の方が正確である。まず列演算、次に `apply()`、それでも重ければ並列化、という順で判断すると迷いにくい。

7.3.1.3 Step 3: 並列化とボトルネックの判断 (Pandarallel)

しかし、`apply()` はどれだけ行数があっても「CPU の 1 つのコア」しか使わない（シングルスレッド）。本当に計算が重い処理の場合、`apply` を使っても処理が長引いてしまう。ここで初めて、マルチコアを用いた「並列化（マルチプロセッシング）」が選択肢に入る。

```
1 from pandarallel import pandarallel
2
3 # 並列環境の初期化。プログレスバーを表示する設定をオンにしている
4 pandarallel.initialize(progress_bar=True)
5
6 # apply を parallel_apply に置き換えるだけで、全CPUコアに処理が分散される
7 df["value_sq"] = df.parallel_apply(lambda row: heavy_computation(row["value"]),
    axis=1)
```

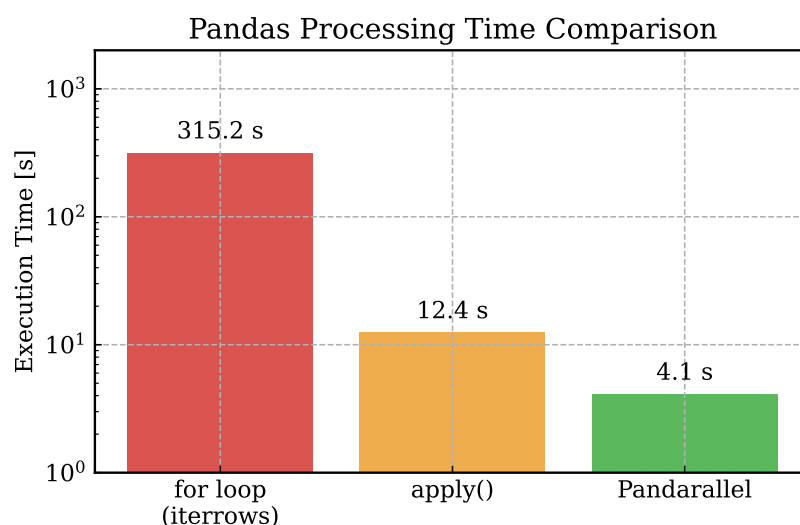


図 7.1 同じ重い計算処理を、Python のループ、`apply` 関数、`Pandarallel` で行った際の処理時間の激変の様子。

図 7.1 を見ると明らかなように、手段の選定によって処理時間は 2 桁変わる。

`pandarallel` は、`DataFrame` を複数プロセスへ分割して送り、`apply()` に近い書き方のまま並列化するためのライブラリである。つまり、「既存の pandas コードを大きく壊さずに、`parallel_apply()` へ置き換えて試せる」ことが利点である。一方で、背後ではプロセス生成やデータコピーが起きるため、軽い処理ではほとんど得をしない。

pandarallel は Linux 系環境での利用が比較的安定している一方で、ネイティブの Windows 環境では設定や挙動で悩みやすいことがある。Windows で本書の流れを再現するなら、前章で触れた WSL を使う方が混乱しにくい。

並列化の罠（判断基準）： 並列化は魔法の杖ではない。pandarallel は背後で「データを複数のプロセス（別のメモリ空間）にコピーして分配する」ため、通信オーバーヘッド（コピーの手間）がかかる。「単純な足し算」などのすぐに終わる軽い処理を parallel_apply にかけても、計算時間よりもコピー時間の方が長くなり、かえって遅くなる。「CPU1 コアでは計算に時間がかかりすぎる重い処理」でのみ並列化を採用するという見極めが必要である。

7.4 さらに大規模なデータへの対応

pandas は強力だが、全データを一度にメモリ（RAM）に載せる必要がある設計上、マシンのメモリを超えるようなテラバイト級のデータを処理させるとクラッシュしてしまう。また、内部的なループの非効率さも残っている。近年では、言語の内部実装に Rust を用いるなどで劇的な高速化とメモリ効率の改善を図った Polars 等のライブラリの利用も急速に広がっている。もし数千万行に及ぶ処理で pandas の限界を感じた場合は、Polars への移行を検討するのも有効なアプローチである。

第 8 章 Matplotlib による可視化

Matplotlib は、Python で図を描くための標準的なライブラリである。本章では、まずデータを手軽に可視化するための基本的なコードの書き方からはじめ、データの性質に応じた図の選び方（実践的な作図）を学ぶ。そして後半では、より複雑な図を制御するために必須となる「オブジェクト指向 API」と構成要素の考え方や、スタイルの管理について解説する。

図はレポートの飾りではない。ヒストグラムや散布図は、計算結果の妥当性を自分で確かめるための道具でもある。数値だけを並べるより、まず図にした方が異常に気付きやすいことは多い。

例えば、平均値や標準偏差が妥当に見えても、分布の裾が長い、二峰性がある、外れ値が混ざっている、といった特徴は図にしないと分かりにくい。可視化は最後の仕上げではなく、途中の確認作業としても重要である。

8.1 基本的な作図と可視化

8.1.1 最小限のヒストグラム

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 values = np.random.normal(loc=0.0, scale=1.2, size=2500)
5
6 plt.hist(values, bins=40)
7 plt.xlabel("value")
8 plt.ylabel("count")
9 plt.savefig("histogram.pdf")
```

解析では PDF 出力が便利である。拡大しても崩れにくく、LaTeX レポートへそのまま入れやすいからである。図 8.1 は、この最小例をそのまま実行したときの典型的な出力である。

`plt.show` による対話的な確認は便利だが、再現可能な解析では `savefig` でファイルへ残す方が重要になる。図を保存するたびに同じファイル名へ上書きするのか、条件ごとに別名で保存するのかも、早い段階で決めておくと混乱しにくい。

8.1.2 軸ラベル、凡例、目盛り

図として最低限必要なのは、軸ラベル、必要なら単位、系列名、そして比較に必要な目盛りである。ここを省くと、図が描けても読めない。

```
1 import numpy as np
```

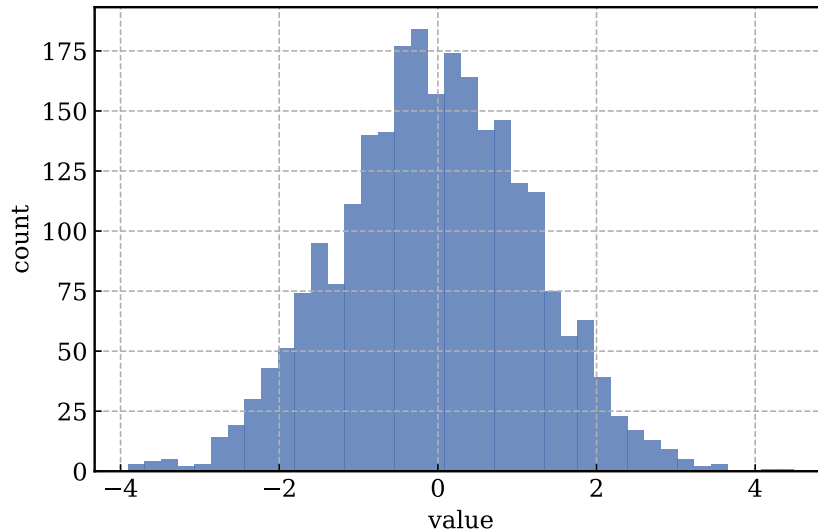


図 8.1 最小限のヒストグラム出力例。最初は、入力配列 `values` が何本の棒へ数え上げられているかだけを追えばよい。

```

2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0.0, 10.0, 60)
5 y = np.exp(-x / 4.0) * (1.0 + 0.08 * np.sin(2.5 * x))
6
7 fig, ax = plt.subplots(figsize=(6, 4))
8 ax.plot(x, y, marker="o", label="sample_A")
9 ax.set_xlabel(r"$t$, [\mathrm{s}]$")
10 ax.set_ylabel("signal")
11 ax.set_title("Labeled_line_example")
12 ax.set_xticks([0, 2, 4, 6, 8, 10])
13 ax.legend(loc="upper_right")
14 fig.savefig("labeled_plot_example.pdf")

```

図 8.2 のように、ラベルと凡例と目盛りがそろっただけで、何を見せたい図なのかが一気に分かりやすくなる。最初は配色に凝るより、まずラベルと凡例を外さない方が重要である。

8.1.3 plot 関数へ渡すデータ

Matplotlib の多くの作図関数は、NumPy 配列や Python リストのような「配列に変換できるもの」を受け取る。加えて、辞書や DataFrame のように列名で参照できるオブジェクトなら、`data=` を使って列名指定で描くこともできる。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 times = [0, 1, 2, 3]
5 values = [1.0, 2.0, 1.5, 3.0]
6
7 fig, ax = plt.subplots(figsize=(6, 4))

```

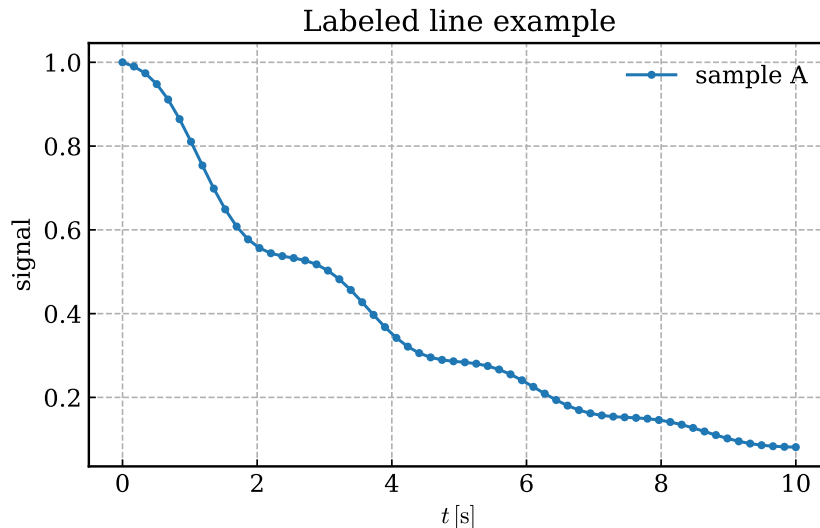



図 8.2 ラベル、凡例、目盛りを付けた最小限の線図。横軸の単位と系列名があるだけで読みやすさが大きく変わる。

```

8 x = np.asarray(times)
9 y = np.asarray(values)
10 ax.plot(x, y)
11
12 table = {"time": x, "value": y}
13 ax.plot("time", "value", data=table)

```

`np.asarray` で NumPy 配列へそろえておくと、型のぶれを減らせる。`data=` は短いコードを書くのに便利だが、列名の打ち間違いに気付きにくいこともあるので、最初は `x` と `y` を明示して渡す方が安全である。

8.2 実践的な作図（データに応じた図の選択）

データ解析において、2つの変数の相関関係を見ることは非常に多い。しかし、これを「とりあえず」描画関数にかけるだけでは、重要な特徴を見落とす。データ量に応じた適切な図の選択プロセスを順番に見ていく。

8.2.1 相関の可視化：論理的改善ステップ

8.2.1.1 Step 1: 直感的な散布図 (Scatter) の限界

2変数の相関を見るのに最も基本となるのが散布図 (scatter) である。しかし、データ点が数十万規模になると、これはすぐに「役に立たない図」と化す。

```

1 # s=10 で大量の点（約60万点）をそのまま描画すると...
2 fig, ax = plt.subplots(figsize=(6, 5))
3 ax.scatter(x, y, s=10, c='black')
4 fig.savefig("scatter_bad.png", dpi=150)

```

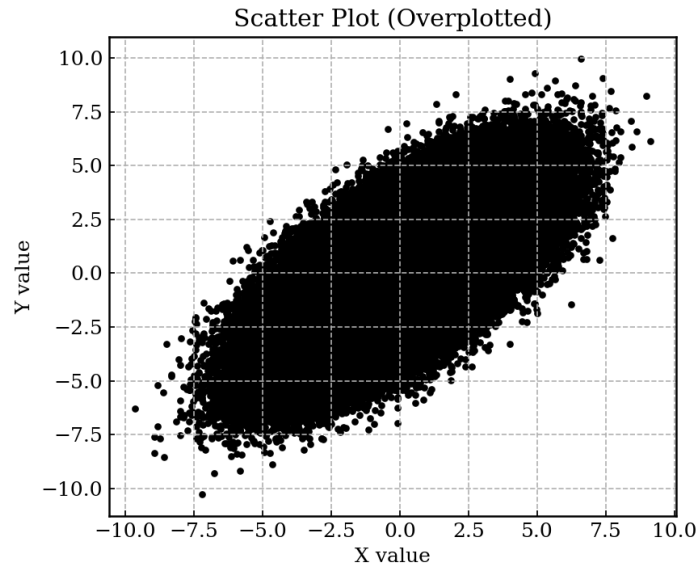


図 8.3 大量データに対する散布図。全てが黒く沈んでおり内部構造が一切見えない。

図 8.3 のような図を作ってしまうてはならない。データがどこにどれだけ集中しているのか（密度）が全く分からないからである。

8.2.1.2 Step 2: 2次元ヒストグラムの導入と、線形スケールの限界

点が潰れるなら、「その領域にいくつ点があるか」を色で表現する 2 次元ヒストグラム (hist2d) が有効である。

```
1 fig, ax = plt.subplots(figsize=(6, 5))
2 h_lin = ax.hist2d(x, y, bins=50, cmap='viridis')
3 cbar = fig.colorbar(h_lin[3], ax=ax, label="Counts_␣(Linear)")
4 fig.savefig("hist2d_linear_bad.pdf")
```

図 8.4 は中心部が明るくなっているため、散布図よりはマシに見える。しかし、極端に集中している一部（例: 2 万カウント）の色に引っ張られ、10 カウント、100 カウントといった周囲のノイズや裾野の広がりやすべて濃紺に塗りつぶされてしまっている。

8.2.1.3 Step 3: 大規模データに必須となる LogNorm (対数カラスケール)

現実の物理イベントは、頻度が「1, 10, 100, 1000」のように桁単位で変化することが多い。この何桁にもまたがる広がりをもつデータを一つの図に収めるための判断基準が「カウントのカラーバーを対数 (log) にするべきか否か」である。

```
1 import matplotlib.colors as mcolors
2
3 fig, ax = plt.subplots(figsize=(6, 5))
4 # norm=mcolors.LogNorm() により色の割当を対数スケールにする
5 h_log = ax.hist2d(x, y, bins=50, norm=mcolors.LogNorm(), cmap='viridis')
6 cbar = fig.colorbar(h_log[3], ax=ax, label="Counts_␣(Log_Scale)")
7 fig.savefig("hist2d_lognorm_good.pdf")
```

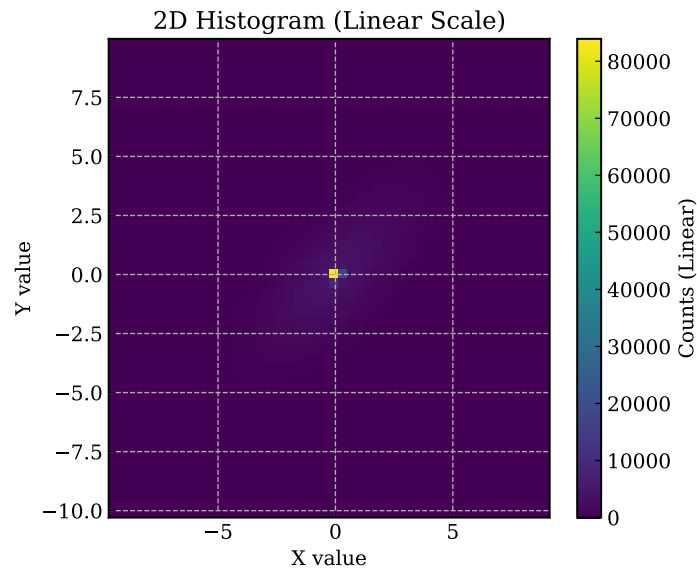


図 8.4 線形スケールを用いた 2D ヒストグラム。最も集中した中心部だけが目立ち、周辺が潰れる。

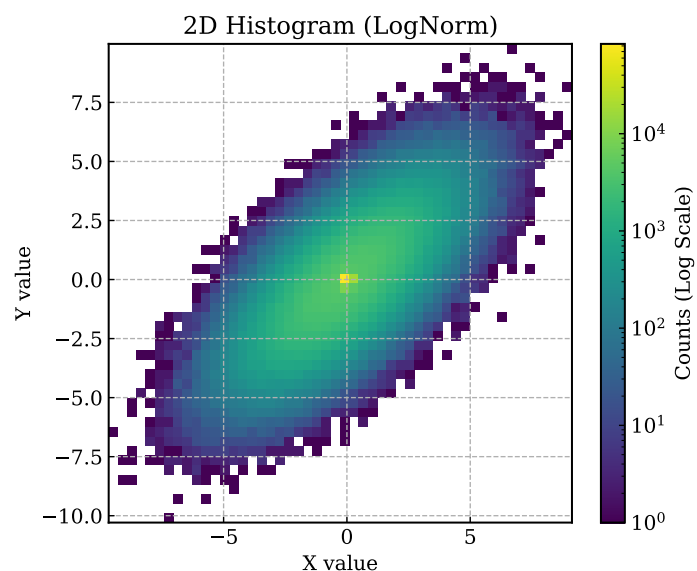


図 8.5 LogNorm を用いた 2D ヒストグラム。中心の密集部分と、周囲の広がりが同時に視認できる。

図 8.5 を見れば、単一のピークの周りに広く緩やかなイベントの相関層が広がっていることが一目で分かる。これが「なぜ LogNorm を知っていなければならないか」の強力な理由である。

8.2.2 複数の図を並べる (subplots)

レポートや論文では、関連する分布を左右や上下に並べて比較することが多い。この場合、1 枚のキャンバス (Figure) の中に複数のパネル (Axes) を配置する機能を利用する。

```
1 # 1行2列（左右に2つ）のパネルを作成する。
2 # fig はキャンバス全体、axes は2つのパネル（配列）を表す。
3 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

```

4
5 # 左のパネルには axes[0] を使って描画する
6 axes[0].hist(data_a, bins=30)
7 axes[0].set_title("Data_A")
8
9 # 右のパネルには axes[1] を使って描画する
10 axes[1].hist(data_b, bins=30)
11 axes[1].set_title("Data_B")
12
13 # 複数のパネルを描くと軸の数字やラベルが隣の図と重なりやすいため、
14 # 最後に必ず tight_layout() を呼んで余白を自動調整する。
15 fig.tight_layout()
16 fig.savefig("subplots_example.pdf")

```

単純な `plt.plot()` ばかり使っていると、この「特定のパネル (`axes[0]`) に対して描画を命令する」という書き方に慣れるのに少し時間がかかるが、複雑な比較図を作る上では避けて通れない技法である。Figure や Axes についての詳細は、8.3 節以降で述べる。

8.2.3 凡例とレイアウト

複数の系列やフィット曲線を同じ図へ重ねるなら、凡例は不可欠である。凡例をどこへ置か、枠を付けるか、何をどの順に書くかで、読みやすさは大きく変わる。

ここでいう `values1` と `values2` は、同じ物理量を別条件で測った 1 次元配列だと考えればよい。例えば、温度条件 A と B で得た信号振幅を左右のパネルへ並べ、同じビン数でヒストグラムを描いて分布形状を比較する場面である。単にコードの書式を見るのではなく、同じ量を別条件で見比べるための雛形だと捉えると、このリストの意図が分かりやすい。

```

1 fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
2 axes[0].hist(values1, bins=40, alpha=0.7, label="sample_A")
3 axes[0].set_title("sample_A")
4 axes[0].set_xlabel("value")
5 axes[0].set_ylabel("count")
6
7 axes[1].hist(values2, bins=40, alpha=0.7, label="sample_B")
8 axes[1].set_title("sample_B")
9 axes[1].set_xlabel("value")
10
11 for ax in axes:
12     ax.legend(loc="upper_right")
13     ax.set_axisbelow(True)
14
15 fig.tight_layout()
16 fig.savefig("subplots_example.pdf")

```

この例では、`axes` は二つの Axes を並べた配列であり、`for ax in axes:` の部分で「凡例位置をそろえる」という共通処理を両パネルへまとめて適用している。`sharey=True` を付けると図 8.6 のように縦軸の尺度がそろうため、左右の高さをそのまま比較しやすい。ここで

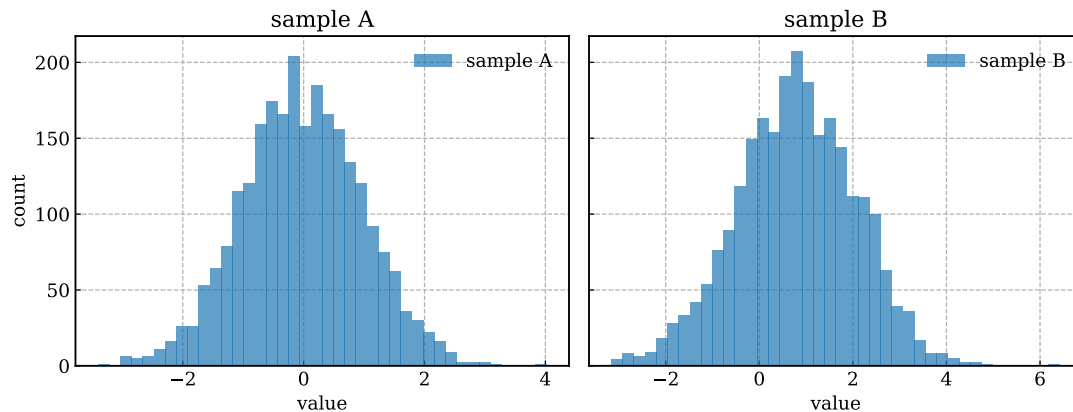


図 8.6 左右に並べたヒストグラムの出力例。縦軸を共有すると、高さの差をそのまま比較しやすい。は各パネルに系列が一つしかないため、凡例はやや大げさに見えるかもしれないが、後からフィット曲線や参照線を重ねるとそのまま拡張できる書き方である。

`tight_layout` は余白調整の基本であるが、すべてを自動で完璧に直してくれるわけではない。タイトルや長いラベルが多い場合は、図のサイズや余白を自分で調整した方がよい。複数パネルの図では、どのパネルを比較させたいのかを意識して配置することが大切である。

凡例がデータを隠すなら、`loc="best"` で自動配置に任せるか、`bbox_to_anchor` を使って枠の外へ逃がす。レイアウト調整は「自動で整うかどうか」ではなく、「比較したい情報が隠れていないかどうか」で判断する方が実践的である。

8.2.4 エラーバーと対数軸 (log-log プロット)

実際の物理データの多くは測定誤差を伴うため、単なる散布図ではなくエラーバー付きのプロットが必要になる。また、横軸や縦軸のスケールが指数関数的に落ち幅を変える場合、対数 (log) のグラフでプロットしなければ分布の様子が潰れてしまうことが多い。

図 8.7 にエラーバーと対数プロットの例を示す。

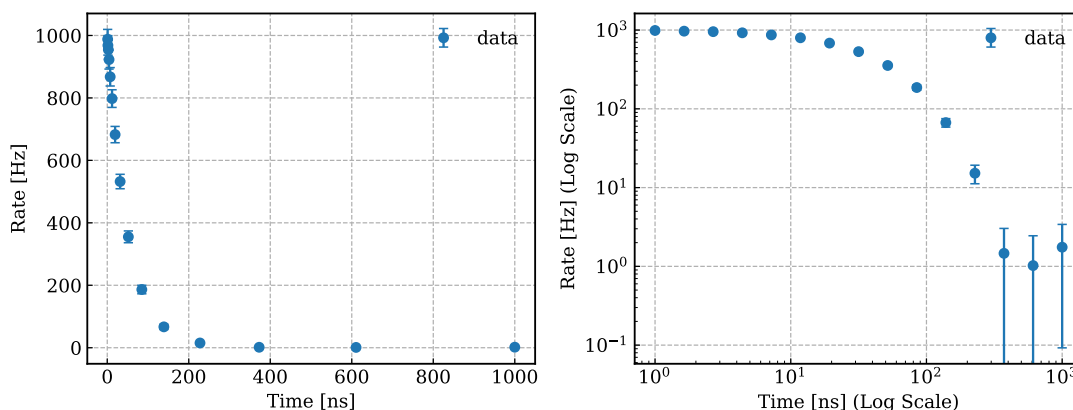


図 8.7 エラーバーと対数軸の設定例。左は通常の線形軸、右は X, Y 両方を対数にした図。

```
1 fig, axes = plt.subplots(1, 2, figsize=(10, 4)) # 横10インチ、縦4インチで 1行2列の
   panelsを作成
```

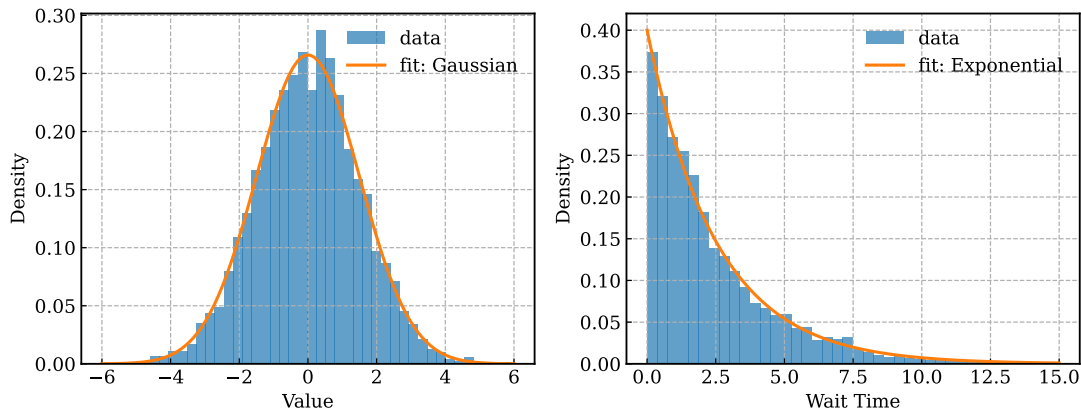


図 8.8 Matplotlib による分布図とフィットの例。左はガウシアン型、右は指数分布であり、どちらもヒストグラムの上にフィット曲線を重ね、凡例に関数形とパラメータを書いている。

```

2
3 # 左のパネル (axes[0]) へのプロット
4 # x_val, y_val を点ごとにエラー y_err でプロット。
5 # fmt='o' は丸い点を意味し、capsize=3 はエラーバーの終端の横棒の長さ。
6 axes[0].errorbar(x_val, y_val, yerr=y_err, fmt='o', capsize=3, label="data")
7 axes[0].set_xlabel("Time_μs")
8
9 # 右のパネル (axes[1]) へのプロット
10 axes[1].errorbar(x_val, y_val, yerr=y_err, fmt='o', capsize=3, label="data")
11 # 対数スケール化
12 axes[1].set_yscale('log')
13 axes[1].set_xscale('log')
14
15 fig.tight_layout() # ラベルや目盛りが重ならないように余白を自動調整
16 fig.savefig("errorbar_example.pdf")

```

8.2.5 結果例の読み方

図を読めるようになるには、単に描けるだけでは足りない。まず軸と単位を確認し、次に分布の中心と幅、最後に裾や外れ値を見る、という順で眺めると整理しやすい。

例えば、図 8.8 の左パネルでは、ヒストグラムの山がフィット曲線とよく重なっているか、左右対称性がどの程度あるかを見る。右パネルでは、短時間側の立ち上がりや長時間側の減衰が同じ時定数で説明できるかを見る。こうした読み方を本文中で言葉にしておくと、図は単なる飾りではなく考察の根拠になる。

8.2.6 画像フォーマットの選択：ラスタとベクタ

解析図を保存する際、適切な画像フォーマットを選ぶことは極めて重要である。

- **ベクタ形式 (.pdf, .svg, .eps)**：どれだけ拡大しても線や文字がぼやけない。論文やレポートで図を用いる場合、文字やプロット点を高品質なまま印刷できるためこれが基本

となる。関数 `savefig("plot.pdf")` で保存できる。

- **ラスタ形式 (.png, .jpg)**：ピクセル（点）の集まりで画像を描画する。拡大するとジャギ（ピクセルのギザギザ）が見える。ただし、例えば数十万点に及ぶ密集した散布図や、ヒートマップのような巨大な 2 次元データをベクタ形式で保存すると、点の一つ一つがファイルに記録されるためファイルサイズがギガバイトを超え、PDF ビューアがフリーズする原因になる。そのような複雑な図の場合は、`savefig("plot.png", dpi=150)` のように解像度（dpi）を指定して PNG で出力する方が適切である。

```
1 fig, ax = plt.subplots(figsize=(6, 4))
2 ax.plot(x, y)
3 ax.set_xlabel("time_μ[ns]")
4 ax.set_ylabel("count")
5
6 fig.savefig("spectrum.pdf")
7 fig.savefig("spectrum.png", dpi=150)
```

同じ図でも、`spectrum.pdf` は拡大しても線と文字が崩れにくく、`spectrum.png` はスライド貼り付けや Web 掲載で扱いやすい。どちらが良いかは「図の複雑さ」と「最終的な使い道」で決まる。線図や通常のヒストグラムなら PDF を第一候補にし、巨大散布図やヒートマップでは PNG を検討する方が実務的である。

8.3 オブジェクト指向 API による複雑な図の管理

8.3.1 Figure, Axes, Axis, Artist の役割

Matplotlib の初学者が最も混乱しやすいのは、Figure と Axes と Axis の区別である。特にややこしいのは、Axes という名前が複数形に見えるにもかかわらず、Matplotlib では「1 枚の作図パネル」を表すクラス名として使われている点である。まずは次の対応で覚えるとよい。

Figure 図全体である。1 つの PDF や PNG、1 つの表示ウィンドウに相当する。`savefig`、`suptitle`、図全体の凡例や `colorbar` などは主に Figure 側の仕事である。

Axes データを実際に描く 1 つのパネルである。`plot`、`hist`、`scatter`、`set_xlabel`、`legend` など、日常的な作図命令の多くは Axes のメソッドとして呼ぶ。

Axis (x Axis, y Axis) 目盛り、目盛りラベル、スケールを管理する部品である。通常は `ax.set_xscale("log")` や `ax.set_xlim(...)` のように Axes 経由で触るが、内部には `ax.xaxis` と `ax.yaxis` がある。

Artist 図の中に描かれるすべての要素の総称である。線は `Line2D`、文字は `Text`、凡例は `Legend`、ヒストグラムの棒は `Rectangle` といった Artist として管理される。以下なども Artist に含まれる。

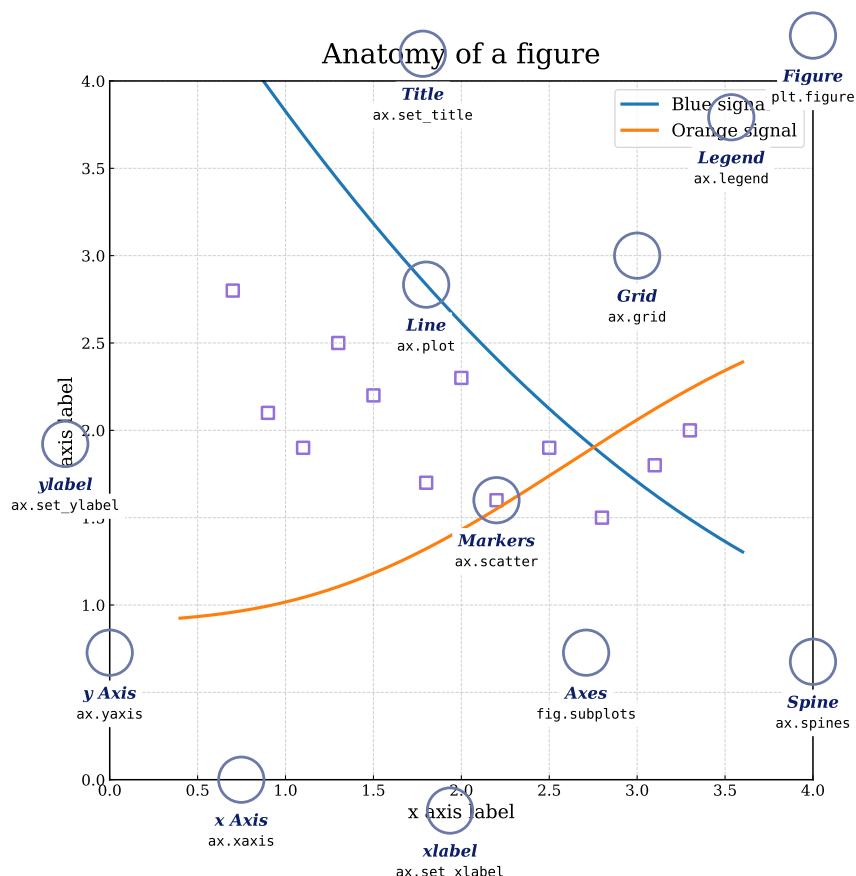


図 8.9 Figure、Axes、Axis、Artist の関係を示した例。1 つの Figure の中に Axes があり、その Axes の中で線、文字、凡例などの Artist が描かれている。

- Title, Legend: 図のタイトル、および各系列を示す凡例。
- Grid, Spine: グリッド線と、座標系を囲む枠線。
- Line, Markers: 折れ線や散布図の点。
- xlabel, ylabel: x 軸・y 軸のラベル。

Figure、Axes、Axis、Artist の階層関係だけを先に図解の構図に合わせて木構造で書くと、次のようになる。

```
Figure
|-- Axes
| |-- Title
| |-- Spine, Grid
| |-- xaxis, yaxis (x Axis, y Axis / xlabel, ylabel)
| |-- Line2D, Markers
| `-- Legend
`-- Figure-level Text / Colorbar / ...
```

図 8.9 に、Figure、Axes、Axis、Artist の位置関係を示す。まずは「Figure の中に Axes があり、その中に Axis と各種 Artist がある」という階層だけ押さえれば十分である。この対応を、実際の Python オブジェクトの型で確かめる短いコードを次に示す。


```

1 import matplotlib.pyplot as plt
2
3 x = [0, 1, 2, 3]
4 y = [1.0, 2.0, 1.5, 3.0]
5
6 fig, ax = plt.subplots(figsize=(6, 4))
7 line, = ax.plot(x, y, label="signal")
8 ax.set_title("Axes_title")
9 ax.set_xlabel("time")
10 ax.set_ylabel("signal")
11 legend = ax.legend()
12
13 print(type(fig).__name__)
14 print(type(ax).__name__)
15 print(type(ax.xaxis).__name__)
16 print(type(line).__name__)
17 print(type(legend).__name__)
18 print(len(fig.axes))
19 fig.savefig("line.pdf")

```

このコードを実行すると、次の出力が得られる。

```

Figure
Axes
XAxis
Line2D
Legend
1

```

最後の 1 は、この Figure の中に Axes が 1 枚だけ入っていることを表す。もし `plt.subplots(2, 2)` と書けば、Figure は 1 つのままで、Axes が 4 枚入る。

Artist という言葉も、線だけを指すわけではない。タイトルや軸ラベルは Text、ヒストグラムの各棒は Rectangle として管理される。次のコードで、どの型が返るかを確かめる。

```

1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(6, 4))
4 title = ax.set_title("Histogram")
5 xlabel = ax.set_xlabel("value")
6 bars = ax.hist([1, 1, 2, 3], bins=[0.5, 1.5, 2.5, 3.5])[2]
7
8 print(type(title).__name__)
9 print(type(xlabel).__name__)
10 print(type(bars).__name__)
11 print(type(bars[0]).__name__)

```

このコードの出力は次のとおりである。

```

Text
Text

```

BarContainer
Rectangle

ここで bars 全体は BarContainer であり、その中の 1 本 1 本は Rectangle である。したがって、「ヒストグラムの棒だけ色を変える」といった細かい調整は、最終的には Artist を触る操作だと理解できる。

Figure 側の要素も、コードで見ると役割が分かりやすい。図全体タイトル `suptitle` や `colorbar` は、特定の Axes だけでなく Figure 全体の見た目に関わることが多い。次のコードでは、その型を確認する。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 fig, ax = plt.subplots(figsize=(5, 4))
5 img = ax.imshow(np.arange(4).reshape(2, 2))
6 suptitle = fig.suptitle("Figure_title")
7 cbar = fig.colorbar(img, ax=ax)
8
9 print(type(suptitle).__name__)
10 print(type(cbar).__name__)
```

このコードでは、次の出力が得られる。

Text
Colorbar

`suptitle` は Figure 全体に付く Text であり、`colorbar` は Figure に追加される補助要素である。複数パネルの図で共通タイトルや共通 `colorbar` を付けたいときは、Axes よりも Figure 側の仕事だと考えると整理しやすい。

Axis は少し見えにくい部品だが、実際には目盛りと表示範囲を管理している。たとえば次のような操作は Axis に関わる設定である。

```
1 fig, ax = plt.subplots(figsize=(6, 4))
2 ax.plot([1, 10, 100], [2, 3, 5], marker="o")
3 ax.set_xlim(1, 100)
4 ax.set_xscale("log")
```

`set_xlim` は表示範囲を、`set_xscale("log")` は目盛りの刻み方を変える。ふだんは `ax.xaxis` を直接操作しなくても、Axes メソッド経由で Axis の設定を変えていると理解すれば十分である。

どのオブジェクトを操作すべきかは、何を変えたいかで決まる。次の対応で考えると迷いにくい。

- 図全体を保存する、図全体タイトルを付ける: Figure
- 線やヒストグラムを描く、軸ラベルや凡例を付ける: Axes
- 対数軸、目盛り位置、目盛りラベルを変える: Axis 関連。ただし実際の操作は Axes メソッド経由が多い

- 特定の線だけ色や太さを変える、凡例や文字を細かく調整する: Artist

この区別が付くと、「保存はなぜ `fig.savefig(...)` なのか」「凡例はなぜ `ax.legend()` なのか」「対数軸はなぜ `ax.set_xscale("log")` で変えるのか」が自然につながる。Matplotlib を使い始めたら、まず `fig` と `ax` のどちらへ命令しているのかを追うだけでも、コードの見通しはかなり良くなる。

8.3.2 オブジェクト指向 API で描く

Matplotlib には `plt.hist(...)` のような簡単な書き方と、Figure と Axes を明示的に扱う書き方がある。図が複雑になるほど、後者の方が整理しやすい。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 values = np.random.normal(loc=0.0, scale=1.2, size=2500)
5
6 fig, ax = plt.subplots(figsize=(6, 4))
7 ax.hist(values, bins=40)
8 ax.set_xlabel("value")
9 ax.set_ylabel("count")
10 fig.savefig("histogram.pdf")
```

`ax` に対して操作を書く形にしておくと、複数図や複数パネルへ発展しやすい。解析スクリプトを後で拡張したり、提出用の図を安定して作り直したりする場面では、この書き方に慣れておく価値が高い。

また、Figure と Axes を区別して考えると、図全体に効く設定と各パネルにだけ効く設定を整理しやすい。タイトルや余白は図全体、軸ラベルや凡例は各パネル、というように役割を分けて考えると見通しが良い。

8.3.3 補助関数で作図をまとめる

同じ体裁の図を何枚も作るなら、Axes を引数にする補助関数へまとめると管理しやすい。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def plot_hist(ax, values, label):
5     ax.hist(values, bins=40, alpha=0.7, label=label)
6     ax.set_xlabel("value")
7     ax.set_ylabel("count")
8     ax.legend(loc="upper_right")
9
10 rng = np.random.default_rng(0)
11 values1 = rng.normal(loc=0.0, scale=1.0, size=2500)
12 values2 = rng.normal(loc=0.8, scale=1.3, size=2500)
13
14 fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
```

```

15 plot_hist(axes[0], values1, "sample_A")
16 plot_hist(axes[1], values2, "sample_B")
17 fig.savefig("subplots_example.pdf")

```

重要なのは、`plt.hist(...)` のようなグローバルな状態へ書くのではなく、「どの Axes に描くか」を引数で受け取る点である。比較図、条件ごとの反復描画、関数の再利用がしやすくなる。

8.4 見た目とスタイルの管理

図を読みやすくするには、少なくとも次の点を意識する。

- 軸ラベルに量と単位を書く
- タイトルを簡潔に付ける
- 余白を調整する
- 色と線幅を統一する
- レポート全体で表記を揃える

色やフォント、線幅をばらばらにすると、個々の図は読めてもレポート全体として落ち着かない。逆に、軸ラベルの位置、単位の書き方、凡例のスタイルをそろえるだけで、見た目の質は大きく上がる。可視化では、派手さより統一感の方が効くことが多い。

8.4.1 rcParams で見た目をそろえる

Matplotlib では、図のフォントや線幅などの既定値の多くを `rcParams` という辞書オブジェクトで一元管理している。図を描くたびに同じ設定を繰り返すより、作図スクリプトの冒頭で一度だけこの辞書を更新するのが基本である。

```

1 import matplotlib as mpl
2
3 mpl.rcParams.update(
4     {
5         "font.family": "serif",
6         "mathtext.fontset": "cm",
7         "font.size": 12,
8         "axes.grid": True,
9         "grid.linestyle": "--",
10        "xtick.direction": "in",
11        "ytick.direction": "in",
12        "axes.linewidth": 1.2,
13        "legend.frameon": False,
14    }
15 )

```

- `font.family`: 既定フォント族を決める。本文と図の雰囲気をもそろえたいときの入口に

なる。

- `font.size`: 文字の基本サイズである。発表スライド用と論文用で最初に変えたい項目である。
- `axes.grid`, `grid.linestyle`: グリッドを既定で表示するか、その線種をどうするかを決める。
- `xtick.direction`, `ytick.direction`: 目盛り線を内向きにするか外向きにするかを定める。
- `axes.linewidth`: 枠線の太さである。細すぎると印刷時に弱く見える。
- `legend.frameon`: 凡例の枠を出すかどうかを決める。

`rcParams` で設定した値は、そのスクリプトで後から作る図へまとめて効く。したがって、「毎回同じ見た目にしたい項目」をここへ集め、「この図だけ変えたい項目」は後で `ax.set_*` などで上書きする、と分けると整理しやすい。

8.4.2 設定ファイル (.mplstyle) で見た目を共通化する

しかし、解析テーマごとに作図スクリプトが増えていくと、すべての Python ファイルの先頭に上記の長い辞書をコピーするのは冗長である。Matplotlib では、これらの設定をプレーンテキストに切り出し、まとめて読み込める。

作業ディレクトリに `my_style.mplstyle` という名前で、次のような設定ファイルを作成する。

```
font.family: serif
mathtext.fontset: cm
font.size: 12
axes.grid: True
grid.linestyle: --
xtick.direction: in
ytick.direction: in
axes.linewidth: 1.2
legend.frameon: False
```

そして、各解析スクリプトの先頭でこれを一度だけ呼び出せばよい。本テキストに含まれる図も、すべてこの設定ファイル（スタイルシート）を読み込んで生成されている。

```
1 import matplotlib.pyplot as plt
2
3 # スクリプトの冒頭で自分のスタイルシートを読み込む
4 plt.style.use("./my_style.mplstyle")
5
6 # 以降の描画はすべて統一されたスタイル (Serif フォント等) が反映される
7 fig, ax = plt.subplots(figsize=(6, 4))
```

このように設定をファイルに分離しておくことで、新しい解析スクリプトを書くたびにフォントやグリッドの有無がバラバラになる事態を防ぐことができる。また、学会発表用の

大きなフォント設定といった別のファイルを容易に切り替えることも可能になる。

8.4.3 共通スタイルの一部だけを上書きする

共通スタイルを決めておくと便利だが、発表スライド用に文字だけ大きくしたい、下書きではグリッドだけ消したい、といった局所的な変更もよく起こる。そのたびに元の `my_style.mplstyle` を書き換えると、別の図まで見た目が変わってしまう。こういう場合は、基準となるスタイルを読み込んだ後に、必要な項目だけを重ねる方が管理しやすい。

```
# presentation_style.mplstyle
font.size: 16
axes.grid: False
legend.fontsize: 11
```

```
1 import matplotlib as mpl
2 import matplotlib.pyplot as plt
3
4 plt.style.use(["./my_style.mplstyle", "./presentation_style.mplstyle"])
5
6 # スタイルファイルを増やしたくないなら、その場で個別に上書きしてもよい
7 mpl.rcParams["axes.titlesize"] = 14
8
9 fig, ax = plt.subplots(figsize=(6, 4))
10 ax.grid(False) # この Axes だけグリッドを消す
```

二つの `.mplstyle` を配列で渡すと、後ろに書いたファイルの設定が優先される。したがって、基準となる `my_style.mplstyle` を保ったまま、文字サイズやグリッドの有無だけを差し替えられる。さらに、`mpl.rcParams` を直接書き換えれば、そのスクリプト以降の図だけを変更でき、`ax.grid(False)` のような Axes メソッドを使えば、単一のパネルにだけ適用できる。設定の効く範囲が、スタイルファイル、スクリプト全体、個別の Axes の三段階に分かれると理解しておくで混乱しにくい。

8.5 比較しやすい図を作る

8.5.1 Matplotlib の数式表記を使う

Matplotlib では、軸ラベルや凡例に数式風の表記を書ける。既定では LaTeX そのものではなく、TeX に似た `MathText` という仕組みが使われる。

```
1 plt.xlabel(r"$\Delta_t$, [ $\mathrm{ns}$ ]")
2 plt.ylabel(r"$N$")
```

単位まで数式の中へ入れるなら、上の例のように `mathrm` を使って立体で書く方が自然である。レポートを LaTeX で書くなら、図中の記号も本文と整合するようにそろえると見通しがよい。

MathText は便利だが、LaTeX そのものではない。よく使う分数、上付き・下付き、ギリシャ文字、

`mathrm` などは扱える一方で、複雑なマクロや一部のパッケージ依存の記法は使えない。通常は追加の LaTeX 環境なしで動くのが利点であり、本文と完全に同じ組版結果まで求めるときだけ `text.usetex=True` のような設定を検討する。

また、図中で使う記号は本文での定義と一致させるべきである。本文で τ を時定数として定義したなら、図の凡例やキャプションでも同じ記号を使う方が親切である。解析内容と図の表現が食い違っていると、読者は不要なところで迷う。

8.6 よくある落とし穴

8.6.1 `plt` と `ax` を混ぜて迷子になる

Matplotlib では `plt.xlabel(...)` のような状態依存の書き方もできるが、複数パネルや複数 Figure を扱い始めると、「いまどの Axes が現在の対象なのか」が分かりにくくなる。複雑な図では `fig, ax = plt.subplots(...)` を起点にして、以後は `ax.set_xlabel(...)` のように書く方が安全である。

8.6.2 文字列がカテゴリ軸になる

数字に見える列でも、実際には文字列として読まれていると、Matplotlib は数値軸ではなくカテゴリ軸として解釈する。すると、1, 2, 10, 20 が数値間隔ではなく、単なる 4 つのラベルとして等間隔に並ぶ。

```
1 import matplotlib.pyplot as plt
2
3 x_str = ["1", "2", "10", "20"]
4 y = [1.0, 4.0, 2.0, 5.0]
5 fig, ax = plt.subplots(figsize=(6, 4))
6 ax.plot(x_str, y, marker="o")
```

図 8.10 の左では文字列がカテゴリ軸として扱われ、右では数値へ変換したので数値軸になっている。CSV を読んだ直後に軸の間隔が変だと感じたら、まず `dtype` を疑う方がよい。

8.6.3 `savefig` と余白調整の順序

`savefig` の直前に `tight_layout()` や `constrained_layout` を使わないと、長いラベルや凡例が切れることがある。また、対話環境では `show()` の後に状態が変わることもあるため、再現可能なスクリプトでは「描く、余白を整える、保存する」の順を守る方がよい。対数軸を使う場合は、0 以下の値をそのまま載せられない点にも注意が必要である。

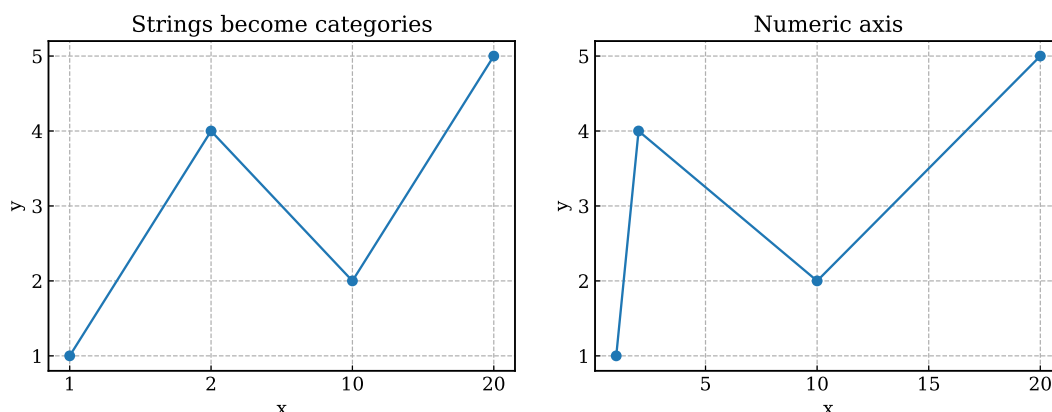


図 8.10 文字列軸と数値軸の違い。左は "1", "2", "10", "20" をカテゴリとして並べており、右は数値として扱っている。

8.6.4 公式資料から似た図を探す

Matplotlib は機能が広いため、作りたい図の名前が分からなくても、公式資料から似たものを探す方が早いことが多い。特に次の 4 つは、章を読み終えた後も繰り返し見る価値が高い。

- Quick start: https://matplotlib.org/stable/users/explain/quick_start.html
- Plot types: https://matplotlib.org/stable/plot_types/index.html
- Gallery: <https://matplotlib.org/stable/gallery/index.html>
- Cheatsheets: <https://matplotlib.org/cheatsheets/>

作りたいグラフのイメージが先にあるなら、まず Plot types で種類を当て、次に Gallery で近い完成例を探し、最後に Cheatsheets で引数名や書式を確認する、という順にたどると迷いにくい。初学者が最初から完全なコードを自力で組み立てるより、近い例を見つけて必要な部分だけ差し替える方が現実的である。

第 9 章 SciPy とフィット

9.1 フィットの考え方

ヒストグラムを描いただけでは、分布の特徴を定量化できないことがある。そこで、ある関数形を仮定してパラメータを求める。これがフィットである。

フィットは万能ではない。まず、なぜその関数形を選ぶのかという物理的、統計的な理由が必要である。さらに、ヒストグラムの範囲、ビン幅、外れ値処理によって見え方が変わるため、「フィット結果が出た」ことと「解釈してよい」ことは別である。

フィットの目的は、曲線をきれいに重ねることではない。分布の中心、幅、時定数などを、比較可能な量として取り出すことが目的である。したがって、フィット前に図を見ておくこと、フィット後に残差やずれを確認することが重要になる。

9.1.1 この本でよく使う分布

データ解析では、ガウシアンと指数分布がよく現れる。

- ガウシアン: 時刻差のばらつき
- 指数分布: 待ち時間や発生間隔

もちろん、実際のデータがいつもきれいな理想分布になるとは限らない。重要なのは「なぜその関数を当てようとしたのか」を説明することである。

分布の名前を覚えるだけでなく、どの量に対して自然に現れるかを知ることが重要である。ガウシアンは独立なばらつきの和として、指数分布はランダムな待ち時間として現れやすい。こうした背景を知っていると、単なる形合わせで終わらない。

9.1.2 SciPy の使い方

SciPy には、分布やフィットのための道具が複数ある。代表的なのは `scipy.stats` と `scipy.optimize.curve_fit` である。図 8.8 のような曲線を重ねると、単に形を見るだけでなく、平均や幅、時定数を数値として比較しやすくなる。

単純な場合は、自分で平均や分散を計算するだけでも十分なことがある。一方で、より一般的なモデルや誤差評価をしたいなら SciPy が役立つ。

`scipy.optimize.curve_fit` を使うと、モデル関数を自分で定義してパラメータを推定できる。最初は少し抽象的に見えるが、関数形を明示してデータへ当てるという流れを一度理解すると、応用範囲は広い。

```
1 import numpy as np
```

```

2
3 values = np.array([1.2, 0.9, 1.5, 1.1, 0.8])
4 mean = values.mean()
5 sigma = values.std(ddof=0)
6 print(mean, sigma)

1 from scipy.optimize import curve_fit
2
3 def linear_model(x, a, b):
4     return a * x + b
5
6 params, cov = curve_fit(linear_model, xdata, ydata)

```

params には推定パラメータが入り、順番はモデル関数の引数 (a, b, ...) に対応する。上の例なら params[0] が傾き a、params[1] が切片 b である。cov には共分散行列が入り、対角成分の平方根 `np.sqrt(np.diag(cov))` を取ると各パラメータの標準誤差の目安になる。一方、非対角成分はパラメータ間の相関を表し、値が大きいと「片方を少し動かすと、もう片方で補えてしまう」ような不安定さがあることを示す。

```

mean = 3.02
sigma = 0.79
tau = 2.47

```

こうした数値が得られたら、次に考えるべきは「図の見た目と整合するか」「別条件のデータと比較して意味があるか」「単位は妥当か」である。数値が出たこと自体より、どう読むかの方が重要である。

SciPy を使うときも、ライブラリ名だけを覚えるのではなく、「どの関数が何を入力として何を返すか」を確認する読み方が重要である。これは Python 公式チュートリアル of 標準ライブラリ章を読むときと同じ姿勢である。

9.2 Astropy で補う時刻と単位

宇宙線や天体観測の解析では、単なる文字列や `datetime` だけでは扱いにくい時刻表現がよく現れる。例えば UTC を前提にした観測時刻、MJD (Modified Julian Date)、Unix time の相互変換である。JD (Julian Date) は非常に古い基準日からの通算日数で、MJD はそれを扱いやすいように `JD - 2400000.5` とした表現である。こうした場面では `astropy.time.Time` が便利である。

```

1 from astropy.time import Time
2
3 t_event = Time("2026-04-04T12:34:56.123", format="iso", scale="utc")
4 print(t_event.mjd)
5 print(t_event.unix)

```

`scale="utc"` は、「この文字列は UTC で書かれている」と Astropy に教える指定である。時刻には UTC, TAI, TT のような複数の time scale があり、どれを前提にしているかが変わると差分計算や他系への変換結果も変わる。`datetime` が不足しているわけではないが、観測データの時刻系をまたいで扱うなら Astropy の方が自然なことが多い。特に、MJD や JD を使うデータを手で換算すると誤差や解釈のずれが入りやすい。時刻表現の変換は、自前で式を書かずに専用ライブラリへ任せた方が安全である。

さらに、`astropy.coordinates` を使うと、赤道座標、銀河座標、地平座標などの変換も行える。たとえば次のように書くと、赤道座標を銀河座標へ変換できる。

```
1 from astropy.coordinates import SkyCoord
2 import astropy.units as u
3
4 coord = SkyCoord(ra=150.0 * u.deg, dec=20.0 * u.deg, frame="icrs")
5 print(coord.galactic.l.deg, coord.galactic.b.deg)
```

座標変換そのものを本書で詳しく扱うわけではないが、Astropy が単なる時刻ライブラリではなく、「天文・宇宙物理でよく出る量の表現を安全に扱う工具箱」であることは知っておく価値がある。

9.2.1 Astropy で単位を扱う

物理量を扱うときは、数値だけでなく単位も一緒に管理した方が安全である。エネルギー、時間、距離を別々の単位系で持っている、と換算漏れがそのままバグになる。Astropy の `units` を使うと、値に単位を持たせたまま計算できる。

```
1 import astropy.units as u
2
3 speed = 100 * u.km / u.s
4 time = 5 * u.min
5 distance = speed * time
6 print(distance.to(u.km))
```

単位付き計算は一見まわりくどく見えるが、研究ノートの段階ではむしろ安心である。最終的に高速化や大量処理のために単位を外すことはあっても、まず式の意味を確認する段階では、単位を持たせておく方が解釈ミスを減らせる。

9.3 フィットを進める手順

実際の解析では、次の順に進めると混乱しにくい。

1. まず図を描き、どの関数形が候補かを考える
2. フィット範囲を決める
3. 初期値が必要ななら妥当な値を与える
4. フィット曲線を図へ重ねる

5. 推定値と残差を確認する

この順序を踏まずにいきなり `curve_fit` を呼ぶと、得られた数値が妥当なのか判断しにくい。フィットは関数呼び出しではなく、仮説と検証の手続きとして理解した方がよい。

9.3.1 初期値と制約

モデルによっては、初期値の与え方で収束先が変わることがある。また、明らかに負になってほしくないパラメータなどには制約を入れた方が自然なこともある。

```
1 params, cov = curve_fit(
2     model,
3     xdata,
4     ydata,
5     p0=[1.0, 0.5],
6     bounds=([0.0, 0.0], [np.inf, np.inf]),
7 )
```

`p0` は初期値、`bounds` はパラメータ範囲である。どちらも乱用すべきではないが、物理的にあり得る範囲が分かっているなら、フィットの安定化に役立つ。

フィット結果を図へ重ねるときは、関数形と推定パラメータを凡例に書く方が見やすいことが多い。図の上に直接テキストを置くと、データや曲線と重なって読みにくくなりやすい。特に、比較する図が複数ある場合は、凡例の書式をそろえると見通しが良くなる。

```
1 fig, ax = plt.subplots()
2 ax.hist(values, bins=40, density=True, alpha=0.7, label="sample")
3 ax.plot(
4     x,
5     gaussian_pdf(x, mean, sigma),
6     linewidth=2.0,
7     label=(
8         "fit:  $\frac{1}{\sqrt{2\pi}\sigma}$ "
9         " $\exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$ "
10        "\n"
11        + fr"$\mu={mean:.2f}, \sigma={sigma:.2f}$"
12    ),
13 )
14 )
15 ax.legend(loc="upper right")
```

図 8.8 でも、フィット曲線の式と推定パラメータは凡例へ入れている。本文、キャプション、凡例の役割を分けると、図全体が読みやすくなる。

9.3.2 残差とフィット範囲

フィットが見た目に良さそうでも、特定の範囲だけ大きくずれていることがある。そのため、可能なら残差も確認したい。

```
1 model_y = gaussian_pdf(x, mean, sigma)
```

```
2 residual = y - model_y
```

また、外れ値や非物理的な領域を含めると、パラメータが不安定になることがある。どの範囲でフィットしたかを書き残すことは、再現性のためにも解釈のためにも重要である。

```
1 mask = (xdata >= 0.0) & (xdata <= 10.0)
2 params, cov = curve_fit(model, xdata[mask], ydata[mask])
```

フィット範囲を切るという行為は、都合の悪い部分を隠すことではなく、どの領域でそのモデルが有効だと思えるかを明示することである。したがって、結果を報告するときは範囲も本文に書くべきである。

9.4 重み付きフィット

物理の実データでは、各データポイントが常に同じ信頼度を持つとは限らない。カウント数が少ない点は誤差（ポワソン誤差）が大きく、カウント数が多い点は誤差が小さい。この情報を無視してフィットを行うと、信頼性の全くない外れ値に引っ張られて誤った結果を生む。

9.4.1 誤差をどう重みに反映するか

9.4.1.1 Step 1: 誤差を無視した素朴なフィットの限界

まずは、各データ点を持つエラーバー（誤差）を全て同じとみなしてフィットしてみよう。

```
1 from scipy.optimize import curve_fit
2
3 def linear_model(x, a, b):
4     return a * x + b
5
6 # sigma を指定せず、単に点と点の距離だけで合わせようとする
7 popt_bad, cov = curve_fit(linear_model, xdata, ydata)
```

9.4.1.2 Step 2: sigma による適切な重み付け

正しいアプローチは、「誤差が大きい（不確かな）点には合わせに行かず、誤差が小さい（確かな）点に重みをおいて」フィットすることである。これを `curve_fit` における `sigma` 引数で指定する。

```
1 # sigma=y_err で各点の重みを伝える。
2 # absolute_sigma=True にすることで、誤差が物理的な絶対値であることを教える（共分散の計算に影響）
3 popt_good, cov = curve_fit(
4     linear_model,
5     xdata,
6     ydata,
```

```

7     sigma=y_err,
8     absolute_sigma=True
9 )
10
11 # パラメータの誤差は共分散行列の対角成分の平方根から算出される
12 errors = np.sqrt(np.diag(cov))

```

sigma には、通常 ydata の各点に対応する 1 次元配列を渡す。最小化される量は概ね $(y - f(x)) / \text{sigma} \text{ ** } 2$ の総和になるので、sigma が大きい点ほど「ずれても仕方ない点」として弱く効き、sigma が小さい点ほど強く効く。カウント数 N が十分大きいポワソン統計なら、典型的には $\sqrt{N}$ を誤差の近似として使う。

absolute_sigma=True は、「今渡した sigma は本当に物理的な標準偏差である」と解釈させる指定である。これを False のままにすると、sigma は相対的な重みとしては使われるが、共分散行列の大きさは残差のばらつきに合わせてあとから再スケーリングされる。そのため、推定値の重み付けだけでなく誤差評価まで行いたいなら、sigma の意味を明確にした上で absolute_sigma の設定も意識する必要がある。

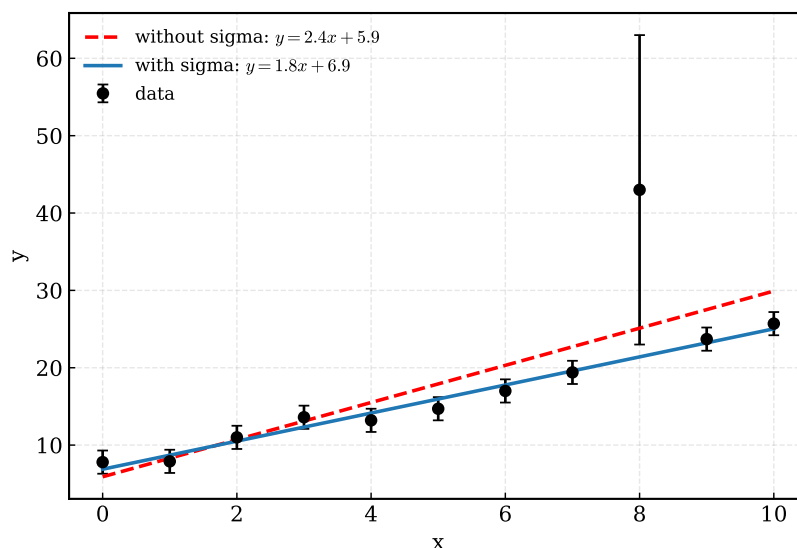


図 9.1 sigma の有無によるフィット結果の激変。中央の点に極端に大きなエラーバー（不確かな値）がある場合、sigma を無視したフィット（赤点線）はこれに引っ張られて真の傾きから大きく外れるが、sigma を指定したフィット（青実線）は不確かな点を無視して正しい傾向を掴んでいる。

図 9.1 から明らかなように、sigma の指定を怠ると結果全体が台無しになる。

第 10 章 Python 解析の高速化

10.1 まず測る

高速化を始める前に、どこが遅いのかを測る。読み込みが遅いのか、探索が遅いのか、図の作成が遅いのかを区別しないまま最適化しても、効果は見えにくい。

ここで役立つのが `time.perf_counter` である。処理の前後で時刻を取り、差を記録するだけでも、どの段階が重いのかの見当が付く。大切なのは、全体時間だけでなく、読み込み、前処理、探索、書き出し、作図のように段階を分けて測ることである。

```
1 import time
2
3 t0 = time.perf_counter()
4 run_analysis()
5 t1 = time.perf_counter()
6 print(f"elapsed={t1-t0:.3f}s")
```

測定するときは、一度だけの結果を絶対視しないことも重要である。キャッシュや OS の状態で多少は揺れるため、条件をそろえ、複数回の傾向を見る方がよい。

10.1.1 Jupyter では %timeit も使える

ノートブック環境で短い式や関数の速度を見たいときは、Jupyter の `%timeit` や `%%timeit` も便利である。複数回の実行を自動で繰り返し、平均的な実行時間を表示してくれるため、小さな差を見たいときに役立つ。

```
1 %%timeit
2 result = array * 2
```

ただし、`%timeit` はノートブックや IPython での簡便な測定であり、スクリプト全体の読み込みや入出力を含めた時間を見るには向かない。小さな部品には `%timeit`、解析全体には `perf_counter` というように使い分けると整理しやすい。

10.2 計算量の見積もり

高速化では、実測だけでなく理屈の見積もりも役に立つ。入力サイズを N としたとき、処理回数が N に比例するのか、 $N \log N$ 程度なのか、 N^2 なのかで、大きなデータに対する伸び方がまるで違う。

厳密な漸近解析までできなくても、「データを 2 倍にしたら計算量は何倍くらいになるか」

を考えるだけで十分に有益である。高速化はテクニック集ではなく、問題の構造を考える作業である。

この点は初学者ほど見落とししやすい。変数名を短くしたり、関数を減らしたりすることは、ほとんど速度に効かない。一方で、比較回数を減らす、探索範囲を絞る、配列演算へ置き換えるといった変更は、本質的に計算量や定数倍を変える。

10.2.1 オーダーとは何か

ここでいう「オーダー」とは、入力サイズ N を大きくしたときに処理回数がどのように増えるかを表す粗い記法である。通常は Big-O 記法で書き、定数倍や低次の項を落として、最終的に支配的になる項だけを見る。

$$3N + 20 \rightarrow O(N), \quad 5N \log N + 100N \rightarrow O(N \log N), \quad N^2 + N \rightarrow O(N^2)$$

よく出るオーダーには、たとえば次のようなものがある。

- $O(\log N)$: 範囲を半分ずつ絞る処理。二分探索など
- $O(N)$: 1 回走査する処理。総和、平均、閾値判定、隣接差分など
- $O(N \log N)$: ソートを含む処理。高速なソート、ソート後の探索など
- $O(N^2)$: 全組合せ比較。全ペアの力づく比較など

たとえば $N = 10^6$ なら、 $\log_2 N$ はおよそ 20、 $N \log_2 N$ はおよそ 2×10^7 、 N^2 は 10^{12} である。どちらも「増える」ことに変わりはないが、その増え方の差は圧倒的である。小さなテストデータでは目立たなくても、大きな入力では急に効いてくる。

ただし、オーダーは定数倍を完全に無視してよい、という意味ではない。同じ $O(N)$ でも 10 倍、100 倍の速度差が付くことは普通にある。ここでいう「ファクター」とは、この定数倍の差のことである。したがって、まずオーダーを疑い、その後で定数倍を詰める、という順序で考えると整理しやすい。

この中で、 $O(\log N)$ や $O(N \log N)$ は初学者には直感を持ちにくい。そこで、まず二分探索で $O(\log N)$ を見てから、ソートで $O(N \log N)$ を見る。

10.2.1.1 二分探索ではなぜ $O(\log N)$ になるか

$\log N$ は、「問題の大きさを毎回半分にできる」場面で現れる。たとえば昇順に並んだ列から目的の値を探す二分探索では、中央を見て、左半分か右半分のどちらかを捨てられる。

```
1 def binary_search(values, target):
2     left = 0
3     right = len(values) - 1
4
5     while left <= right:
6         mid = (left + right) // 2
7         if values[mid] == target:
8             return mid
```



```

9     if values[mid] < target:
10         left = mid + 1
11     else:
12         right = mid - 1
13     return -1

```

要素数が N 個あっても、1 回ごとに候補数が半分になるなら、必要な反復回数は $N \rightarrow N/2 \rightarrow N/4 \rightarrow \dots \rightarrow 1$ という形になる。これが $\log_2 N$ 回程度である。したがって、二分探索は $O(\log N)$ で動く。重要なのは、「1 個ずつ減らす」のではなく「毎回半分に減らす」ことが $\log N$ を生む、という点である。

10.2.1.2 ソートではなぜ $O(N \log N)$ になるか

ソートは、計算量の違いを理解するための基本例である。たとえば、挿入ソートやバブルソートのような初等的な方法は、一般に $O(N^2)$ で動く。一方、マージソートやヒープソートは $O(N \log N)$ であり、Python の `sorted()` も実用的な高速ソートを使っている。

ここで $N \log N$ と書くなら、その意味も説明する必要がある。典型例であるマージソートでは、まず列を半分に分け、さらにその半分へ分ける、という分割を繰り返す。この分割の深さは、およそ $\log_2 N$ 段である。一方、各段では、分かれた小列どうしを併合する過程で、全体として各要素を 1 回ずつ見るので $O(N)$ かかる。

$$\text{各段の仕事量 } O(N) \times \text{段数 } O(\log N) = O(N \log N)$$

ここで重要なのは、「ソートは前処理にすぎない」と軽視しないことである。データを一度並べ替えるだけで、その後の探索や近傍判定がずっと安くなることがある。つまり、 $O(N \log N)$ の前処理へ投資することで、後続の $O(N^2)$ を避けられるなら、全体として大幅に得をする。

データが 10 倍に増えたとき、 $O(N^2)$ のソートはおおよそ 100 倍重くなるが、 $O(N \log N)$ のソートは 10 倍強で済む。この差を理解しておく、単にソートの名前を知るだけでなく、「なぜその方法を選ぶのか」を説明できるようになる。

10.2.2 オーダーが改善する例: 重複判定

問題をはっきりさせよう。長さ N の整数列に、同じ値が 2 回以上現れるかを判定したいとする。

最も素朴な方法は、すべての組 (i, j) を比べることである。

```

1 def has_duplicate_slow(values):
2     for i in range(len(values)):
3         for j in range(i + 1, len(values)):
4             if values[i] == values[j]:
5                 return True
6     return False

```

この方法では、最悪の場合ほぼすべてのペアを比べるので $O(N^2)$ になる。データ量が 10 倍になると、比較回数はおおよそ 100 倍に増える。

しかし、いったんソートしてしまえば、同じ値どうしは必ず隣り合う。したがって、隣接要素だけを見れば重複を判定できる。

```
1 def has_duplicate_fast(values):
2     values_sorted = sorted(values)
3     for i in range(len(values_sorted) - 1):
4         if values_sorted[i] == values_sorted[i + 1]:
5             return True
6     return False
```

`sorted()` の部分は $O(N \log N)$ 、その後の 1 回走査は $O(N)$ なので、全体は $O(N \log N)$ である。元の $O(N^2)$ より明確に良い。ここで本質なのは、Python の書き方が少し変わったことではなく、「全比較」という問題の解き方をやめたことである。

Python では `set(values)` を使えば平均的には $O(N)$ で重複判定できることも多いが、ここでは「ソートすると後続の走査が安くなる」という典型的な発想を示すために、あえてソートを使った例を出している。

同じ考え方は探索でも現れる。未整列の列から目的の値を探す線形探索は $O(N)$ だが、整列済みの列なら二分探索で $O(\log N)$ にできる。したがって、「一度だけ探す」のか「何度も探す」のかによって、先にソートする価値があるかどうかが変わる。前処理に時間を使っても、その後の探索を何度も繰り返すなら、全体としては得になることが多い。

10.2.3 オーダーは同じで定数倍だけ改善する例

今度は、 $N \times N$ 行列 A が対称行列かどうか、すなわち $A_{ij} = A_{ji}$ が成り立つかを調べる問題を考える。

最も素朴な方法は、すべての (i, j) について比較することである。

```
1 def is_symmetric_slow(A):
2     n = len(A)
3     for i in range(n):
4         for j in range(n):
5             if A[i][j] != A[j][i]:
6                 return False
7     return True
```

これは n^2 回程度の比較を行うので $O(N^2)$ である。

しかし、対称性の判定では、上三角と下三角は同じ情報を二重に見ている。したがって、片側だけ見れば十分である。

```
1 def is_symmetric_fast(A):
2     n = len(A)
3     for i in range(n):
4         for j in range(i + 1, n):
5             if A[i][j] != A[j][i]:
```

```
6         return False
7     return True
```

こちらもオーダーは同じく $O(N^2)$ だが、比較回数はおよそ $n(n-1)/2$ 回になり、素朴な実装のほぼ半分で済む。つまり、問題の大きさに対する増え方は変わっていないが、無駄な比較を消したことで定数倍が改善している。

この例は、「同じオーダーでも速くなる」ことを示している。図 10.1 の隣接差分フィルタの比較も、基本的にはこの種類の改善である。比較回数そのものを 2 次から 1 次へ落としたのではなく、同じ種類の処理をより効率よく実装しているのである。

10.3 実装を軽くする

10.3.1 NumPy と pandas による高速化

アルゴリズムを見直した後は、NumPy や pandas へ処理を移すことで、Python レベルのループを減らせる。特に差分計算、ソート、列抽出、フィルタは、これらのライブラリが得意とする。図 10.1 からも、配列演算が速度に与える効果を視覚的に確認できる。

ここで注意したいのは、NumPy や pandas を使えば自動的に何でも速くなるわけではないことである。根本的に無駄な比較をしているなら、配列演算へ移しても限界がある。まず計算量を減らし、そのうえで実装をベクトル化するという順序が自然である。

10.3.2 メモリとコピーの管理

速度だけを見ていると、メモリ使用量が大きくなりすぎて逆に遅くなることがある。巨大な中間配列を何度も作る実装では、CPU よりもメモリ転送が支配的になる場合がある。

そのため、高速化では「何回ループしたか」だけでなく、「何回コピーしたか」も見る必要がある。不要なコピーを減らし、必要な列や範囲だけを扱う設計は、速度と安定性の両方に効く。

10.3.3 分割読込み、並列化、ストリーミング

大規模解析では、全部を一度にメモリへ載せない設計も重要である。ファイル単位で処理し、必要な情報だけを持ち回る。さらに、独立な単位に分割できるなら並列化も検討できる。

ストリーミング処理では、「今必要な範囲だけを覚える」という考え方が中心になる。全履歴が不要なら保持しない、集計済みの量は逐次更新する、といった設計が有効である。メモリ節約と速度向上が同時に得られることも多い。

10.4 プロファイリングと低レベル実装

10.4.1 プロファイラを使う

本格的に遅い理由を調べるなら、プロファイラも役立つ。例えば `cProfile` を使うと、どの関数に時間が使われているかを確認できる。

```
1 import cProfile
2
3 cProfile.run("run_analysis()", sort="cumtime")
```

ただし、プロファイラの数字も文脈なしには読めない。回数が多いから遅いのか、1 回が重いのか、入出力を待っているのかを区別する必要がある。測定値とコード構造を一緒に見ることが重要である。

```
ncalls tottime cumtime function
100000 1.25 1.40 check_pair
      10 0.02 2.10 read_all_files
```

この結果例では、`check_pair` は 1 回は軽い回数が多く、`read_all_files` は内部の処理込みで累積時間が大きいと読める。どの列が何を意味するかを理解すると、改善方針が立てやすくなる。

10.4.2 C や C++ へ書き換えるという選択肢

Python は開発速度が高いが、計算コアのすべてを Python で書く必要はない。アルゴリズムを十分に整理したうえで、どうしても重い部分だけを C や C++ へ移す、というのは自然な選択肢である。

ただし、言語を変える前に Python 側で何が遅いかを把握しておく必要がある。ボトルネックが入出力や高水準の設計にあるなら、C++ 化しても大きな改善にならない。低レベル言語への移行は、最後の切り札として考える方がよい。

```
1 for (std::size_t i = 0; i < n; ++i) {
2     for (std::size_t j = i + 1; j < n; ++j) {
3         if (std::abs(times[j] - times[i]) <= window) {
4             ++count;
5         }
6     }
7 }
```

この書き換えは、定数倍の改善をもたらすことがある。しかし、比較回数そのものが多いすぎるなら本質的な改善ではない。アルゴリズムと実装言語を分けて考える姿勢が必要である。

10.4.3 Python から C/C++ を呼び出す

方法はいくつかある。たとえば `ctypes` で共有ライブラリを呼ぶ方法、Python C 拡張を書く方法、`pybind11` を使って C++ を Python から自然に使える形へ包む方法などがある。さらに、Python と C の中間に位置する Cython を使い、Python に近い構文のまま重いループだけをコンパイルする方法も、この節に含まれる重要な選択肢である。

```
1 cdef int i
2 cdef double total = 0.0
3
4 for i in range(n):
5     total += values[i]
```

ここで `cdef` は Cython 特有の宣言であり、変数を Python オブジェクトではなく C の整数や浮動小数点数として扱わせるために使う。これにより、ループ内部の型判定やボックス化のコストを減らせる。

どの方法を選ぶかは、必要な速度、保守性、ビルドの難しさで決まる。目安としては次のように考えるとよい。

- `ctypes`: 既にある C の共有ライブラリを手早く呼びたいときに向く。設定は比較的軽い
が、引数や戻り値の型を手で合わせる必要がある
- Python C 拡張: Python 本体の C API を直接使う最も低レベルな方法で、自由度は高い
が記述量も多い
- `pybind11`: C++ の関数やクラスを Python から自然に使える形で公開しやすい。C++
側の実装をすでに持っている場合に相性がよい
- Cython: Python に近い構文を保ちながら、重いループや数値計算だけへ型を付けてコン
パイルしたいときに向く

短い試作なら `ctypes`、Python らしい使い勝手まで整えたいなら `pybind11`、既存の Python 風コードを保ちながら内側だけ速くしたいなら Cython、といった選択が考えられる。重要なのは、インタフェースを狭く保ち、重い部分だけを外へ出すことである。

```
1 result = fast_module.count_pairs(times, window)
```

Python 側の呼び出しが単純であれば、内部実装を後から差し替えやすい。まず Python で正しい API を決め、その内部だけを高速化するという順が自然である。なお、Cython も強力だが、アルゴリズムそのものが無駄な場合には決定打にならない。この点は C++ 化と同じであり、まず Python 側で何が遅いのか、どのループが本当に支配的なのかを見極めた後に使う方が効果的である。

10.4.4 外部実装を参考にする時の見方

高速化を学ぶときは、外部の C や C++ の実装例を読むことが参考になる。ただし、そこで大切なのは固有名詞を覚えることではない。まず見るべきなのは、どの問題を解いている

のか、どのアルゴリズムを選んでいるのか、どの部分だけを低レベル言語へ移しているのか、という点である。

高速化の章では、このような例を参照しながら、次の二つを常に区別して考える。

- アルゴリズムを変えたから速くなったのか
- 実装言語を変えたから速くなったのか

多くの場合、最初に効くのは前者である。

したがって、高速化のレポートを書くときは、「NumPy にした」「C++ にした」で終わらせない方がよい。何が変わった結果として速くなったのかを、計算量、比較回数、メモリ配置、ループの位置といった観点で説明すべきである。

10.5 結果例と報告の仕方

高速化の結果は、本文中で比較できる形に整理すると読みやすい。例えば、次のような簡潔な結果でも十分に意味がある。

```
slow version : 182.4 s
vectorized   : 12.7 s
streaming    : 4.8 s
```

この表現だけでは理由が分からないので、続けて「比較回数を減らした」「Python ループを配列演算へ移した」「全件保持をやめた」といった説明を添える。数値と理由の両方がそろって初めて報告になる。

特に、高速化の報告では「 $O(N^2)$ から $O(N \log N)$ や $O(N)$ へ落とした改善」と、「同じ $O(N)$ や $O(N^2)$ のまま NumPy, Cython, C++ などで定数倍を詰めた改善」を分けて書くといい。前者は問題の解き方を変えた改善であり、後者は実装の効率を上げた改善である。この二つを混同しない方が、読者にも自分にも何が効いたのかが伝わりやすい。

```
1 import numpy as np
2
3 values = np.array(data, dtype=np.float64)
4 diffs = np.diff(values)
5 count = np.count_nonzero(diffs > 0.25)
```

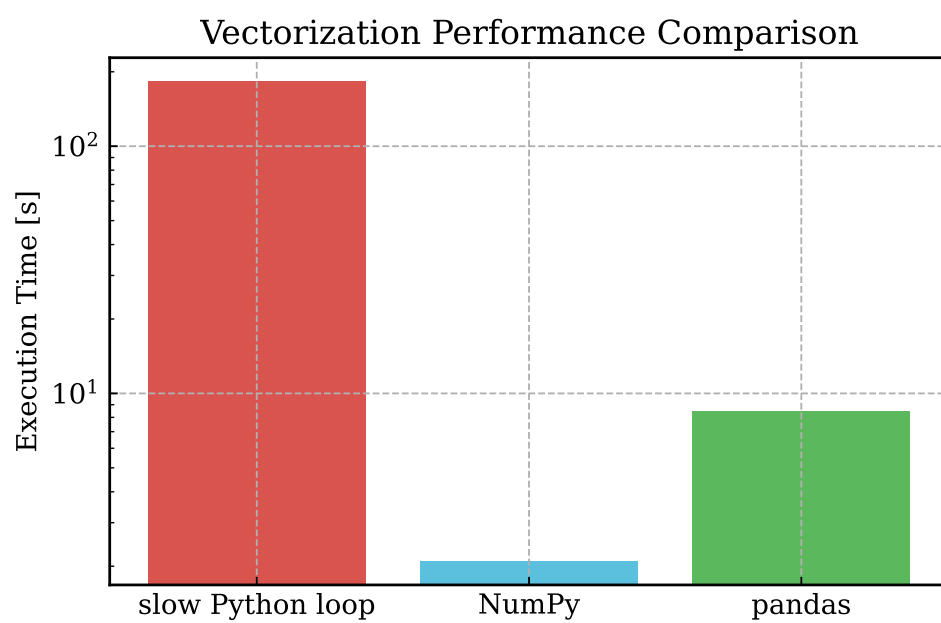


図 10.1 同じ隣接差分フィルタを、純粋な Python ループ、NumPy、pandas で実行した例。縦軸は実行時間で、対数目盛を使っている。

第 11 章 課題: 同時観測イベントの探索

11.1 課題の準備

この課題では、次のファイルを配布する。

- events_data.zip
- slow_event_analysis.py
- 本テキスト

events_data.zip を展開すると、events_data/ というディレクトリが得られる。日ごとに gzip 圧縮されたイベントファイルがあり、年・月ごとのディレクトリに分かれている。

課題のスクリプトやデータは、GitHub リポジトリ (https://github.com/i-komae/python_textbook) から入手できる。手元に配布物がそろっていない場合は、まずリポジトリを取得し、リポジトリ直下にある slow_event_analysis.py と events_data.zip の配置を確認する。events_data.zip を展開すると events_data/ が得られ、これを入力ディレクトリとして使う。

配布物は意図的に最小限にしてある。したがって、何を入力とし、どのファイルが正解判定の対象で、どこからどこまでを自分で実装するのかを最初に整理してから着手した方がよい。

11.1.1 まず配布コードを動かす

まずは、配布された遅いコードをそのまま動かす。

```
$ python3 slow_event_analysis.py events_data --outdir slow_output
```

この実行は環境によってかなり時間がかかる。最初は数日分だけ切り出して試してもよい。

重要なのは、最初から完成版を書こうとしないことである。まず配布コードが本当に遅いことを自分の環境で確認し、どの出力が得られるかを見てから変更を始める方が、修正の意味が分かりやすい。

11.1.2 gzip ファイルの中身を見る

データの中身を見ずに解析を始めてはいけない。まずは 1 ファイルを開き、列の意味を確認する。

```
$ gzip -cd events_data/2024/01/events_2024_01_01.txt.gz | head
```

各行の列は次のようになっている。

```
event_id year month day hhmmss ns detector_id signal
```

列名を見たら、それぞれの型と役割を自分で言葉にしてみるとよい。整数として扱うべき列、時刻を構成する列、識別子としてだけ使う列、物理量として解釈する列が混ざっている。ここを曖昧にしたまま進むと、後でバグの原因になる。

この課題では、2 事象の時刻差を ns 単位まで含めて評価し、 $|\Delta t| \leq 250 \text{ ns}$ を満たすものを「同時観測」と定義する。したがって、hhmmss 列だけを見ればよいわけではなく、ns 列も含めて時刻を一つの量として組み立てる必要がある。秒境界や日付境界をまたぐ場合にどう振る舞うかも、この定義を満たす形で確認しなければならない。

11.1.3 小さいデータで確認する

いきなり 500 日分全体に対して試すのではなく、まずは数日分を別ディレクトリにコピーして試すとよい。

```
$ mkdir -p events_data_small/2024/01
$ cp events_data/2024/01/events_2024_01_01.txt.gz events_data_small/2024/01/
$ cp events_data/2024/01/events_2024_01_02.txt.gz events_data_small/2024/01/
$ cp events_data/2024/01/events_2024_01_03.txt.gz events_data_small/2024/01/
```

小さい入力での確認は、速度のためではなく、正しさを素早く確かめるためである。数十秒で終わる条件を作っておけば、修正してすぐ再実行できる。大きなデータを回すのは、その後で十分である。

```
n_pairs_total = 241
analysis_sec = 18.4
```

この段階では、数値そのものよりも、修正のたびに一貫した結果が出るかが大切である。件数が毎回変わる、出力ファイルの形式が揺れる、といった状態では、その先の高速化に進むべきではない。

11.2 課題で求めること

最低限、次の内容に取り組むこと。

1. 配布コードの処理時間を確認する
2. 同時観測イベント数が正しいかを検証する
3. バグを修正する
4. 高速化する
5. 同時観測イベントの時刻差分布を描き、フィットする

この並びにも意味がある。いきなり高速化から始めるのではなく、まず元のコードの振る舞いを把握し、次に正しさを確認し、その後で改善する。間違ったコードを高速化しても、

間違いが速くなるだけである。

11.2.1 出力としてほしいもの

最低限ほしい出力は次の三つである。

- 同時観測イベントの一覧
- 同時観測イベントを構成する 2 事象の時刻差の分布
- 同時観測イベントどうしの発生間隔、またはそれに相当する量の分布

ここでいう「2 事象の時刻差」は、同時観測と判定した 2 つのイベントの時刻差である。たとえば $\Delta t = t_2 - t_1$ と定義してよい。ただし、符号付きで使うのか絶対値を使うのか、検出器番号で向きをそろえるのかなどは、自分で決めて明記すること。

分布を描くときは、ビン幅、横軸の範囲、単位も明記すること。図は出しただけでは不十分で、どの量をどう定義して数えたのかが分からなければ、他人は解釈できない。フィットを行うなら、どの範囲を使ったかも書くべきである。

```
coincidence_pairs.csv
pair_dt_hist.pdf
pair_mean_gap_hist.pdf
summary.txt
```

ファイル名の流儀を最初に決めておくと、後で解析を繰り返したときにも混乱しにくい。どのファイルが一覧で、どれが図で、どれが数値要約なのかが名前から分かるようにしておくといよい。

11.2.2 正しさ確認の目安

今回の配布データは 500 日分であり、正しく同時観測イベントを拾えていれば、全期間で得られる同時観測イベント数は

```
n_pairs_total = 12033
```

になる。この値は最終確認の目安として使ってよい。

ただし、この値に合ったからといって、それだけで実装が正しいとは限らない。偶然件数だけ一致している可能性もある。可能なら、個々のイベント対の内容や、ファイル境界付近の振る舞いも確認した方がよい。

```
n_pairs_total = 12033
analysis_sec = 248.6
plot_sec = 1.8
```

このような要約ファイルを出しておくと、コード変更のたびに結果を比較しやすい。件数だけでなく処理時間も同じ場所へ残すと、正しさと速度を一緒に追える。

11.3 提出物と発展課題

提出物は次の二つである。

- 解析に使った Python コード
- LaTeX で書いたレポートと、その PDF

レポートには、少なくとも次の内容を書くこと。

- 元のコードの問題点
- 正しさをどう検証したか
- バグをどう修正したか
- どう高速化したか
- 改善前後の処理時間
- 作成した図
- フィット結果とその解釈

レポートを書くときは、変更点の羅列よりも、問題の発見、原因の切り分け、修正方針、検証方法、結果、考察の順で並べると読みやすい。図や表を入れるときは、本文の中で参照しながら説明すること。

1. 元のコードの観察
2. バグの特定
3. 修正内容
4. 高速化方針
5. 結果の比較
6. 図とフィットの考察

この順序で書けば、読者は「何が問題で、どう直して、何が改善したのか」を追いやすい。レポートは成果物であると同時に、思考過程の記録でもある。

11.3.1 発展課題

余裕があれば、図の作成やフィットまで含めた全体処理をさらに高速化してよい。さらに、NumPy や pandas を使うだけでなく、自分でアルゴリズムを設計し直したり、C や C++ で計算コアを実装したりしてもよい。

この発展課題では、特定の解法を要求しない。pandas を深く使う方向もあれば、ストリーミング処理を突き詰める方向もあり得るし、重い部分だけを別言語へ切り出す方向もあり得る。重要なのは、自分の方針を説明できることである。

付録 A SSH, 鍵認証, ファイル転送

A.1 SSH の基本

SSH は、離れたマシンへ安全に接続してコマンドを実行するための仕組みである。研究室や計算サーバーへ入り、大きなデータ処理や長時間ジョブを回すときによく使う。GitHub へ SSH で接続するときにも同じ仕組みを使うが、GitHub はシェルへログインする相手ではなく、Git 操作の認証先である点は区別しておいた方がよい。

A.1.1 サーバーへ接続する

最も基本的な形は次である。

```
$ ssh user@server.example.jp
```

user はサーバー上の自分のユーザー名、server.example.jp はホスト名である。ポート番号が既定の 22 でないなら -p を付ける。

```
$ ssh -p 2222 user@server.example.jp
```

接続後は、ローカルのターミナルと似た形でコマンドを打てる。ただし、そこで見えているファイルは自分の手元ではなく、サーバー側のファイルである。この区別が曖昧になると事故の原因になるので、hostname、whoami、pwd を見て今どこにいるかを確かめる習慣を持った方がよい。

A.1.2 SSH 鍵を作る

毎回パスワードを打つ代わりに、SSH 鍵を使うことが多い。鍵は公開鍵と秘密鍵の 2 つ 1 組であり、公開鍵をサーバーへ登録し、秘密鍵は自分だけが持つ。

```
$ ssh-keygen -t ed25519 -C "your_email@example.com"
```

既定値のまま進めると、多くの場合は ~/.ssh/id_ed25519 が秘密鍵、~/.ssh/id_ed25519.pub が公開鍵になる。 .pub が付いている方だけを相手へ渡す。 .pub が付いていない方は秘密鍵なので、決して共有してはいけない。

```
$ ls -l ~/.ssh
```

秘密鍵の権限が緩いと SSH が拒否することがある。その場合は次のように直す。

```
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/id_ed25519
$ chmod 644 ~/.ssh/id_ed25519.pub
```

A.1.3 公開鍵をサーバーへ登録する

公開鍵をサーバーへ登録する最も簡単な方法は `ssh-copy-id` である。

```
$ ssh-copy-id user@server.example.jp
```

特定の公開鍵を使いたいなら `-i` で指定できる。

```
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server.example.jp
```

`ssh-copy-id` は、指定した公開鍵をサーバー側の `~/.ssh/authorized_keys` へ追加する。つまり、「手元の公開鍵を安全に送り、向こうで追記する」という面倒な作業を代わりにやってくれる。

A.1.3.1 `ssh-copy-id` が使えない場合

`ssh-copy-id` が使えない環境では、公開鍵の中身を手で登録する。

```
$ cat ~/.ssh/id_ed25519.pub
```

表示された 1 行全体を、サーバー側の `~/.ssh/authorized_keys` へ追記する。サーバーへ通常のパスワードログインができるなら、自分でログインして追記してもよい。

A.1.4 `ssh-agent` を使う

パスフレーズ付きの鍵を使う場合は、`ssh-agent` に読み込ませておくと毎回の入力を減らせる。

```
$ eval "$(ssh-agent -s)"
$ ssh-add ~/.ssh/id_ed25519
```

A.2 接続設定と GitHub 連携

A.2.1 `~/.ssh/config` を書く

接続先が増えると、毎回長いホスト名やポート番号や鍵ファイル名を書くのは面倒になる。そこで `~/.ssh/config` を使う。

```
Host labserver
  HostName server.example.jp
  User ikomae
  Port 2222
  IdentityFile ~/.ssh/id_ed25519
```

これを書いておけば、以後は次のように短く書ける。

```
$ ssh labserver
```

config ファイル自体も秘密情報に近いので、`chmod 600 ~/.ssh/config` としておく方が安全である。

A.2.2 GitHub に SSH 鍵を登録する

通常のサーバーとは違って、GitHub には `ssh-copy-id` を使わない。GitHub はシェル接続先ではなく、アカウント設定に公開鍵を登録する相手だからである。

最も確実なのは、GitHub の設定画面の SSH and GPG keys へ公開鍵を登録する方法である。もし GitHub CLI `gh` を使うなら、認証時に SSH 鍵の作成や登録を案内させることもできる。

```
$ gh auth login --git-protocol ssh --web
```

GitHub CLI を使う場合は、既存の鍵を検出してアップロードするか、新しい鍵を作るかを対話的に選べる。GitHub CLI を使わない場合は、`cat ~/.ssh/id_ed25519.pub` で表示した公開鍵を Web 画面へ貼り付ければよい。

A.3 ファイル転送

小さなファイルを 1 回だけ送るなら `scp` でもよいが、解析ではディレクトリ全体を何度も同期することが多い。その場合は `rsync` を使う方が自然である。

A.3.1 scp の最小例

```
$ scp result.csv user@server.example.jp:~/work/  
$ scp user@server.example.jp:~/work/result.csv .
```

1 行目は手元からサーバーへ送り、2 行目はサーバーから手元へ持ってくる。ディレクトリごと送りたい場合は `-r` を付ける。

A.3.2 rsync を推奨する理由

`rsync` は、差分だけを転送し、途中で止まっても再開しやすく、除外パターンも柔軟に書ける。そのため、解析用ディレクトリやコード一式をサーバーへ送る用途では `scp` より向いている。

```
$ rsync -aP ./analysis/ user@server.example.jp:~/work/analysis/
```

よく使うオプションの意味は、次のように 1 つずつ読んだ方が理解しやすい。

- a archive モードである。再帰コピーを行い、時刻や権限などもまとめて保つ。
- P `--partial --progress` の省略形である。途中ファイルを残しつつ進捗を表示する。
- v 何を転送しているか詳しく表示する。
- z 転送時に圧縮する。低速回線では有利だが、既に圧縮済みの大きなデータでは効果が薄いこともある。

--exclude 特定のファイルやディレクトリを転送対象から除外する。

```
$ rsync -aPv \  
--exclude='.venv/' \  
--exclude='__pycache__/' \  
--exclude='build/' \  
--exclude='.mypy_cache/' \  
./analysis/ user@server.example.jp:~/work/analysis/
```

仮想環境、キャッシュ、ビルド成果物はサーバー側で作り直せることが多いので、転送から外した方が軽くなる。

A.3.2.1 末尾のスラッシュに注意する

rsync では、送信元パスの末尾に / を付けるかどうかで意味が変わる。

```
$ rsync -aP analysis user@server:~/work/  
$ rsync -aP analysis/ user@server:~/work/analysis/
```

1 行目は analysis というディレクトリごと送る。2 行目は analysis の中身を送る。この違いは非常に重要であり、意図しない階層を作る原因になりやすい。

付録 B Git と GitHub の基本

B.1 Git の基本概念

Git は、手元のファイルの変更履歴を記録するためのソフトウェアである。これに対して GitHub は、Git リポジトリを置いて共有するためのサービスである。

- Git: ローカルマシンでも単独で使える版管理システム
- GitHub: Git リポジトリをホストし、共有、Issue、Pull Request などを提供するサービス

したがって、`git init`、`git add`、`git commit` は Git の話であり、`git push` で GitHub へ送る段階になって初めて GitHub が関係してくる。

B.1.1 リポジトリと公開範囲

リポジトリは、コード、データ、履歴をまとめて管理する単位である。GitHub へ置く場合は、公開範囲も意識する必要がある。

- `public`: 誰でも見られる
- `private`: 明示的に許可した人だけが見られる
- `internal`: 組織内だけで見られる。主に GitHub Enterprise の文脈で使う

授業用の配布物、研究中のコード、機密データを含むリポジトリでは、公開範囲の設定を誤ると問題になる。GitHub で新しいリポジトリを作るときは、まず `visibility` を確認した方がよい。

B.1.2 Git で何をしているのか

Git は、ファイルを 1 回で全部自動保存してくれるわけではない。ふつうは次の 3 段階で進む。

1. 作業中のファイルを編集する
2. 記録したい変更を `git add` で選ぶ
3. `git commit` で履歴として確定する

この 2 段階目の「いったん選ぶ場所」をステージングエリアという。`git add` と `git commit` の違いを理解するには、ここが重要である。

最初はここが見えにくいので、次の 3 層に分けて考えるとよい。

- working tree: いま自分がエディタで直接書き換えている実ファイル
- staging area (index): 次のコミットへ入れる予定の変更を一時的に集めた場所
- commit history: すでに確定して履歴へ残った状態

たとえば、ファイルを編集した直後は変更は working tree にしかない。git add を実行すると、その時点の内容が staging area へコピーされる。git commit は、その staging area の内容を履歴として確定する操作である。したがって、git add の後にさらにファイルを編集すると、「ステージ済みの内容」と「作業中の内容」がずれることがある。

B.2 ローカルで履歴を作る

B.2.1 最初の設定

最初に、自分の名前とメールアドレスを設定しておく。

```
$ git config --global user.name "Your Name"
$ git config --global user.email "your_email@example.com"
```

これをしておかないと、コミット時に誰の変更なのかが分からなくなる。

B.2.2 新しいリポジトリを作る

まだ Git 管理されていないディレクトリなら、git init で始める。

```
$ mkdir analysis_work
$ cd analysis_work
$ git init
$ git status
```

git status は最重要コマンドである。今どのファイルが未追跡か、どれが変更済みか、何がコミット待ちかを確認できる。迷ったらまず git status を見る、という習慣を付けるとよい。

B.2.3 編集、確認、記録の流れ

日常的な流れは、編集して、差分を確認して、必要なものだけをコミットする、である。

```
$ git status
$ git diff
$ git add analyze.py
$ git diff --cached
$ git commit -m "Add first analysis script"
$ git log --oneline
```

git diff は、まだ add していない変更を見る。git diff --cached は、add 済みで次のコミットへ入る変更を見る。git log --oneline は、これまでのコミット履歴を短く確認する。

B.2.3.1 git status の読み方

git status は、少なくとも次の 3 種類を見分けられるようになるとよい。

- untracked: まだ Git が追跡していない新規ファイル
- modified: 追跡中だが、まだ add していない変更
- changes to be committed: add 済みで、次のコミットへ入る変更

つまり、git status は「いま何がどの段階にあるか」を示す画面である。

B.2.3.2 git add は何をしているのか

git add は「変更を次のコミット候補として選ぶ」コマンドである。保存ではない。したがって、git add した後でも、コミット前にさらに編集すれば、その追加分はまだコミット対象へ入っていない。

```
$ git diff
$ git add analyze.py
$ git diff --cached
```

この 2 回の diff を見分けられるようになると、Git の挙動がかなり分かりやすくなる。

B.2.4 .gitignore を使って除外する

生成物まで毎回コミットすると、履歴が見にくくなる。Python 解析なら、仮想環境、キャッシュ、ビルド成果物などは除外することが多い。

B.2.4.1 .gitignore を書く

.gitignore は、Git に「このパターンのファイルは追跡しなくてよい」と教えるためのファイルである。リポジトリのルートに置くことが多い。

```
.venv/
__pycache__/
*.pyc
build/
dist/
.pytest_cache/
```

このファイルを作ると、まだ追跡されていない .venv/ や build/ は git status に出にくくなり、誤ってコミットしにくくなる。

B.2.4.2 すでに追跡済みなら git rm --cached が必要

.gitignore は「これから追跡しない」ための設定である。すでに一度コミットしたファイルには、そのままでは効かない。

```
$ git rm --cached -r .venv __pycache__ build
$ git commit -m "Stop tracking generated files"
```

`git rm --cached` は、作業ディレクトリの実ファイルを消さずに、インデックスからだけ外す。そのため、「手元には残したまま Git の追跡対象から外す」という目的に向いている。

B.2.4.3 uv プロジェクトで何をコミットするか

uv を使うプロジェクトでは、通常 `.venv/` はコミットしない。一方で、`pyproject.toml`、`uv.lock`、`.gitignore`、そしてプロジェクト単位で版を固定したいなら `.python-version` はコミット候補になる。

B.3 GitHub と接続する

B.3.1 既存のリポジトリを持ってくる

他人のプロジェクトや GitHub 上の教材は、`git clone` で手元へコピーする。

```
$ git clone https://github.com/example/project.git
$ cd project
$ git remote -v
```

`git clone` は、履歴も含めて手元へ持ってくる。`git remote -v` は、どの URL が接続先として設定されているかを確認する。

B.3.2 手元のリポジトリを GitHub へ初めて載せる

ここまでは「GitHub 上にあるものを取得する」話だったが、逆に、手元で `git init` して育てたプロジェクトを自分の GitHub へ載せたい場面も多い。その場合は、まず GitHub 側で空のリポジトリを作り、次に手元のリポジトリから接続先を登録して `push` する。

B.3.2.1 GitHub 側で空のリポジトリを作る

GitHub の Web 画面で `New repository` を開き、リポジトリ名と公開範囲を決める。このとき、手元にすでに履歴があるなら、最初は空のリポジトリとして作る方が簡単である。つまり、`Add a README file`、`Add .gitignore`、`Choose a license` には最初は触れない方がよい。

最初から `README` や `.gitignore` を GitHub 側で作ると、リモート側に 1 個目のコミットができる。その状態で手元の履歴をそのまま `push` すると、履歴の始点が一致しないため、初心者には扱いにくい状況になりやすい。したがって、既存のローカルリポジトリを載せるときは「空の GitHub リポジトリ」を作る、という方針で覚えた方が安全である。

B.3.2.2 README や LICENSE を先に作っていた場合

すでに GitHub 側で `README` や `LICENSE` を作ってしまったなら、リモート側には先にコミットが 1 つ以上存在する。その場合は、いきなり `git push` するのではなく、まずリモート側の履歴を取り込んでから `push` しなければならない。

```
$ git branch -M main
$ git remote add origin git@github.com:owner/project.git
$ git pull origin main --allow-unrelated-histories
$ git push -u origin main
```

--allow-unrelated-histories が必要なのは、手元のリポジトリと GitHub 側のリポジトリが別々に作られており、履歴の出発点が一致していないからである。この pull によって、GitHub 側の README や LICENSE のコミットを自分の履歴へ取り込んでから、その続きとして push できるようになる。

もし手元にも README や LICENSE があり、同じ名前のファイルを両側で作っていたなら、git pull の途中で競合が起こることがある。その場合は、どちらの内容を残すかを自分で決めて修正し、git add と git commit で競合解消を確定してから、あらためて git push -u origin main を実行する。

B.3.2.3 接続先を登録して最初の push を行う

GitHub が表示する URL を確認し、手元のリポジトリで origin として登録する。多くの場合、次の流れで足りる。

```
$ git status
$ git branch --show-current
$ git branch -M main
$ git remote add origin git@github.com:owner/project.git
$ git remote -v
$ git push -u origin main
```

git branch --show-current は、いま自分がどのブランチにいるかを確認する。git branch -M main は、既定ブランチ名を main へそろえるための書き方である。main ではなく既存のブランチ名をそのまま使いたいなら、その名前で push してよい。

git remote add origin ... は、GitHub の URL を origin という別名で登録する。git remote -v を見れば、正しい URL が設定されたかを確認できる。git push -u origin main の -u は、以後このローカルブランチが origin/main を追跡する、という設定である。初回にこれを付けておけば、次回以降は git push や git pull を短く書きやすい。

B.3.2.4 origin がすでにある場合

すでに origin が登録されている状態で git remote add origin ... を実行すると、エラーになる。その場合は、いま何が設定されているかを確認してから、必要なら URL を差し替える。

```
$ git remote -v
$ git remote set-url origin git@github.com:owner/project.git
$ git remote -v
```

教材を別のリポジトリから複製して流用した場合や、HTTPS から SSH へ切り替えたい場

合に、この操作が必要になることがある。

B.3.3 GitHub と同期する

ローカルでコミットしただけでは、まだ GitHub には反映されない。GitHub 側から最新を取り込むのが pull、手元のコミットを送るのが push である。

```
$ git pull --ff-only
$ git push origin main
```

git pull --ff-only は、自分がまだ追加のコミットを持っていないときだけ安全に進める書き方である。履歴が分岐している場合は止まるので、意図しないマージを避けやすい。

B.3.4 GitHub で使う URL には HTTPS と SSH がある

GitHub のリポジトリ URL には、少なくとも HTTPS と SSH の 2 通りがある。

```
https://github.com/owner/repo.git
git@github.com:owner/repo.git
```

HTTPS は見慣れた URL で扱いやすい。一方 SSH は、公開鍵認証を済ませておけばパスワードやトークン入力を減らしやすい。どちらを使ってもよいが、混在させると混乱しやすいので、最初に方針を決めた方がよい。

B.4 認証と最小ワークフロー

B.4.1 GitHub の認証: PAT と SSH 鍵

GitHub では、HTTPS 経由の Git 操作で通常のアカウントパスワードは使えない。現在は、個人用アクセストークン (PAT) を使うか、SSH 鍵を登録して使うのが基本である。

B.4.1.1 PAT を使う

HTTPS を使う場合、プライベートリポジトリへの push や pull では PAT を使う。

1. GitHub の Settings を開く
2. Developer settings から Personal access tokens へ進む
3. fine-grained token か classic token を作る
4. 対象リポジトリと必要最小限の権限を選ぶ
5. 発行したトークンを安全に保管する

認証時には、ユーザー名は GitHub のアカウント名、パスワード欄には通常のパスワードではなく PAT を入れる。組織アカウントでは、SAML SSO の承認が別に必要なこともある。その場合は、発行したトークンをその組織用に追加承認しないと使えない。

fine-grained token は、どのリポジトリへ、どの操作権限だけを与えるかを細かく絞れ

る新しい方式である。個人の通常利用では、まずこちらを選ぶ方が安全である。これに対して classic token は古い方式で、広い権限をまとめて与える形になりやすい。古いツールや特殊な用途ではまだ必要になることがあるが、理由がなければ fine-grained の方を優先した方がよい。

SAML SSO は、組織アカウントで「そのトークンが本当に組織のリソースへアクセスしてよいか」を追加で承認する仕組みである。つまり、トークンを作っただけでは終わらず、対象組織に対して明示的に利用許可を与えないと push や pull に失敗することがある。個人リポジトリしか使わないなら通常は意識しなくてよいが、大学や研究所の GitHub 組織では重要になる。

B.4.1.2 SSH 鍵を使う

SSH を使う場合は、付録 A で作った公開鍵を GitHub の SSH and GPG keys へ登録する。GitHub CLI `gh auth login --git-protocol ssh --web` を使うと、その作業を対話的に進めやすい。

B.4.2 初心者が押さえるべき最小ワークフロー

最初のうちは、次の順序だけ守れば十分である。

1. `git status` で状況を見る
2. `git diff` で変更内容を読む
3. `git add` で記録したいファイルを選ぶ
4. `git diff --cached` でコミット内容を再確認する
5. `git commit -m "..."` で記録する
6. GitHub とやり取りするなら `git pull --ff-only` と `git push` を使う

この流れを身に付ければ、教材の取得、uv プロジェクトの再現、共同研究でのコード共有まで一通り対応できる。最初からブランチ運用や複雑な履歴操作まで覚える必要はないが、`status` と `diff` を読まずに `add` や `commit` をする癖だけは避けた方がよい。

付録 C 正規表現の基本

正規表現は、文字列の集合を短く記述するための書き方である。grep、awk、Python の re モジュールなどで共通して使われ、ログの抽出、ファイル名の検査、検出器 ID の選別、時刻文字列の確認といった場面で役に立つ。ここでは、本文で触れた grep や awk の延長として、解析で本当に必要になる範囲をまとめる。

C.1 正規表現とは何か

C.1.1 文字どおりの一致とメタ文字

最初の一步は、普通の文字列検索と正規表現の違いを区別することである。det_A という文字列だけを探したいなら、文字をそのまま並べればよい。

```
$ grep "det_A" events.txt
```

これに対して、. や * のような特別な意味を持つ記号を使うと、「どの文字でもよい」「0 回以上くり返す」といった条件を書けるようになる。こうした特別な意味を持つ文字をメタ文字という。

```
.  
*  
+  
?  
^  
$  
[]  
()  
|
```

たとえば det_. は det_A にも det_B にも一致する。. は「任意の 1 文字」を表すからである。逆に、ピリオドそのものを探したいなら、前にバックスラッシュを付けて \. のようにエスケープしなければならない。これは + や ? でも同じである。

C.1.2 ワイルドカードとの違い

シェルのワイルドカードと正規表現は似て見えるが、同じものではない。ls *.txt の * は「任意の文字列」を表すが、正規表現の * は「直前の要素の 0 回以上のくり返し」を表す。したがって、正規表現で「任意の文字列」を書きたいときは .* と書く。

この違いを混同すると、grep "*.txt" のように書いて期待通りに動かない。シェルが展開する記法なのか、正規表現エンジンが読む記法なのかを意識することが重要である。

表 C.1 ワイルドカードと正規表現の違い

場面	記法	意味
シェル	*.txt	任意の名前で .txt で終わるファイル
正規表現	.*\.txt\$	任意の文字列で始まり .txt で終わる文字列

C.2 よく使う部品

C.2.1 アンカー（行頭と行末）

^ は行頭、\$ は行末を表す。したがって、文字列の一部ではなく「行全体がその形である」ことを確かめたいときに使う。

```
$ grep '^ERROR' run.log
$ grep 'txt$' filelist.txt
$ grep '^det_A$' detectors.txt
```

1 行目は ERROR で始まる行、2 行目は txt で終わる行、3 行目は行全体がちょうど det_A である行だけを取る。特に 3 行目のように両端をアンカーで挟むと、det_AB や xdet_A が紛れ込むを防げる。

C.2.2 文字クラス [...] と代表的な省略記法

[abc] は a か b か c の 1 文字、[A-Z] は英大文字 1 文字を表す。検出器名やセンサー番号のように、限られた候補から 1 文字だけ変わる文字列を扱うときに便利である。

```
^det_[AB]$
^run_[0-9][0-9][0-9]$
^[A-Za-z_][A-Za-z0-9_]*$
```

1 行目は det_A または det_B、2 行目は run_001 のような 3 桁番号付きの名前、3 行目は Python 風の識別子に近い形を表す。範囲指定の [0-9] は非常によく使う。

Python の re では、数字 1 文字を \d、空白 1 文字を \s と書ける。

```
1 import re
2
3 re.fullmatch(r"det_[AB]", "det_A")
4 re.fullmatch(r"\d\d:\d\d:\d\d", "12:34:56")
5 re.search(r"\s+", "det_A_0015.2")
```

ただし、grep や awk では \d や \s がそのまま使えないことがある。コマンドラインでは [0-9] や [:space:] を使う方が移植性が高い。

C.2.3 量指定子 *, +, ?

量指定子は、直前の要素が何回くり返されるかを表す。* は 0 回以上、+ は 1 回以上、? は 0 回または 1 回である。

```
ab*c
ab+c
colou?r
```

ab*c は ac、abc、abbbc に一致する。ab+c は少なくとも 1 個の b が必要なので ac には一致しない。colou?r は color と colour の両方に一致する。

解析では、「空白が 1 個以上続く」「符号が付くかもしれない」「小数点以下があるかもしれない」といった曖昧さを扱うときに量指定子が役立つ。

```
1 pattern = re.compile(r"^[+-]?[d+](\.[d+])?$")
2
3 for text in ["12", "-3.5", "+0.25", "1.2.3"]:
4     print(text, bool(pattern.fullmatch(text)))
```

このパターンは、符号付き整数や小数を表し、1.2.3 のような壊れた文字列は弾く。

C.3 物理・データ解析での具体例

C.3.1 ログから必要な行だけを探す

実験ログやジョブログでは、まず異常行や特定の run だけを抜き出したい。アンカーと文字クラスを組み合わせると、かなり具体的な条件を書ける。

```
$ grep '^ERROR' run.log
$ grep '^2026-04-04 .* det_[AB] ' run.log
$ grep -E '^(WARN|ERROR)' run.log
```

3 行目では grep -E を使って | による選択を書いている。拡張正規表現を使うと括弧や + を素直に書けるので、条件が少し複雑になったらこちらの方が読みやすい。

C.3.2 検出器 ID やセンサー名を選別する

検出器名が det_A, det_B, det_C のように並んでいるなら、文字クラスや選択を使って対象を絞れる。

```
$ grep '^det_[AB]$_' detector_ids.txt
$ awk '$1 ~ /^det_[AB]$/ {print $1, $3}' summary.txt
```

awk では \$1 ~ /.../ の形で、1 列目がその正規表現に一致するかどうかを判定できる。列を見ながら条件を書くときは grep より awk の方が自然なことが多い。

C.3.3 Python で抽出と変換を同時に行う

正規表現は「一致したかどうか」だけでなく、「どの部分が一致したか」を取り出すのにも向いている。Python では `re.search` や `re.fullmatch` の戻り値から、必要部分を取り出せる。

```
1 import re
2
3 line = "2026-04-04_12:34:56_det_A_signal=15.2"
4 match = re.search(r"signal=(\d+\.\d+)", line)
5
6 if match:
7     signal = float(match.group(1))
8     print(signal)
```

括弧 (...) で囲んだ部分はキャプチャされ、`group(1)`、`group(2)` のように取り出せる。ログ 1 行から run 番号、検出器名、信号値を抜き出して数値へ変換するような処理では、この形が便利である。

C.4 Python の re と grep/awk の使い分け

C.4.1 Python では raw string を使う

Python の文字列中で \ を多用すると読みにくくなる。そのため、正規表現を書くときは `r"..."` のような raw string を使うのが基本である。

```
1 import re
2
3 pattern = re.compile(r"^det_[A-Z]\d+$")
4 print(bool(pattern.fullmatch("det_A12")))
```

raw string を使わないと、`"\\d+"` のようにバックスラッシュを二重に書かなければならない。正規表現の意図が見えにくくなるので、特別な事情がなければ raw string を選ぶ方がよい。

C.4.2 search, match, fullmatch を使い分ける

`re.search` は文字列のどこかに一致があればよいとき、`re.match` は先頭から一致していればよいとき、`re.fullmatch` は文字列全体がその形かを確認したいときに使う。ファイル名や識別子を検証するときは、意図せず余分な文字が混ざらないよう `fullmatch` を使う方が安全である。

```
1 import re
2
3 print(re.search(r"\d+", "run_42").group())
4 print(bool(re.match(r"run", "run_42")))
```

```
5 print(bool(re.fullmatch(r"run_\d+", "run_42")))
```

C.4.3 コマンドラインで十分な場面と、Python に移すべき場面

単に条件に合う行を見つけない、件数を数えたい、特定列だけを抜きたい、といった用途なら `grep` や `awk` が速い。逆に、一致した部分を数値へ変換して次の計算へ渡したい、複数の条件分岐を組み合わせたい、抽出結果を `DataFrame` にしたい、といった場合は Python へ移した方が自然である。

- `grep`: 行全体の抽出や件数確認をすばやく行いたいとき
- `awk`: 列条件と正規表現を組み合わせたいとき
- Python `re`: 抽出した結果をそのまま解析処理へつなげたいとき

正規表現は強力だが、複雑な 1 行を無理に作ると自分でも読めなくなる。まずはアンカー、文字クラス、量指定子の 3 つで書けないかを考え、それでも足りないときに括弧や選択を足す方が安全である。解析コードと同じく、「まず読めること」を優先した方が長く使える。